

Sharif University of Technology
Department of Computer Engineering

B.Sc. Project Report

A Speech-to-Speech Large Language Model for Persian

By:

Mehran Bakhtiari

Supervisor:

Dr. Hossein Sameti

October 2026

English edition of the Persian report

Acknowledgements

I am sincerely grateful to the supervisor of this project, Dr. Hossein Sameti, for accepting to guide this work, for his precise questions about the boundary between engineering evidence and scientific claims, and for insisting that negative results be reported as honestly and completely as positive ones. Much of the final shape of this report, including the clear separation of automatic measures from human evaluation, is the result of that guidance.

I also thank the Department of Computer Engineering at Sharif University of Technology for providing the educational and research setting of this project.

The computationally heavy parts of the project, including the low-rank adaptation of the direct model and the speech recognition and speaker diarization services, ran on institutional GPU infrastructure; I thank everyone who made access to these resources and to the Persian speech archive possible. The internal research use of these resources was recorded with the supervisor's approval and does not imply permission to redistribute the raw audio or captions.

Finally, I thank my family for their support and patience during the months of this project.

Abstract

This project designs, implements and evaluates a prototype Persian speech-to-speech system that operates in full-duplex mode: the user microphone stays open while the assistant speaks, and the system reacts to interruptions. Two tracks were pursued: low-rank adaptation of the end-to-end Moshika model to Persian, and a streaming cascade of NeMo speech recognition, the Qwen3-4B-Instruct-2507 responder and Piper speech synthesis. A conversational corpus of 6,754 user/response pairs (108.584 hours, session-disjoint splits) was derived from an archive of Persian spoken content.

The final cascade detects the end of speech automatically and speculatively, streams the reply clause by clause, and handles interruptions in two stages: it ducks the assistant after one frame of speech louder than the expected residual echo, then confirms lexically with the speech recognizer after discarding listener tokens and the assistant's own echoed words. In automated real-browser loopback, end-of-speech to first assistant audio had a mean of 372 ms and a 90th percentile of 431 ms, with every turn under 500 ms and the GPU side of the stack using 11.1 GiB. Interruption handling reached accuracy 0.933 and F1 0.880, with a median of 120 ms to the duck, in a deployment simulation built from real recordings, and accuracy 0.825 (F1 0.817) in the browser; an energy/F0/MFCC classifier alone reached only 0.574 on real podcast events. The responder passed all seven predeclared automatic gates of a 40-row final test (relevance 3.150 versus 1.425). All components were re-selected on fresh panels, including speech recognition on human transcripts and synthesis with two automatic listeners. A pre-registered comparison of 15 responder models and their quantized builds, judged by two models from families with no candidate, selected Gemma 4 12B at four bits for a 24 GB card (1.2 to 1.75 points of 10 above the best 4B model, 39 ms to the first token); in the live stack it met the memory and latency limits but fell short of the coherence rule after late interruptions, so it remains an opt-in preset. For a 12 GB card, post-hoc checks of the same model (including a llama.cpp setting that cut its first token from about 180 to 25 ms) failed the same coherence rule.

The direct track produced no deployable model. Moshi's English tokenizer needs 6.6 tokens per Persian word and exceeds its one-token-per-frame budget; phoneme romanization doubled the best runtime result but the speech was still not intelligible, and a speech-input Gemma 4 model also underperformed the cascade. Latency and interruption evidence is automated and on an H100 (the memory of a 12 GB card was respected, its compute speed was not), labels are automatic, and the human listening study was not conducted for lack of participants.

Keywords: Speech-to-speech, full-duplex dialogue, barge-in detection, Persian language, large language models, streaming cascade, tokenization, parameter-efficient adaptation

Contents

1 Introduction **1**

1.1 Problem Definition 1

1.2 Motivation 2

1.3 Defined Requirements 3

1.4 Research Questions 3

1.5 Main Challenges 4

1.6 Two-Track Strategy 5

1.7 Contributions 5

1.8 Structure of This Report 6

2 Preliminaries **8**

2.1 The Speech Signal and Acoustic Features 8

2.1.1 Energy and Zero-Crossing Rate 8

2.1.2 Fundamental Frequency and the YIN Method 9

2.1.3 Mel-Frequency Cepstral Coefficients 9

2.1.4 Context Window and Decision Vector 10

2.2 Speech Recognition 10

2.3 Text-to-Speech 11

2.4 Large Language Models and Low-Rank Adaptation 11

2.4.1 Autoregressive Language Models 11

2.4.2 Teacher Forcing and Exposure Bias 11

2.4.3 Parameter-Efficient Adaptation and LoRA 12

2.5 Neural Audio Codecs and Speech-to-Speech Models . . . 12
2.5.1 Residual Vector Quantization . . . 12
2.5.2 Language Modelling on Audio Tokens . . . 13
2.5.3 The Moshi Architecture and the “Inner Monologue” . . . 13
2.6 Full Duplex and Turn-Taking . . . 14
2.7 Classification with Gradient Boosting . . . 14
2.8 Evaluation Measures and Statistical Concepts . . . 15
2.8.1 Word and Character Error Rates . . . 15
2.8.2 Detector Measures . . . 15
2.8.3 Confidence Intervals and Group Correlation . . . 15
2.8.4 Group Splits of the Data . . . 16
2.8.5 Automatic Judging with a Language Model . . . 16
2.8.6 Latency Measures . . . 16

3 Related Work **18**
3.1 From Cascades to End-to-End Models . . . 18
3.2 Open Speech-to-Speech Models . . . 19
3.2.1 The Mini-Omni Family . . . 19
3.2.2 SpeechGPT and Discrete Speech Units . . . 19
3.2.3 The LLaMA-Omni Family . . . 19
3.2.4 Freeze-Omni and the Frozen-Model Approach . . . 20
3.2.5 Qwen2.5-Omni and the “Thinker–Talker” Architecture . . . 20
3.2.6 Moshi and the Final Choice of the Direct Track . . . 20
3.2.7 PersonaPlex and Role Control in a Full-Duplex Model . . . 21
3.3 Persian Speech Resources . . . 21
3.4 Low-Latency Streaming Inference . . . 22
3.5 Older Adults and Voice Assistants . . . 23
3.6 Interruption Detection and Turn-Taking . . . 23
3.7 Automatic Evaluation of Dialogue Quality . . . 24

3.8 Position of This Project 25

4 Proposed Method and Implementation 26

4.1 Early Attempt and Revision of the Architecture 26

4.2 Construction of the Persian Conversational Dataset 27

4.2.1 Source and Strategy 27

4.2.2 Pipeline Stages 27

4.2.3 Response-Pair Extraction and Interaction Labels 29

4.2.4 Group Splits and Re-Synthesis of the Response Voice 29

4.3 Architecture of the Streaming Cascade 30

4.3.1 Browser Client and Frame Format 30

4.3.2 End-of-Speech Detection and Speculative Start 31

4.3.3 Speech Recognition 31

4.3.4 Streaming Text Reply Generation 31

4.3.5 Clause Chunking and Speech Generation 32

4.4 Full-Duplex Control and Interruption Detection 32

4.4.1 Stage One: Ducking 32

4.4.2 Stage Two: Lexical Confirmation with Echo Rejection 33

4.4.3 Three Levels of Evaluation 34

4.5 Design of the Direct-Track Experiments 34

4.5.1 Training Objective and Acceptance Condition 34

4.5.2 Experimental Generations 35

4.5.3 Version Seven: Testing the Root Cause in the Tokenizer 36

4.5.4 The Speech-Input Track: Gemma 4 36

4.6 Evidence-First Method 37

4.7 Implementation and Reproducibility 38

5 Experiments and Results 40

5.1 Coverage of Acceptance Conditions and Evidence Status 40

5.2 Dataset . . . 40
5.3 Cascade System: Mechanical Panel . . . 42
5.4 Automatic Approximation of the Intelligibility of Generated Speech . . . 44
5.5 Automatic Semantic Test . . . 45
5.5.1 Qualification Stage . . . 45
5.5.2 Final Test . . . 45
5.5.3 Responder Adaptation with LoRA . . . 46
5.6 Re-Selection of Components on Fresh Panels . . . 47
5.6.1 Speech Recognition on Human Transcripts . . . 48
5.6.2 Responder with a Panel of Elderly Requests . . . 50
5.6.3 Responder Comparison Version Three: Quantized Builds and Two Memory Profiles . . . 52
5.6.4 Voice with Two Automatic Listeners . . . 56
5.7 Interruption Detection . . . 57
5.7.1 Synthetic Test . . . 57
5.7.2 Approximation on Recorded Data . . . 58
5.7.3 Deployment Simulation . . . 59
5.8 Continuity of the Full-Duplex Transport . . . 60
5.9 Direct Track: Seven Experimental Generations . . . 60
5.9.1 Overall Result . . . 60
5.9.2 Version One: Positive Held-Out Loss, Failed Runtime . . . 61
5.9.3 Version Two: The Clearest View of the Gap . . . 62
5.9.4 Versions Three to Five: Testing Targeted Hypotheses . . . 63
5.9.5 Version Six: Separating Capacity from Generation . . . 64
5.9.6 Version Seven: Removing the Tokenizer Bottleneck . . . 65
5.10 The Speech-Input Track . . . 66
5.11 Latency, Memory and Browser Loopback . . . 66
5.12 Error Analysis . . . 69
5.13 Human Study . . . 69

5.14 The Latency, Size and Quality Trade-off . . . 70
5.15 Final Status of the Requirements . . . 71

6 Discussion and Limitations **75**
6.1 The Gap Between Reducing Loss and Valid Generation . . . 75
6.2 Why the Word Error Rate Is Much Higher Than the Character Error Rate . . . 77
6.3 Why Adapting the Responder Lowered the Loss but Did Not Improve Quality . 77
6.4 A Structural Conflict Between the Requirements . . . 78
6.5 The Value and Boundary of the Automatic Semantic Result . . . 79
6.6 Limitations . . . 79
6.7 Ethical, Privacy and Legal Considerations . . . 80
6.7.1 Provenance and Rights of the Data . . . 80
6.7.2 Privacy in the Human Study . . . 81
6.7.3 Licence and Its Boundaries . . . 81
6.7.4 Use and Misuse . . . 81
6.8 Statements This Report Avoids . . . 81

7 Conclusions and Future Work **83**
7.1 Summary . . . 83
7.2 Answer to the Research Question . . . 84
7.3 Answers to the Research Questions of the Proposal . . . 85
7.4 Future Work . . . 86
7.4.1 Completing the Evidence . . . 86
7.4.2 Improving the System . . . 87
7.4.3 The Direct Track . . . 87
7.5 Closing Remarks . . . 88

References **89**

A Implementation and Reproduction Details **95**

A.1 Repository Layout 95
A.2 Hyperparameters of the Direct-Track Experiments 95
A.3 Hyperparameters of the Detector and the Responder 95
A.4 Reproduction Commands 96
A.5 Map of Evidence Artifacts 96
A.6 Runtime Environment 97
A.7 Building This Report 97

List of Tables

1.1 The seven requirements of the project definition . . . 3

3.1 This project compared with prior work on the capabilities required by the project definition . . . 25

4.1 Design of the seven experimental generations of the direct track and the hypothesis each generation tested . . . 35

4.2 Evidence classes and the claim power of each class . . . 38

5.1 The project's official acceptance conditions and the automatic evidence of the final stage . . . 41

5.2 Characteristics of the derived Persian conversational dataset and its automatic audit . . . 42

5.3 Result of the fixed validation panel of the cascade . . . 43

5.4 Measurements of each row of the cascade validation panel . . . 44

5.5 Result of the automatic re-recognition approximation of the generated speech . . . 46

5.6 Result of the final automatic semantic test on 40 group-disjoint rows . . . 47

5.7 Predefined automatic conditions of the final test and the observed values . . . 48

5.8 Comparison of speech recognition candidates on 344 PSRB clips with human transcripts . . . 49

5.9 Candidate responders under two judges (scale 0–4; gain relative to the deployed system) . . . 51

5.10 Larger responders under a 22 GiB memory limit (120 written requests and turns; scale 0–4; gain relative to the best 4B, Qwen3.5-4B) . . . 52

5.11 Finalists of responder comparison version three on the test panel (252 rows; scale 0–10; latency on the shared H100 at the deployment configuration) 53

5.12 Automatic intelligibility of the Persian Piper voices (mean character error rate of two listeners) 56

5.13 Result of the interruption detector approximation on recorded audio events with held-out sessions 58

5.14 Development runs of the simulation (training split only) 59

5.15 One-time result of the simulation on 600 test scenarios (95% Wilson intervals) 59

5.16 Summary of the seven experimental generations of the direct track. The "best run" column is the largest number of the nine rows of the runtime panel passed among all checkpoints of that generation. 61

5.17 A sample of version-two checkpoints: the validation loss decreases monotonically while the number of rows passing the runtime panel does not increase . . 62

5.18 Result of each checkpoint of versions four and five on the nine-row runtime panel 63

5.19 Version v6.2: a positive in-sample learning signal against a negative autoregressive runtime result 64

5.20 Number of accepted rows of the nine-row panel in the in-sample capacity test . 65

5.21 The speech-input track against the cascade (Qwen3.8-27B judge) 66

5.22 Latency from end of speech to the assistant's first audio in the browser loopback test (ms). Main panel: 61 PSRB utterances and 140 corpus utterances; elderly: the same method with speed 0.85 and 3 dB quieter; barge-in: the base turns of the two interruption panels. The last column adds the browser's reported output latency. 67

5.23 Failure modes of the system and the measured cause of each 72

5.24 The size, quality and latency trade-off of the candidates (quality and latency from the tests of the previous sections) 73

5.25 Final status of the seven requirements of the project definition and the class of the supporting evidence 74

A.1 Top-level layout of the project's code repository 96

A.2 Hyperparameters of the seven generations of low-rank adaptation in the direct track 97

A.3 Configuration of the interruption detection module and the text responder adapter 99

A.4 Reproduction commands of the project's main stages 100

A.5 Locked evaluation scripts 100

A.6 Map of claims and reference artifacts. All paths are relative to the `results` directory. 101

A.7 Main components of the runtime environment and the models used 102

List of Figures

4.1 Stages of the Persian conversational dataset pipeline. Each stage is a separate command with an auditable output. 28

4.2 Gradual reduction of the amount of audio from the channels' caption inventory to the audited stereo training export. The light band shows the 100–200 hours targeted by the project definition. 30

4.3 The streaming architecture of the delivered system. The bottom row is the reply generation chain while listening and the top row is interruption control while the assistant speaks. The dashed arrow shows the transfer of the start of the user's speech to the next turn. 30

5.1 (a) Full reply generation time against input transcript length for the nine rows of the validation panel, with the fitted line and the descriptive Pearson correlation. (b) Word and character error rates of re-recognition for the same nine outputs, with the micro-aggregated values. 45

5.2 (a) Mean relevance and coherence scores of the three blind arms, with the predefined thresholds. (b) Full distribution of the relevance score over the 80 judge calls of each arm. 48

5.3 (a) Search for the decision threshold on the validation sessions; the vertical line is the selected threshold of 0.70. (b) Comparison of the proposed detector with the energy baseline on the test sessions; the error bars are session-level block confidence intervals. 58

5.4 (a) Best offline loss of each generation. (b) Largest number of rows passing the nine-row runtime panel in each generation, against the acceptance condition of 9 of 9. 61

5.5 (a) Teacher-forced training loss of version one, with a rolling mean. (b) Re-evaluated validation loss by text and audio component; the vertical line is the selected checkpoint. 62

5.6 Version two: the validation loss on 202 segments decreases monotonically over the checkpoints (line), while the number of rows passing the nine-row runtime panel (bars) never reaches the acceptance condition. 63

5.7 Version v6.2: (a) the in-sample loss decreases steadily, and the text loss falls by 32.20% from step 50 to the best later step. (b) Nevertheless all four checkpoints pass 0 of the 9 rows of the runtime panel. 64

Chapter 1

Introduction

Spoken conversation is the most natural form of human–machine interaction. Nevertheless, until recently spoken dialogue systems were almost always built from a chain of independent components: voice activity detection, speech recognition, a text dialogue engine, and then text-to-speech. The introduction of models such as Moshi [1] and Freeze-Omni [2] showed that spoken dialogue can be modelled as a single "speech-to-speech" problem with one integrated model: a model that listens and speaks at the same time and needs no explicit speech segmentation for turn-taking.

This shift has happened for high-resource languages such as English and Chinese. For Persian, none of the open speech-to-speech models offers official support, and Persian conversational datasets that cover overlapping speech, interruptions and backchannels are not publicly available either. The practical question of this project arises from this gap: with the computing and time budget of a bachelor's project, how far can one go in building a full-duplex Persian speech-to-speech system, and, more importantly, what can be "proven" about it with auditable evidence?

1.1 Problem Definition

The defined goal of this project is to design, implement and evaluate a prototype speech-to-speech large language model for Persian. The system must work in *full-duplex* mode: unlike half-duplex systems, which listen to the user only after the reply has finished, the user's microphone must stay active while the reply plays, and the system must be able to react if the user *interrupts*.

Three properties separate this problem from a conventional spoken question answering system:

- **Listening and speaking at the same time.** While playing its own audio output, the system must also process the microphone input stream and decide whether to continue or stop playback.
- **Time-critical decisions.** The decision to stop playback must be made within a few tens of milliseconds, otherwise the user feels that the system does not hear them.
- **Continuity of interrupted speech.** What the user says at the moment of the interruption must not be lost; it must be carried to the next turn, otherwise the user has to repeat the sentence.

1.2 Motivation

Addressing this problem for Persian matters for several reasons.

First, **accessibility**. A speech interface can be the only effective way to use digital services for users who are not comfortable with a Persian keyboard, for users with low vision, and in particular for older adults. In the natural conversation of older adults, longer pauses and mid-sentence corrections are more frequent; a half-duplex system that does not listen until the end of its own reply creates an irritating experience in these conditions.

Second, **Persian is low-resource in this particular area**. Persian speech recognition and text-to-speech datasets are available; a prominent example is ManaTTS [3], with more than 86 hours of single-speaker speech. But these resources are single-speaker and read, and the interactive phenomena that a full-duplex model has to learn (overlap, interruptions and backchannels) do not occur in them.

Third, **computational cost**. Current speech-to-speech models, such as Qwen2.5-Omni [4], need far more memory to run than a laboratory graphics card. Showing what is "achievable" and what is "not achievable" within a memory range of 12–24 GB is itself a useful engineering result.

1.3 Defined Requirements

The project definition sets seven specific requirements. They are used throughout this report as the yardstick of achievement and are listed in Table 1.1. For each requirement, the results chapter (Chapter 5) reports the final status and the class of the supporting evidence.

Table 1.1: The seven requirements of the project definition

ID	Requirement	Specified measure
R1	Persian speech-to-speech prototype	Persian spoken reply generated from Persian spoken input
R2	Full-duplex operation	Listening during playback and reacting to interruptions
R3	End-to-end latency	At most 500 ms from receiving speech to producing the spoken reply
R4	Persian conversational dataset	100–200 hours of speech with noise, overlap and interruptions
R5	Interruption detection module	Accuracy above 80% with energy, pitch and MFCC features
R6	Evaluation on target hardware	A GPU with 12–24 GB of memory
R7	Human evaluation	5–10 Persian speakers, at least two aged over 60

The expected final outputs comprise four items: a working prototype, documented source code, a Persian conversational dataset, and findings on how older Persian-speaking adults interact with the system. For the fourth item, the measurement protocol and infrastructure are ready; its empirical finding can only be claimed once the human study is complete, so this report makes no claim about the experience of older adults.

1.4 Research Questions

The project proposal poses four research questions, each of which Chapter 7 (Section 7.3) answers with the evidence of this report:

1. **Question 1.** How can an existing English or Chinese speech-to-speech model be adapted to Persian with the least retraining cost?

2. **Question 2.** Which architectural changes, such as a dual buffer or an interruption detection module, are needed for full-duplex operation with a latency below 500 ms?

3. **Question 3.** What is the trade-off between latency, model size and speech quality for Persian in real-time operation?

4. **Question 4.** How do older Persian-speaking adults perceive the responsiveness and naturalness of such a system?

1.5 Main Challenges

Over the course of the project, four decisive challenges appeared that changed the final shape of the work.

No official training path for Persian. Mini-Omni2 [5], which the project definition names as an example base model, and LLaMA-Omni2 [6] have published repositories for inference but provide no complete, official speech-to-speech trainer. Among the models reviewed, only the Moshi family had published an official low-rank adaptation trainer [7]; for this reason the direct track of this project was built on Moshika, the 7-billion-parameter version of that family.

No labelled Persian conversational data. No public Persian corpus with interruption and overlap labels was available. The dataset of this project therefore had to be built from an internal archive of Persian spoken content with an automatic pipeline of window reconstruction, speaker diarization and response-pair extraction. The consequence of this choice is that the resulting interaction labels are "automatic" and not human-verified; this boundary is kept throughout the report.

The gap between reducing loss and valid generation. This is the most important negative finding of the project. Over seven experimental generations, low-rank adaptation of the direct model steadily reduced the *teacher-forced* loss, but never reached valid *autoregressive* generation of Persian speech. The final audit revealed its structural cause: the model's English tokenizer needs more than six tokens per Persian word and exceeds the budget of one token per frame.

The boundary between engineering evidence and scientific claims. Automatic measures were available, but 12–24 GB target hardware, human labelling of interruptions and a large human study were not available within the project deadline; instead, a browser loopback test with the memory budget of a 12 GB card, independent two-pass automatic labels and a small

listening study were designed. The project's methodological decision was to label every measure with its "evidence class" and never to present an automatic approximation in place of official evidence. Section 4.6 turns this decision into a way of working.

1.6 Two-Track Strategy

To reduce execution risk, the project pursued two tracks in parallel:

1. **The direct track.** Low-rank adaptation of Moshika on Persian conversational data, in the hope of obtaining a single Persian speech-to-speech model. This track answers the spirit of the project definition directly.

2. **The cascade track.** A modular chain of Persian speech recognition, text reply generation with a large language model, and text-to-speech, on top of which full-duplex control and interruption logic are built separately. This track guarantees that the project has a working system to demonstrate.

Both tracks used the same dataset. The final outcome, as reported in Chapter 5, was asymmetric: the cascade track reached a streaming full-duplex system that meets the numeric latency and interruption criteria in automated tests, while the direct track, and a speech-input track with Gemma 4, are reported as negative results together with their measured causes.

1.7 Contributions

The contributions of this project can be summarized in six items.

1. **A streaming full-duplex Persian voice assistant.** The system works with an open microphone, detects the end of speech automatically and speculatively, and generates and plays the reply clause by clause. In loopback testing with a real browser, the mean latency from end of speech to the first audio of the reply was 372 ms and its 90th percentile 431 ms, no turn exceeded 500 ms, and the GPU side of the system fit in 11.1 GiB of memory.

2. **Two-stage interruption detection with lexical confirmation.** Ducking after one frame of speech louder than the expected echo, and confirmation with the speech recognizer after removing listener tokens and the assistant's own echo; accuracy 0.933 and F1 0.880 in a deployment simulation and accuracy 0.825 in a real browser, against 0.574 for an

acoustic classifier alone; the reply after an interruption, keeping only the heard part of the interrupted reply in the history, was as relevant as the reply to a normal turn.

3. **A derived and audited Persian conversational dataset.** 6,754 "user speech / response" pairs with 108.584 hours of stereo audio, session-based group splits and zero group leakage; an independent automatic listener confirmed the turn boundary in 93.8% of samples.

4. **Evidence-based component selection.** A final semantic test of the responder (relevance 3.150 versus 1.425), and re-selection of the recognizer on human transcripts, the responder on requests of older adults and the synthesizer with two automatic listeners, all under pre-registered protocols; a comparison of 15 responder models with their quantized builds and serving settings for 12 GiB and 24 GB cards, judged by two models from families with no candidate; and optimization of the lab synthesizer without retraining, which found and fixed several of its text front-end errors (a dropped final word, misread numbers).

5. **A measured cause for the failure of the direct track.** Seven experimental generations of Moshika and one speech-input model; it was shown that the tokenizer bottleneck is real and that removing it doubles the best result, but is not enough for intelligible speech.

6. **An evidence-first way of working.** Protocols and thresholds were registered before running, test splits were opened only once, result artifacts are write-once, and a *fail-closed* aggregator prevents automatic approximations from being promoted to official evidence.

The latency and interruption evidence is automated and on an H100, the labels are not human, and the listening study was not conducted for lack of participants. The exact scope of these results is reported in Chapter 6.

1.8 Structure of This Report

This report is organized in seven chapters.

Chapter 2 introduces the necessary preliminaries: the representation of the speech signal and acoustic features, speech recognition and text-to-speech, large language models and low-rank adaptation, neural audio codecs and the architecture of speech-to-speech models, and the measures and statistical concepts used in evaluation.

Chapter 3 reviews prior work and positions this project among cascade systems and end-to-end spoken dialogue models, Persian speech resources, and prior work on interruption detection.

Chapter 4 describes the proposed method: the dataset construction pipeline, the cascade architecture, the interruption detection module and the full-duplex transport layer, the design of the direct-track experiments, and the evidence-first way of working.

Chapter 5 presents the results with the corresponding tables and figures and states, for each result, its permitted claim boundary.

Chapter 6 interprets the results, in particular the gap between reducing loss and valid generation, and then lists the limitations and the ethical and legal considerations.

Chapter 7 presents the conclusions and suggestions for future work. The appendix gives the reproduction commands, the exact hyperparameters of the experiments, and the map of evidence artifacts.

Chapter 2

Preliminaries

This chapter introduces the concepts and tools used in the following chapters. It starts from the speech signal and acoustic features, moves to the components of the cascade and then to the architecture of end-to-end speech-to-speech models, and ends with the measures and statistical concepts of evaluation.

2.1 The Speech Signal and Acoustic Features

In this project speech is represented as a mono signal sampled at 16 kHz with 16-bit integer samples. The continuous signal $x[n]$ is divided into fixed-length *frames*; in this project a frame is 20 ms long, equivalent to 320 samples. Framing rests on the assumption that the statistical properties of speech are approximately constant over short intervals.

2.1.1 Energy and Zero-Crossing Rate

The simplest feature is the **frame energy**. For frame t with N samples, the log energy is computed as

$$E_t = \log\left(\frac{1}{N}\sum_{n=1}^{N} x_t^2[n] + \epsilon\right), \tag{2.1}$$

where ϵ is a small positive value that avoids the logarithm of zero. Energy alone is not enough to detect speech, because loud noise also has high energy. A complementary feature is the **zero-crossing rate**, which counts the sign changes of the signal within the frame:

$$Z_t = \frac{1}{2(N-1)} \sum_{n=2}^{N} \left| \operatorname{sgn}(x_t[n]) - \operatorname{sgn}(x_t[n-1]) \right|. \tag{2.2}$$

Voiced sounds have a low zero-crossing rate and fricatives a high one; combining the two features is the basis of classical voice activity detectors [8].

2.1.2 Fundamental Frequency and the YIN Method

The **fundamental frequency**, F_0, is the vibration frequency of the vocal folds and is perceived as pitch. In this project F_0 is estimated with the YIN method [9], which rests on the squared difference function:

$$d_t(\tau) = \sum_{n=1}^{W} \big(x_t[n] - x_t[n+\tau] \big)^2, \tag{2.3}$$

where τ is the lag and W the length of the analysis window. A "cumulative mean normalized" version of this function is then built, and the first local minimum that falls below a threshold is chosen as the period $\hat{\tau}$; the fundamental frequency is $F_0 = f_s/\hat{\tau}$. If no minimum falls below the threshold, the frame is marked "unvoiced". In this project the search range of the fundamental frequency is 60–400 Hz, which covers male and female speakers.

F_0 matters for interruption detection because the onset of human speech is usually accompanied by the appearance of voiced structure, while ambient noise or loudspeaker playback has no stable pitch pattern.

2.1.3 Mel-Frequency Cepstral Coefficients

MFCCs, the **mel-frequency cepstral coefficients** [10], are the most widely used short-term representation of speech; they compress the frame spectrum in a way that matches human auditory perception. They are computed as follows:

1. apply a window (for example a Hamming window) and compute the discrete Fourier transform of the frame;
2. compute the power spectrum and pass it through a bank of triangular filters whose centres are spread on the mel scale. The mel scale relates to linear frequency as

$$m(f) = 2595 \log_{10}\!\left(1 + \frac{f}{700}\right); \tag{2.4}$$

3. take the logarithm of each filter output;

4. apply the discrete cosine transform to decorrelate the components and keep the first coefficients.

This project keeps 13 coefficients. So that the features also carry information about *change* over time, the first derivative of these coefficients, known as the "delta", is computed and added to the feature vector. The features of each frame therefore comprise energy, zero-crossing rate, fundamental frequency, a voicing flag, 13 MFCCs and the 13 corresponding deltas.

2.1.4 Context Window and Decision Vector

A decision about an interruption cannot rely on a single frame. The features of the frames of a "context window" are therefore aggregated. In this project the context window is 400 ms, that is, 20 consecutive frames. For each dimension of the frame feature vector, three statistics (the mean, the standard deviation and the maximum) are computed over the window and concatenated into a single vector. This vector is the input of the classifier.

2.2 Speech Recognition

Speech recognition maps an audio signal to a sequence of words. Current architectures usually combine a neural encoder with an alignment-free training objective. Two common objectives are Connectionist Temporal Classification (CTC) [11], which marginalizes over all possible alignments under a conditional independence assumption, and the RNN Transducer (RNNT) [12], which also involves a language prediction network in the decision and is better suited to streaming recognition.

The encoder used in the project's recognition service belongs to the Conformer family [13], which combines self-attention [14] with convolution: self-attention models long-range dependencies and convolution local spectral patterns. The model is a hybrid RNNT-CTC model with subword units, implemented with NeMo [15] and served as a local service.

An alternative approach is large-scale weakly supervised training; its prominent example is Whisper [16], trained on hundreds of thousands of hours of multilingual data, which this project used only in component comparison tests.

2.3 Text-to-Speech

Current **text-to-speech** systems are usually end-to-end. The VITS [17] architecture, on which the Piper engine [18] is based, combines a conditional variational autoencoder with adversarial training and a flow-based duration predictor, and generates the waveform directly without a separate vocoder.

For Persian, text pre-processing matters particularly. Persian script does not write short vowels, the zero-width non-joiner (ZWNJ) marks the boundaries of the parts of compound words, and the written form of words can differ across sources. For this reason the project applies a deterministic Persian text normalizer before both evaluation and speech generation, so that character forms, ZWNJs and digits are unified.

Using a *deterministic* speech engine with a fixed voice is a methodological decision: because the audio output for a given input text is always the same, measurements based on it become reproducible.

2.4 Large Language Models and Low-Rank Adaptation

2.4.1 Autoregressive Language Models

A **language model** factorizes the probability of a token sequence $y_{1:T}$ as a chain:

$$p_\theta(y_{1:T}) = \prod_{t=1}^{T} p_\theta(y_t \mid y_{<t}), \tag{2.5}$$

and is fitted by minimizing the cross-entropy loss on training data. The dominant architecture is the decoder-only Transformer [14]. *Instruction-following* models are built with a supervised fine-tuning stage on instruction–response pairs [19]; the project's text responder belongs to this class.

2.4.2 Teacher Forcing and Exposure Bias

A point that is decisive for interpreting the results of Chapter 5 is the difference between training and inference conditions. In training, the model receives the "correct" previous tokens as its condition at every step; this is called **teacher forcing**. The loss reported in training and validation is computed under this condition.

At inference, however, the model must condition on *its own generated* tokens. If the distribution of generated tokens drifts from that of the reference tokens, the model enters a region of the state space it has not seen in training, and errors can grow cumulatively. This mismatch is known as **exposure bias** [20], and its standard remedy is "scheduled sampling", which gradually replaces reference tokens with generated ones during training [21].

The practical consequence of this concept is explicit: "lower loss under teacher forcing does not by itself guarantee that the model gives usable output in autoregressive generation". The direct-track experiment of this project is a clear example of this gap.

2.4.3 Parameter-Efficient Adaptation and LoRA

Full fine-tuning of a multi-billion-parameter model on limited hardware is not practical. **Parameter-efficient adaptation** methods train only a small number of parameters [22, 23]. The method used in this project is LoRA [24]. Its idea is that the update needed for a weight matrix has *low rank*. For a frozen weight $W_0 \in \mathbb{R}^{d\times k}$, instead of learning a full ΔW, two small matrices $A \in \mathbb{R}^{r\times k}$ and $B \in \mathbb{R}^{d\times r}$ with $r \ll \min(d, k)$ are trained, and the layer output becomes

$$h = W_0 x + \frac{\alpha}{r} BAx. \tag{2.6}$$

At the start, B is zero, so the adapted model equals the base model. The parameter r is the rank and α a scale factor. This project tried ranks 64 and 128. The key advantage of LoRA for this work is that the base weights stay frozen, so a failed adapter never damages the base model and can simply be removed.

2.5 Neural Audio Codecs and Speech-to-Speech Models

2.5.1 Residual Vector Quantization

For a language model to generate speech, audio must be turned into a sequence of discrete tokens. A **neural audio codec** does this with an autoencoder with a Residual Vector Quantizer (RVQ) at its bottleneck. Prominent examples are SoundStream [25] and EnCodec [26].

In residual quantization the latent vector z is approximated successively. If Q_i is the i-th codebook, then with $r_0 = z$:

$$q_i = Q_i(r_{i-1}), \qquad r_i = r_{i-1} - q_i, \qquad \hat{z} = \sum_{i=1}^{N_q} q_i. \tag{2.7}$$

The first codebook carries coarse and mostly "semantic" information, and the following codebooks add "acoustic" detail. This hierarchy is why a larger weight is given to the first codebook's loss in training speech models; this weight was the changed factor in one of the project's experiments.

2.5.2 Language Modelling on Audio Tokens

Once audio is discrete, the same language modelling machinery can be applied to audio tokens; this idea is used in AudioLM [27] and VALL-E [28]. The main challenge is that each time step has not one token but N_q tokens (one per codebook). The solution is the RQ-Transformer: a large "temporal" Transformer works along the time axis, and a small "depth" Transformer generates the N_q tokens of each time step autoregressively [29].

2.5.3 The Moshi Architecture and the "Inner Monologue"

Moshi [1] models dialogue as speech-to-speech, and three of its properties matter for this project.

First, the **Mimi codec**: a streaming audio codec that combines a semantic representation with an acoustic one in residual codebooks and works with an 80 ms time step.

Second, **parallel streams**: the model models two audio streams at the same time, its own speech and the user's. The explicit notion of a "turn" therefore disappears, and overlap, interruptions and backchannels fit naturally into the model. This property is what makes Moshi a natural candidate for the full-duplex requirement.

Third, the **Inner Monologue**: before generating the audio tokens of each step, the model predicts time-aligned "text" tokens. This intermediate text noticeably improves the linguistic quality of the generated speech. Its consequence for debugging is that the text loss and the audio loss can be monitored separately; Chapter 5 uses this separation to show where the direct track failed.

The version of the model used in this project is Moshika: a 7-billion-parameter model with a female voice, 32 Transformer layers, a latent dimension of 4096 and 16 audio codebooks, eight of which are predicted in the depth Transformer.

2.6 Full Duplex and Turn-Taking

In human conversation, turn-taking is a precise and fast process. Classical work on turn organization showed that the gaps between turns are short and regular [30]; later engineering reviews report the typical gap as about two tenths of a second [31].

From an engineering point of view, three levels of capability should be distinguished:

- **Half duplex**: the system does not listen to the microphone while playing its reply. The user has to wait.
- **Full duplex with external control**: the microphone is active during playback and a *separate* component decides whether to stop playback. This is the architecture implemented and evaluated in this project.
- **Native full duplex**: the generative model itself models the user's stream and its own at the same time, and turn-taking is part of the learned distribution, as in Moshi.

For the three interaction events labelled in this report, the following distinction holds: an "interruption" is when the second speaker starts talking, continues and takes the turn; a "backchannel" is a short affirming utterance that does not take the turn; and a "clean turn" is when the second speaker starts only after the first speaker has finished.

2.7 Classification with Gradient Boosting

The project's interruption detector uses a **gradient-boosted tree model** [32] implemented in scikit-learn [33]. This method builds an ensemble in which each new tree fits the negative gradient of the loss of the current model:

$$F_m(x) = F_{m-1}(x) + \nu\, h_m(x), \tag{2.8}$$

where h_m is the new tree and ν the learning rate. This model family was chosen for three reasons: the project definition explicitly asks for a module based on hand-crafted features; its inference on the CPU fits within the few-millisecond budget of each audio step; and with a limited number of training events, tree models are more stable than deep neural networks.

2.8 Evaluation Measures and Statistical Concepts

2.8.1 Word and Character Error Rates

The **Levenshtein** distance [34] is the minimum number of insertions, deletions and substitutions needed to turn one string into another. The word error rate and character error rate are defined on it. This report gives two forms of aggregation. The "micro" form divides the total number of edits by the total length of the references:

$$\mathrm{WER}_{\mathrm{micro}} = \frac{\sum_i d\big(\hat{w}^{(i)}, w^{(i)}\big)}{\sum_i \big|w^{(i)}\big|}, \tag{2.9}$$

and the "macro" form is the simple mean of the per-sample error rates. For Persian, the definition of a word boundary matters: in this project word tokens are separated by whitespace, and the ZWNJ also counts as a boundary. Character tokens are Unicode code points, excluding whitespace and ZWNJ. This definition makes the word error rate in Persian much stricter than the character error rate, a point interpreted in Chapter 6.

2.8.2 Detector Measures

For the two-class event "interruption versus non-interruption", the confusion matrix gives four values: true positives, false positives, true negatives and false negatives. From them the accuracy, the F1 score of the interruption class, the False Accept Rate (FAR) and the False Reject Rate (FRR) are computed. In full-duplex systems these two error rates are not equivalent: a false accept interrupts the system's reply for no reason, while a false reject makes the system ignore the user's interruption.

2.8.3 Confidence Intervals and Group Correlation

For proportions, the **Wilson** confidence interval [35] is reported; unlike the simple normal approximation, it remains valid for small samples and for proportions near the boundary.

More importantly, events drawn from one speech session are not *independent*; the speaker, the acoustic conditions and the conversational style are shared. An event-level confidence interval therefore underestimates the real uncertainty. The solution used in this project is a *session-level block bootstrap*: in each resampling repetition, *sessions* are drawn with replacement, not

individual events [36]. Chapter 5 reports both intervals so that the effect of this difference can be seen.

2.8.4 Group Splits of the Data

If two parts of one speech session fall into the training and the test set, the model can memorize the speaker or the topic and the test result becomes optimistic. To avoid this "group leakage", the split unit in this project is the source session identifier: all pairs from one session fall entirely into one and only one split. The assignment uses deterministic hashing of the session identifier, so that the split is reproducible.

2.8.5 Automatic Judging with a Language Model

The semantic quality of a dialogue reply cannot be measured with string-level measures, because one question has many correct answers. A standard substitute is to use a large language model as a "judge" [37]. In this method, the candidate reply and the comparison replies are given to the judge *double-blind* and in randomized order, and the judge scores them with a predefined rubric.

The known limitations of this method are bias towards presentation order, bias towards reply length, and bias towards replies generated by models of the judge's own family. This project took three steps to reduce these effects: blinding the order, deterministic decoding of the judge at temperature zero, and repeating each call and reporting the agreement rate. Nevertheless, as Chapter 6 stresses, this measure is an "automatic approximation" and no substitute for human evaluation, especially since in this project the judge and the candidate are of the same model family.

2.8.6 Latency Measures

Two time measures are defined in this report:

- $T_{\text{first_audio}}$: the time from the end of the user's speech to the first *audible* audio sample of the reply. The official value of this measure must be reported by the client, because only the browser knows that the audio source was actually scheduled.
- $T_{\text{barge_in}}$: the time from the acoustic onset of the user's interrupting speech to the moment playback of the reply *actually* stops; this value also needs client confirmation.

An important methodological point is that "the server-side time to generate the whole reply" is not the $T_{\text{first_audio}}$ measure. Server generation time is a useful measurement for debugging, but not the latency acceptance condition. Chapter 5 keeps this distinction carefully.

Chapter 3

Related Work

This chapter reviews related research along four axes: cascade versus end-to-end spoken dialogue systems, open speech-to-speech models and their full-duplex capabilities, Persian speech resources, and prior work on interruption detection and turn-taking. At the end of the chapter, the position of this project relative to prior work is stated.

3.1 From Cascades to End-to-End Models

The conventional architecture of spoken dialogue systems is a chain of four independent components: voice activity detection, speech recognition, a text dialogue engine and text-to-speech. Its advantage is modularity: each component can be improved, debugged and replaced with the best available model for the language separately. For low-resource languages this modularity is a decisive advantage, because for each component the chance of finding an acceptable Persian model is higher than the chance of finding a Persian speech-to-speech model.

Against it, the authors of Moshi [1] list three fundamental objections to the cascade architecture. First, latency accumulates along the chain and stretches the gap between two dialogue turns to several seconds. Second, non-linguistic information is lost: since text is the only interface between input and output, emotion, tone and non-speech sounds that change the meaning disappear. Third, it depends on an explicit segmentation of speech into turns, which cannot represent overlap, interruptions or short affirming utterances.

The observations of Chapter 5 support both sides of this argument. The latency objection shows in the measurements of the cascade track: the time to generate a complete reply in the cascade, even on powerful hardware, is in the order of seconds and grows with long inputs

(Section 5.3). At the same time the advantage of modularity also shows: only this architecture reached a working, evaluable Persian system within the project's time frame.

3.2 Open Speech-to-Speech Models

3.2.1 The Mini-Omni Family

Mini-Omni2 [5], which the project definition names as an example base model, is a multimodal assistant based on Qwen2-0.5B that uses Whisper and CLIP encoders for audio and image input and generates speech with a "text-guided delayed parallel generation" strategy. Two of its properties matter for our problem: first, its interruption mechanism is "command-based" rather than a continuous acoustic decision; second, the project has not published a complete, official speech-to-speech trainer, so adapting it to Persian would have required rewriting the training pipeline from scratch.

3.2.2 SpeechGPT and Discrete Speech Units

SpeechGPT [38] is one of the first large language models that accepts and generates speech directly as discrete units in its vocabulary, instead of converting speech to text through a separate recognizer and synthesizer. It was trained in three stages: pre-training to adapt to the speech modality, cross-modal instruction tuning, and "chain-of-modality" instruction tuning, in which the model first thinks the reply as text and then turns it into speech units. The idea of relying on intermediate text inside one integrated model is what Moshi implements as an "inner monologue" synchronized with the audio. SpeechGPT, however, is turn-based and non-streaming: it takes the whole user utterance before replying and is not full duplex, so it was not a suitable base model for the latency and interruption goals of this project.

3.2.3 The LLaMA-Omni Family

The first version of this family [39] used a non-autoregressive speech decoder to reduce latency. LLaMA-Omni2 [6] is a set of modular speech models from 0.5 to 14 billion parameters built on Qwen2.5-Instruct models; it uses a Whisper encoder for input and an autoregressive streaming speech decoder inspired by CosyVoice 2 [40] for output. It was trained on only 200 thousand multi-turn dialogue samples and outperformed GLM-4-Voice [41], which was trained on millions of hours of speech.

This model was the main option at the start of the project, because its 0.5-billion-parameter size fitted the target memory budget and its stack (a Whisper encoder plus Qwen2.5) matched the available tools. But the published repository covers inference only. The project's first attempt to build a training pipeline for this architecture, as Section 4.1 describes, reached an artifact that in fact learned to reconstruct the input rather than generate a conversational reply; this experience was the main motivation for moving to an official trainer.

3.2.4 Freeze-Omni and the Frozen-Model Approach

Freeze-Omni [2] takes a different strategy: the weights of the text language model stay *frozen* throughout training, and only the speech input encoder and the speech output decoder are trained. This design has two important advantages that are very relevant to low-resource conditions: the intelligence of the base model stays intact and "catastrophic forgetting" does not occur, and training becomes possible with much less data and compute. Full-duplex capability is obtained by adding a state classification layer on the last layer of the language model, which predicts whether the user has interrupted the conversation.

In hindsight, this approach is one of the most promising options for Persian and is suggested as future work in Chapter 7; at the time of this project's decision, an official, available trainer for a new language did not exist.

3.2.5 Qwen2.5-Omni and the "Thinker–Talker" Architecture

Qwen2.5-Omni [4] is an end-to-end multimodal model whose "Thinker–Talker" architecture enables simultaneous text and speech generation: the Thinker is a language model that generates text, and the Talker is a dual-track autoregressive model that generates audio tokens directly from the Thinker's latent representations. It was set aside for our project for two reasons: its 7-billion-parameter version needs far more memory than the target range of 12–24 GB, and its published spoken input and output languages do not include Persian.

3.2.6 Moshi and the Final Choice of the Direct Track

Among the models reviewed, Moshi [1] was the only one that met all three conditions this project needed: *native* full-duplex capability through parallel modelling of two audio streams, open weights, and, most importantly, an "official low-rank adaptation trainer" published directly by the model's authors [7]. As explained in Section 2.5, the model's "inner monologue" also

allows the text loss and the audio loss to be monitored separately, which was of practical value for debugging the failed experiments.

The published experience closest to this project's direct track is J-Moshi [42], which brought Moshi to Japanese. Its authors also found Moshi's English tokenizer inefficient for Japanese, replaced it with a Japanese SentencePiece tokenizer with 32,000 words, and re-initialized the text embedding tables. They then pre-trained the model on about 60,000 hours of Japanese spoken dialogue (the 69,000-hour J-CHAT corpus) on 128 V100 GPUs for 36 hours, and afterwards fine-tuned it on 344 hours of two-channel dialogue and 602 hours of synthetic dialogue. In other words, adapting Moshi to a new language, even with a suitable tokenizer, required pre-training at the scale of tens of thousands of hours of dialogue.

Closer prior work to this idea is dGSLM [43], which introduced generative modelling of spoken dialogue with two parallel streams and no text intermediary, and showed that phenomena such as overlap and backchannels can be reproduced in a learned way.

3.2.7 PersonaPlex and Role Control in a Full-Duplex Model

PersonaPlex [44] is a full-duplex speech-to-speech model built on the Moshi architecture that conditions the assistant's role and voice with a combined text prompt and audio sample. Its open weights are published and keep the native full-duplex capability. For this project, however, it had two decisive limitations: its release is English-centred and does not support Persian, and its documented tested hardware is outside the 12–24 GB range of requirement R6. In addition, an official trainer to adapt it to a new language was not available at the time of this project's decision. It was therefore considered as a comparison option and not chosen for the direct track.

3.3 Persian Speech Resources

For Persian, the situation of resources is asymmetric across three areas.

In **speech recognition**, usable models exist. The multilingual Common Voice corpus [45] has a Persian portion, and localized models based on the Conformer architecture in NeMo [15] have reached acceptable quality. The recognition service used in this project belongs to this class.

In **text-to-speech**, the most prominent public resource is ManaTTS [3]: the largest public single-speaker Persian corpus, with about 86 hours of audio at 44.1 kHz under an open licence, together with an open processing pipeline including a sentence segmentation tool and a forced

alignment method designed for low-resource languages. The Persian voice used in this project is derived from this resource. ParsVoice [46] is a larger corpus of Persian audiobooks with several thousand hours of speech and hundreds of speakers, built for multi-speaker synthesis; an XTTS model fine-tuned on it was measured in Chapter 5.

The project proposal names FarsTalk and Persian VoxCeleb as example Persian corpora; this project's search found no published corpus under these names. The closest published conversational resource is the Corpus of Conversational Persian Transcripts [47]: transcripts of about 20 hours of informal conversation in the Tehrani dialect, whose audio is not published and which is therefore of no use for training a speech model.

In **full-duplex conversational data**, however, there is a serious gap. What a full-duplex model needs to learn from is natural two-sided speech with overlap, interruptions and backchannels. The existing Persian resources are mostly single-speaker and read and do not contain these phenomena. Persian conversational telephone corpora exist in research archives, but they have restrictive licences and their scale is below the 100–200 hours targeted by the project definition. This gap is what made the derived dataset of Chapter 4 necessary.

3.4 Low-Latency Streaming Inference

A low-latency speech system must run each component *streaming*, that is, start the output before the whole input has arrived. A common speed measure is the **real-time factor** (RTF): the ratio of processing time to audio duration; an RTF below one means processing does not fall behind playback. For dialogue, however, the RTF is not enough; what the user feels is the "latency to the first output".

In **speech recognition**, full attention of the encoder needs the whole utterance. The common remedy is chunked attention: the encoder processes each chunk with a limited past context and a short look-ahead [48]. A cache-aware Conformer encoder [49] limits past and future context during training too and keeps the activations of previous chunks, so that step-by-step inference matches training. In the **language model**, token-by-token generation is inherently streaming; for long dialogues, a sliding window over the key–value cache together with keeping a few initial tokens ("attention sinks") maintains stability [50]. In **speech synthesis**, clause-by-clause or chunk-by-chunk generation lets playback start before the whole reply is generated.

This project uses all three levels, but with simpler choices (Section 4.3): user utterances are short and the CTC recognizer processes each utterance in about 60 ms after its end is detected;

the language model streams with a prefix cache; the synthesizer works clause by clause; and speculative end-of-speech detection starts the work before the end is confirmed.

3.5 Older Adults and Voice Assistants

The voices of older adults differ acoustically from those of younger adults, and this difference harms speech recognition: in one study, the word error rate for older speakers was about 9 percentage points higher, and the fundamental frequency, jitter, shimmer and harmonicity differed significantly between the two groups [51]. The same pattern appeared on this project's human panel: the character error rate of the deployed recognizer on older speakers was 15.6% against 8.6% for the others (Section 5.6).

On the user side, a three-week deployment of a smart speaker in the homes of seven older adults with little technology experience showed that information seeking, especially health information, was the most persistent use, and that the reliability of spoken answers was a serious concern for them; reminders were considered useful but were used less because of doubts about their reliability [52]. Other workshops with older adults listed both the opportunities and the barriers of voice interaction [53]. These findings explain several design choices of this project: short, simple replies that are honest about limitations (the elderly prompt), tolerance of longer pauses in end-of-speech detection, slightly slower speech, and referral to a doctor or the emergency services on risky topics. The effect of these choices on the experience of older adults can only be measured by the human study.

3.6 Interruption Detection and Turn-Taking

Prior work in this area has two branches.

The **classical** branch rests on hand-crafted acoustic features and statistical detectors. The statistical-model-based voice activity detector [8] is the reference example of this branch. Interruption detection in traditional telephone dialogue systems was in practice a voice activity detector with a conservative threshold; the known problem of these systems was that loudspeaker playback or ambient noise was mistaken for an interruption. A comprehensive review of turn-taking mechanisms in dialogue systems and human–robot interaction is in [31], which shows that turn-taking is not merely a silence detection problem and depends on prosodic and syntactic cues. The linguistic basis of the topic goes back to the classical work [30], which showed that the gaps between turns in human conversation are remarkably short and regular.

The **learned** branch places turn-taking inside the generative model. In Moshi and dGSLM there is no separate detector, and turn-taking is part of the learned distribution over two parallel streams. Freeze-Omni takes a middle way: a light classification head on the language model predicts the dialogue state.

The project definition explicitly asks for the classical branch: a module based on energy, pitch and MFCC with a target accuracy above 80%. This project's implementation follows that path, with the methodological difference that instead of manual thresholding, a gradient-boosting classifier is trained on the context-window feature vector and its decision threshold is chosen on validation sessions only.

To label interaction events in recorded data automatically, **speaker diarization** was used. The reference tools in this area are pyannote.audio [54] and multi-scale diarization with dynamic weighting [55]; the diarization service used in this project belongs to the second family.

To train and measure the detector under near-deployment conditions, speech is mixed with noise and room responses. The common corpora for this are MUSAN [56] (music, speech and noise), DEMAND [57] (multichannel noise of various environments) and the OpenSLR room impulse response and noise collection [58].

3.7 Automatic Evaluation of Dialogue Quality

Measuring the quality of a dialogue reply with string-matching measures is not meaningful. The approach that has become standard in recent years is to use a large language model as a judge. The reference work in this area [37] showed that automatic judging with strong models correlates highly with human preference, but the same work documented its systematic biases: position bias, length bias and self-enhancement bias towards the output of same-family models.

This project used this method as the *only* semantic measure available, but with three restrictions: blinding and randomizing the order of the arms, deterministic decoding, and repeating each judge call and reporting the agreement rate. More importantly, because the judge and the candidate in this project are of the same model family, the result is explicitly labelled a "same-family automatic approximation" and is not counted as a substitute for human evaluation or an independent benchmark. For human measurement, the reference protocol is the mean opinion score based on recommendation ITU-T P.800 [59], which this project designed but did not run.

3.8 Position of This Project

Table 3.1 summarizes the position of this project relative to prior work.

Table 3.1: This project compared with prior work on the capabilities required by the project definition

Work	Full duplex	Open weights	Official trainer	Persian support	Memory range
Mini-Omni2	command-based	yes	no	no	fits
LLaMA-Omni2	no	yes	no	no	fits
Freeze-Omni	yes	yes	partial	no	fits
Qwen2.5-Omni	yes	yes	no	no	does not fit
Moshi	native	yes	yes	no	does not fit
PersonaPlex	native	yes	partial	no	does not fit
This project, cascade	external control	yes	not needed	yes	fits
This project, direct	native	yes	yes	failed	does not fit

The "memory range" column refers to the fit with the 12–24 GB range of the project definition; for this project's direct track it is reported from the training memory measured on the available hardware, not from a measurement on target hardware.

Three points follow from this comparison.

First, none of the existing end-to-end models supports Persian; any direct track therefore has to adapt a model, and adaptation needs an official trainer. Only Moshi met this condition completely. PersonaPlex is also natively full duplex, but it is English-centred and its adaptation trainer was not available for Persian.

Second, the models with native full-duplex capability do not fit within the target memory range. This conflict between requirements R2 and R6 of the project definition is a structural constraint, not an implementation defect.

Third, this project's distinctive contribution lies elsewhere. Prior work publishes *models*; this project builds, for one specific low-resource language, "a data pipeline and an evidence-first evaluation framework" and shows which claims can and cannot be proven with this evidence. In particular, the negative result of the direct track, reported in Section 5.9, is a citable finding for later researchers: in low-rank adaptation of a speech-to-speech model to a new language, a reduction of the teacher-forced loss is not a sufficient criterion for checkpoint selection.

Chapter 4

Proposed Method and Implementation

This chapter describes the structure of the system, how the data were built, and the design of the experiments. The order is: first the early attempt and the lesson learned from it; then the dataset construction pipeline; then the architecture of the delivered cascade, the interruption detection module and the full-duplex transport layer; then the design of the direct-track experiments; and finally the evidence-first way of working that frames all evaluations of the next chapter.

4.1 Early Attempt and Revision of the Architecture

The project's first implementation was built on the LLaMA-Omni2 architecture: a frozen Whisper speech encoder, a trainable projector, the Qwen2.5-0.5B [60] language model with a LoRA adapter, and separate heads for recognition and for generating speech tokens. This artifact was trained and its loss decreased.

An internal code audit, before any result was reported, revealed three fundamental defects that must be recorded for the methodological honesty of this report:

- The language model did not take part in the forward pass; what was trained was therefore an audio mapping network, not a dialogue-oriented language model.
- The recognition target used per-process randomized hashes, so the training labels were unstable across runs.
- The speech generation target was "reconstruction of the same input utterance", not the generation of a conversational reply; and to hear the output, an external speech engine bypassed the path of the learned artifact.

The conclusion of this audit was explicit: that artifact was not a speech-to-speech model and could not be reported as one. This experience led to two decisive decisions. First, serving was moved entirely to a cascade of real, testable components. Second, for the direct track only a model with an "official trainer" was accepted, so that the training objective and data pipeline are set by the model's authors rather than rewritten locally. These two decisions shaped the final project.

4.2 Construction of the Persian Conversational Dataset

4.2.1 Source and Strategy

Since no public Persian corpus with interruption labels was available (Section 3.3), the dataset of this project was derived from an internal archive of Persian spoken content. The archive contains long conversational programmes from four channels with different styles. Internal research use of these resources was recorded with the supervisor's approval; this approval is not a licence to redistribute the raw audio or captions, and no restricted data were placed in the code repository.

A methodological point corrected midway is worth recording. In the first design, the text label of each segment was produced by re-running the speech recognition service. But that service had itself been tuned earlier on the captions of these same channels. Using it as the "teacher" therefore created a "circular loop" and fixed its own systematic errors as reference labels. Once this problem was identified, the text labels were switched to the original source captions, and the recognition service was kept only as a runtime component and a measurement tool.

4.2.2 Pipeline Stages

Figure 4.1 shows the stages of the data pipeline.

Inventory and selection. The measurable inventory of the archive was 775.887 hours of captions in 1,442 long programmes. A weighted, *deterministic* selection was run on this inventory, aimed at balancing the channels and avoiding the dominance of one speaking style. The output of this stage is 309 programmes and 1,129 windows with a total of 219.946 candidate hours.

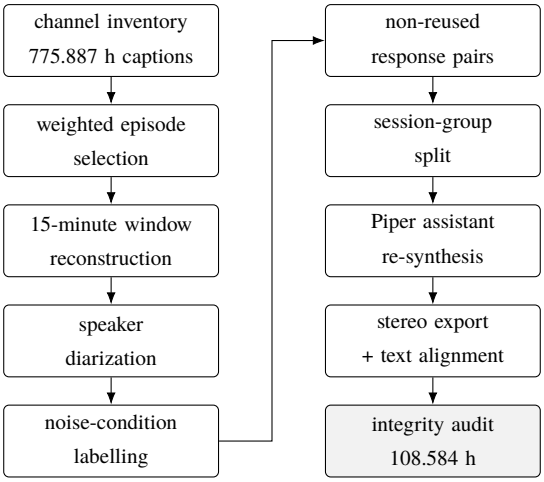


Figure 4.1: Stages of the Persian conversational dataset pipeline. Each stage is a separate command with an auditable output.

Window reconstruction. The source programmes were stored as small chunks. To preserve the structure of the conversation, the chunks were reconstructed into continuous 15-minute windows. This length is long enough to contain several complete dialogue turns.

Speaker diarization. Each window was given to the diarization service, which returns the time boundaries and speaker identity of each speech segment as well as the overlap segments. The total of automatically classified multi-speaker segments is 207.154 hours.

Noise-condition labelling. For each segment, a noise-condition label based on an estimated signal-to-noise ratio was added, so that the effect of acoustic conditions can be separated in later analyses.

Why the MFA forced aligner was not used. The proposal suggests the Montreal Forced Aligner [61] for marking turns and overlaps. Forced alignment places known text within the audio of *one speaker*, but it does not say which speaker talks when or where two voices overlap; the turn boundaries and overlap segments therefore came from speaker diarization, and the text of each turn from the caption timing. Word-by-word timing was needed only for the assistant speech in the seventh version of the direct track (Section 4.5.3), and there no statistical aligner was required: that speech had been generated by Piper itself, and the time of each phoneme was taken from the same synthesizer's phoneme-to-sample alignment in an identical re-synthesis.

4.2.3 Response-Pair Extraction and Interaction Labels

From the diarized segments, “user speech and the next speaker’s response” pairs are extracted. First, consecutive segments of one speaker less than half a second apart are merged into a complete turn. Then a pair is built for every two consecutive turns of two different speakers.

An important constraint of this stage is “no reuse”: each segment of the source audio appears in only one pair. This constraint is needed because reuse would artificially inflate the number of reported hours and would act as data repetition in training.

For each pair an automatic interaction label is also computed. If the second turn starts before the first turn ends, there is overlap. In that case, if the second turn is short and ends before the first turn ends, the label is “backchannel”, and otherwise “interruption”. If the overlap cannot be attributed to a speaker, the label “unattributed overlap” is recorded.

The output of this stage is 6,754 pairs with 123.796 hours of source audio. In addition, 770 conservative interaction-event candidates (717 interruption-like and 53 backchannel-like) were recovered from the speaker boundaries.

These labels are entirely automatic. *The forty-row listening review of the data and the twenty-four-row review of interaction events were not run, under a documented time waiver. Therefore no row is reported as “human-verified” anywhere in this report, and the number of human-verified labels is zero.*

4.2.4 Group Splits and Re-Synthesis of the Response Voice

Training, validation and test splits were made by source session identifier with deterministic hashing (Section 2.8.4). The result is 6,419 training pairs, 131 validation pairs and 204 test pairs, and the automatic audit confirms zero group leakage.

For the direct track, the “assistant” channel must not carry the voices of the many source speakers, because the model has to learn one stable target voice. The text of the response turn was therefore re-synthesized with the Persian speech engine and one fixed, *deterministic* voice. This is standard practice in training speech-to-speech models. The final output is one stereo audio file per pair, with channel zero the assistant’s speech and channel one the user’s speech, together with an adjacent text alignment file.

The total exported stereo audio is a measured 108.584 hours, which lies within the 100–200 hours targeted by requirement R4. Figure 4.2 shows the gradual reduction in volume from the raw inventory to the audited export.

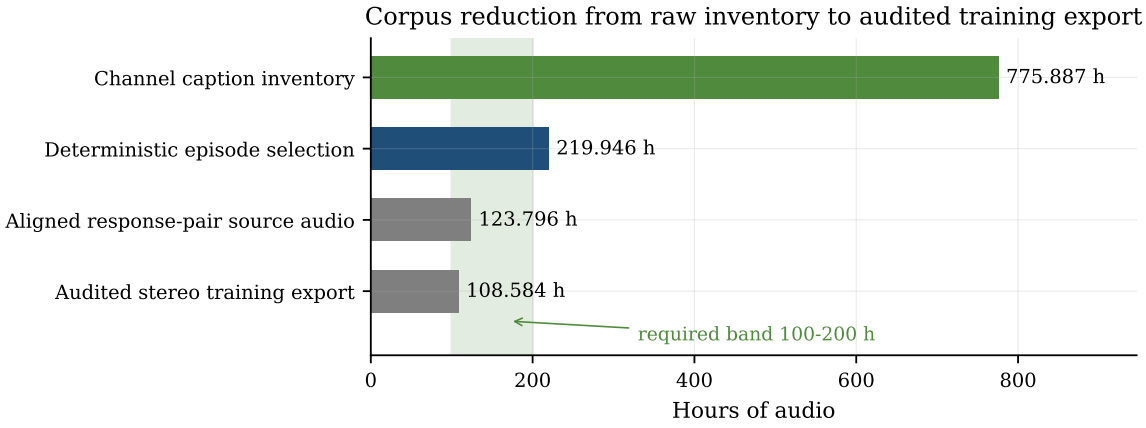


Figure 4.2: Gradual reduction of the amount of audio from the channels' caption inventory to the audited stereo training export. The light band shows the 100–200 hours targeted by the project definition.

4.3 Architecture of the Streaming Cascade

The delivered system is a streaming cascade on top of which full-duplex control is built (Figure 4.3). The first version of this chain worked with a "push-to-talk" button, built the whole reply before playback and ran all three stages one after another inside one process; our estimate of its latency from end of speech to first audio was about 2.2 seconds (median). In the final version each stage overlaps with the next: the end of speech is detected automatically, reply generation starts before the end of speech is confirmed, tokens arrive as a stream, and each clause of the sentence is synthesized and played as soon as it is complete.

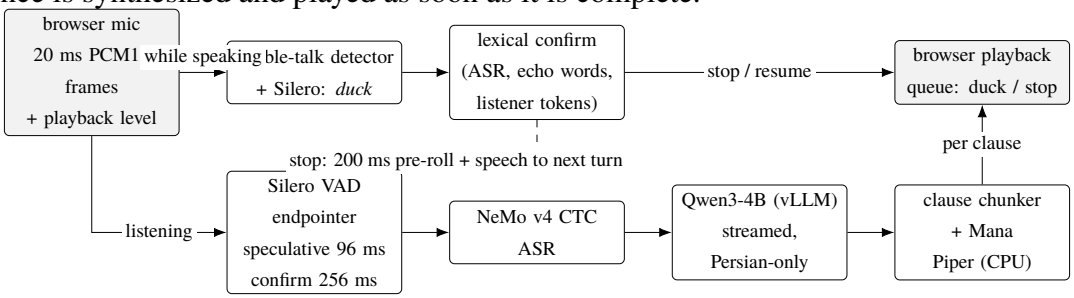


Figure 4.3: The streaming architecture of the delivered system. The bottom row is the reply generation chain while listening and the top row is interruption control while the assistant speaks. The dashed arrow shows the transfer of the start of the user's speech to the next turn.

4.3.1 Browser Client and Frame Format

The browser client captures the microphone continuously (with the browser's echo cancellation), resamples it to 16 kHz and sends one frame every 20 ms: a four-byte sequence number, the RMS

level of the assistant audio playing at that moment (measured with a Web Audio analyser node) and 320 PCM16 samples. The server sends the audio clauses of the reply as binary messages (turn number, clause number, PCM16) and control events as JSON. The client schedules the clauses on a playback queue, ducks the playback volume or stops the source when told to, and reports timings with its own clock: from the frame holding the acoustic end of speech to the scheduled start of the first clause ($T_{\text{first_audio}}$), and from the frame of the interruption onset to the duck and the stop.

4.3.2 End-of-Speech Detection and Speculative Start

Each frame is scored by the Silero (ONNX) voice activity detector. After 96 ms of silence (chosen on development utterances; the first run used 160 ms), a "speculative" turn starts: recognition, text generation and speech generation run, but their audio is held behind a gate. If the silence reaches 256 ms, the end of speech is confirmed and the held clauses are released in order; if the user speaks again before that, the speculative turn is cancelled and the speech is added to the same turn. The sample timeline is absolute over the whole session, so that the end-of-speech times of later turns stay consistent with the client's frames (a counter-reset bug found in loopback testing was closed with a regression test).

4.3.3 Speech Recognition

The user's turn is sent to the local speech recognition service: the hybrid Conformer model version 4 with a CTC decoder, which the second-version recognition comparison (Chapter 5) found to be the most accurate candidate within the latency budget. The hash of the model file is recorded in the server's health report and in every receipt.

A point that became important in the audit is the "choice of input channel". The dataset files are stereo, with channel zero the assistant's speech and channel one the user's. An earlier development panel had mistakenly given channel zero to recognition, that is, it in effect transcribed the reconstructed assistant voice. This defect was found and fixed before any validation row was run, and the protocols require the use of the *audited* channel one.

4.3.4 Streaming Text Reply Generation

The reply is generated by Qwen3-4B-Instruct-2507 [62] with the locked second-version prompt; the hashes of the model tree and the prompt are recorded in every receipt. The model is served by

vLLM as a stream, and the last two dialogue turns are given to it as history. The "Persian script only" constraint is no longer applied after generation with retries, but during decoding with regular-expression-based structured output: the model simply cannot produce a Latin character. In loopback testing on the shared host, the first token arrived with a median of about 40 ms after recognition finished.

The comparison of responder candidates and a "speech-input" path that removes recognition (Gemma 4 E4B, Section 4.5.4) did not reach the pre-registered replacement threshold, so this responder was kept in the 12 GiB stack. A third comparison (Section 5.6.3) selected, for a 24 GB card, Gemma 4 12B with Google's 4-bit weights served by vLLM; the live server offers it as a separate preset with the same prompt for older users, the same Persian-only decoding and at most 96 new tokens. It failed one of the four checks of its deployment qualification (coherence after interruption), so it is opt-in and not the default.

4.3.5 Clause Chunking and Speech Generation

The streamed text is cut at clause boundaries: the first fragment as soon as it reaches 2–4 words (at the first boundary or at most four words), and later fragments at Persian end-of-clause marks (؛, «.», ؟, «!», ։) or at most 14 words. Each fragment is synthesized immediately with the Piper engine and the Persian Mana voice on the CPU (ONNX Runtime with eight threads and deterministic settings) and sent; generating the first four words takes about 60–100 ms. In the live server, a Piper failure raises an error and does not fall back to formant synthesis. The comparison of voices (six Persian Piper voices, ParsVoice-XTTS and the lab VITS model; Chapter 5) found the Mana voice the most intelligible candidate.

4.4 Full-Duplex Control and Interruption Detection

While the assistant speaks, the microphone frames are given to a two-stage controller. The first stage is fast and reversible, the second more accurate and slower.

4.4.1 Stage One: Ducking

A frame must meet two conditions to count as the "user's voice": the Silero detector must consider it speech, and it must be louder than the expected echo of the assistant's own voice. The expected echo is derived from the playback level reported by the client: the maximum level of the last eight frames (to cover the delay of the echo path) plus a coupling factor; the frame must

be at least 14 dB louder than that (a Geigel-style double-talk detector). The coupling factor is assumed conservatively as 0 dB at the start of each reply and approaches its real value gradually while the user is silent; initializing it from the first playback frame, which arrives before the delayed echo, was the main cause of spurious ducking at the start of replies in the simulation. Without a reference level, a rolling noise floor plus 12 dB is used. One frame of user voice is enough to duck the assistant; this is the "detect and adapt the ongoing reply" reaction that the proposal requires within 150 ms.

4.4.2 Stage Two: Lexical Confirmation with Echo Rejection

After ducking, the speech recorded since the onset is transcribed by the same recognition service: after 400 ms of speech, then every 200 ms until a decision is made, and also when the user falls silent (320 ms). From the normalized transcript, these words are set aside: listener tokens (a list of 35 words such as آره, بله, اوهوم, خب), closed-class function words (prepositions, pronouns, demonstratives, conjunctions) that say nothing about a wish to take the turn, and the words of the reply the assistant is playing, which are the residual echo; echo matching accepts identity, one edit for words of three letters or more, and a prefix or suffix of three letters or more, because words at the edge of the recording window are clipped. If any word remains, the interruption is accepted: playback stops, generation is cancelled, and the recorded speech with a 200 ms pre-roll becomes the start of the user's next turn. The interrupted reply stays in the dialogue history only as far as the user heard it: with the stop acknowledgement, the client reports how much of the reply had played before the user started speaking, and the server drops the rest of the reply from the history and ends it with «…», so that the next reply does not assume the user heard all of it. If, until the user is silent or until 1.6 seconds of speech, the transcript is empty or contains only listener tokens, the volume returns to normal and the reply continues. The recognition time is added to the decision time, and its call runs outside the server's event loop.

The reason for this design was the result of the development runs of the simulation (Chapter 5): confirmation based on speech duration or on an acoustic classifier (the features of Section 2.1 with a gradient-boosting classifier or the multilayer perceptron of the proposal) cannot separate long backchannels, loud noise and residual echo from interruptions. The acoustic classifiers are still implemented and were measured in the second-version detector test.

4.4.3 Three Levels of Evaluation

Interruption detection is measured at three levels: (1) a classifier test on interaction events from podcasts with independent two-pass automatic Gemini labels; (2) a deployment simulation built from real recordings, in which the microphone hears the user's speech together with the assistant's residual echo, background noise and a room response (the room impulse responses and point-source noises of the OpenSLR collection [58], whose noises are taken from MUSAN [56]); (3) loopback testing with a real browser and a fake microphone. To avoid circular reasoning, the backchannels of the simulation and of the loopback test were selected with a different recognition model (SeamlessM4T v2), not with the model that performs the lexical confirmation.

4.5 Design of the Direct-Track Experiments

4.5.1 Training Objective and Acceptance Condition

The direct track is low-rank adaptation of Moshika with its official trainer. The input data are the stereo files of Section 4.2: channel zero the reconstructed assistant speech and channel one the user's speech, together with the text alignment that the "inner monologue" mechanism needs. The total loss is a weighted combination of the text-token loss and the audio-token loss of each codebook.

The central methodological decision of this section was to define a "runtime acceptance condition" before training started. For each candidate checkpoint, the adapter is loaded in the model's official streaming server, not in a local script, and for each of nine predefined input rows several conditions must hold at once: the model must produce text, it must produce non-silent speech, and the generated text must pass the Persian-script constraint. A checkpoint is "eligible" only if it passes all 9 rows.

This condition is strict, and deliberately so. The experience of the first version showed that the checkpoints with the lowest validation loss give almost silent audio without text in autoregressive generation. If checkpoint selection rested on loss alone, that useless artifact would have been reported as a "Persian model".

4.5.2 Experimental Generations

The experiments were designed as a chain of falsifiable hypotheses, and in each generation we tried to change "only one factor". Table 4.1 summarizes this design.

Table 4.1: Design of the seven experimental generations of the direct track and the hypothesis each generation tested

Version	Changed factor	Tested hypothesis	Rank
v1	base configuration, 20-second window, all embeddings trained	the model learns Persian speech with enough data	64
v2	text window of 100 seconds	a short window does not fit the start of the reply in context	64
v3	all embeddings frozen and a higher rank	training the embeddings damages the base representation	128
v4	only the two text embeddings trained	only the Persian text space needs adaptation	128
v5	first-codebook loss weight from 100 to 10	a heavy semantic-codebook weight stifles acoustic learning	128
v6.2	in-sample capacity test with text-input dropout and a 12-second window	the model has capacity and the problem is exposure bias	64
v7a	romanized phoneme inner-monologue text with word-level timing (Section 4.5.3)	the English tokenizer breaks the one-token-per-frame budget	64

The first three versions follow the standard path of hyperparameter tuning. Versions four and five test sharper hypotheses that came from analysing the failures of the earlier versions: if the problem is a missing Persian text representation, selectively training only the two text embeddings should suffice; and if the problem is the semantic loss dominating the acoustic loss, lowering the first-codebook weight should fix it.

Version six has a different design and is deliberately a "falsification test". In this version the model is trained on only 32 samples with a 12-second text window and evaluated on the *same* 32 samples. The aim is not to measure generalization but to answer a two-way question: does this adapter have the *capacity* to learn the target or not? If the model cannot reduce the loss

enough even on its own training data, the problem is capacity; if it can but fails in autoregressive generation, the problem is exposure bias (Section 2.4.2). Version v6.2 also adds "scheduled dropout of the text input": the share of reference text tokens hidden in training rises gradually from 0.25 to 0.75, so that the model learns to rely on its own predictions.

4.5.3 Version Seven: Testing the Root Cause in the Tokenizer

The final audit of the project showed that the six earlier generations had not tested a structural factor. Moshi produces at most one text token per 12.5 Hz frame, and the "inner monologue" text must run slightly ahead of the audio. The model's English SentencePiece tokenizer needs on average 6.6 tokens per Persian word, and 55% of the tokens are byte fallbacks (only 8 of the 33 Persian letters have their own piece); as a result, 98.5% of assistant turns needed more than one token per frame, and turn-level alignment hid this shortfall. The same speech, if the espeak-ng phonemes that Piper actually pronounced are written in Latin letters, needs 2.7 tokens per word, and only 0.3% of turns exceed the frame budget.

Version v7a changes exactly two things relative to the capacity test of version six: the inner-monologue text is a deterministic romanization of the spoken phonemes, and the time of each word comes from the exact alignment of Piper's phonemes. The same 32 samples, the same nine-row panel and the same "all nine rows" condition were used; by the pre-registered protocol, full training (v7b) and opening the sealed test were allowed only if this condition was passed. PersonaPlex was also examined; it has the same architecture and tokenizer, learned only English and its training code is not published, so the same root cause holds for it and it was not trained.

4.5.4 The Speech-Input Track: Gemma 4

The second direct track is a language model that hears audio directly and removes speech recognition. Gemma 4 E4B (the QAT w4a16 quantized version) accepts audio input of up to 30 seconds. Three arms were compared with the deployed system: recognition followed by text Gemma, Gemma hearing the audio directly (zero-shot), and the same with a speech LoRA adapter. The adapter, with rank 16 on the attention and feed-forward layers of the language model (audio encoder frozen), learned two tasks: transcription, and DiVA-style "self-distillation", that is, replying to the audio the way the model replies to its text transcript. The training data were 3,817 user turns (11.2 hours from 137 training sessions), and no session of the evaluation panels was in them. The quantized weights were converted back to bf16 for training, and the adapter was served on the same quantized base. Judging used the panels and criterion

of the second-version responder comparison, and the decision rule was registered before the adapter was trained: the speech path counts as positive only if its mean relevance is at most 0.10 below the deployed system and its time to first token is lower.

4.6 Evidence-First Method

A large part of the project's methodological contribution lies in how results are *reported*. The experience of the early attempt (Section 4.1) showed that an artifact can have a decreasing loss and look successful while supporting no meaningful claim. To avoid repeating this situation, five operating rules were defined and enforced in code.

One. Prior definition of the protocol. For each evaluation, a protocol document with the exact indices of the test rows, the acceptance thresholds, the model revisions and the file hashes is written and committed to the repository before running. An acceptance threshold is never set after seeing the result.

Two. Separating the qualification stage from the final test. Semantic evaluation has two stages. First a qualification stage runs on a small number of validation rows. Only if this stage is passed completely is the final test opened on the test rows, *once*. This design prevents selection after seeing the result. The judge of this evaluation is Qwen3.8-27B-FP8 at revision 017b9c7af6b5689d5dd426a76e0bc077eb5ca20a, at temperature zero, with blind arm order and two repetitions per row. Because the judge and the candidate are of the same family, the result stays in the class of "same-family automatic approximation" and does not prove a requirement of the project definition.

Three. Write-once artifacts. Each final result is recorded in a structured file containing the measured values, the status of each acceptance condition, the input hashes, the hardware details and an explicit list of limitations. This file is not overwritten.

Four. Evidence classes. Each measurement is labelled with an "evidence class". Table 4.2 defines these classes. The basic rule is that only the official class can prove that a requirement of the project definition is met.

Table 4.2: Evidence classes and the claim power of each class

Evidence class	Required conditions	Proves a requirement
Official end-to-end	live session with user consent, playback confirmation by the client, real microphone and eligible hardware	yes
Unofficial live	same protocol, but the hardware or study conditions are not eligible	no
Server-side component	generation or detector timing, without proof of the client's audible behaviour	no
Synthetic approximation	generated audio or simulated events	regression test only
Same-family automatic approximation	judging by an LLM of the candidate's family	no

Five. Fail-closed aggregation. An aggregator component reads all result artifacts and builds the overall status of the project. This component is *fail-closed*: if the artifact supporting a condition is missing, or its hash does not match the expected input, or its evidence class is below what is needed, the condition is reported as "failed", not "unknown". The component also explicitly prevents an approximation from being promoted to official evidence.

The consequence of this way of working is that the evidence status of each condition, as reported in Chapter 5, is stated independently of whether the report is ready for presentation and examination. The final result is a clear separation of what was built from what was proven with official evidence.

4.7 Implementation and Reproducibility

The project's code is organized as a Python package with a single command-line interface that offers each stage of the data pipeline, training, evaluation and serving as a separate subcommand. The model implementations rest on PyTorch [63] and the Transformers library [64]. Two separate virtual environments are kept: one for the main system and one for the direct-track stack, which has stricter dependencies.

Three mechanisms support reproducibility. First, an “upstreams” lock file that records the exact revision and weight hashes of each external model and repository. Second, a fixed random seed for each experiment, reported in the result artifact. Third, an automated test suite with continuous integration that runs code style checks, type checks, a dependency vulnerability audit, an audit of the committed artifacts and test coverage.

The reproduction commands of each stage are listed in Appendix A.4.

Chapter 5

Experiments and Results

This chapter presents the measured results. For each result three things are stated: the observed value, the denominator and measurement conditions, and the "permitted claim boundary" based on its evidence class (Table 4.2). All numbers in this chapter are read from the project's recorded and hashed artifacts, and none is an estimate.

5.1 Coverage of Acceptance Conditions and Evidence Status

Table 5.1 places the status of the project's "official" acceptance conditions, as produced by the fail-closed aggregator, next to the automatic evidence of the final stage. The official conditions are deliberately strict (human labels, physical target hardware, a complete human study) and the automatic evidence does not replace them; the third column shows how far each condition is covered by automatic evidence.

Five engineering conditions have sufficient official evidence. For the three conditions of the detector, latency and memory, the automatic evidence meets the numeric criteria of the proposal, but with automatic labels and on an H100; the direct model is negative and the human study was not conducted.

5.2 Dataset

Table 5.2 shows the final characteristics of the dataset.

The 108.584 hours lie within the 100–200 hours of requirement R4, and the automatic integrity audit, including checks of channel order, file hashes and group splits, passed without failure.

Table 5.1: The project's official acceptance conditions and the automatic evidence of the final stage

Acceptance condition	Official status	Final automatic evidence
Working Persian speech-to-speech prototype	sufficient	streaming system, browser loopback test
Passing the final automatic semantic test	sufficient	–
Audited export within 100–200 hours	sufficient	–
Data policy resolved under a documented review waiver	sufficient	–
Source code licence chosen	sufficient	–
Deployable direct Persian model	negative	Moshi v7a: 6/9, unintelligible speech; Gemma speech input: negative
Detector with accuracy above 80% on independent labels	not human	simulation 0.933/0.880; browser 0.825/0.817
Fit and live latency on 12–24 GB hardware	not target hardware	memory 11.1 GiB; mean 372 and 90th percentile 431 ms on an H100
Completion of the human study	not conducted	registered instrument and protocol; no participants were available

Independent verification of the labels. The turn-boundary and overlap labels were measured with an independent automatic listener (Gemini 3.8 Flash, two prompts with different wording and agreed labels only) on 300 pairs of the second balanced version; the sample was drawn from the 1,353 pairs whose source audio was still on disk after the audio was moved to cloud backup (mostly two of the four channels). At 93.8% of turn boundaries, the independent listener heard another speaker start (two-level confidence intervals of about 87–97%). The marked overlaps were heard as overlaps in 75.7% of cases, but the listener also heard overlap at a third of the unmarked boundaries; that is, diarization under-reports short overlaps. The turn captions differ from the listener's verbatim transcript by a character error rate of 15.9%. The agreement of the two prompts was 71–73%.

This result is an automatic *audited export. The interaction labels are automatic candidates and the number of human-verified rows is zero; this dataset can therefore not be called "human-labelled"; the verification above is also automatic and was run on an available sample. The source data are not redistributable.*

Table 5.2: Characteristics of the derived Persian conversational dataset and its automatic audit

Indicator	Value
Caption inventory of the channels	775.887 hours
Long programmes in the inventory	1,442
Selected programmes	309
Candidate windows	1,129
Candidate hours after selection	219.946 hours
Automatically classified multi-speaker hours	207.154 hours
Response pairs without reuse	6,754
Source audio hours of the pairs	123.796 hours
Speakers	407
Sessions	162
Exported stereo pairs	6,754
Exported stereo hours	108.584 hours
Train / validation / test split	6,419/131/204
Group leakage	0
Missing files in the audit	0
Automatic audit failures	0
Automatic interaction-event candidates	770
of which interruption-like / backchannel-like	717/53
Human-verified rows	0

5.3 Cascade System: Mechanical Panel

The validation panel is nine evenly spaced rows of the 131-row validation manifest. The indices of these rows were locked before running. The chain run is the real speech recognition service, the local Qwen2.5-0.5B-Instruct responder and the real speech engine, and the input channel is the audited user channel one. This panel measures the mechanics of the chain, not the semantic quality of the Qwen3-4B responder of the delivered system. The aggregate result of the panel is in Table 5.3.

Table 5.4 gives the details of each row. The time column is the duration of "full reply generation", not the latency of the first audio.

A retrospective, privacy-preserving descriptive analysis was also run on the same report; it reads only aggregate values and hashes and does not read or publish transcript or reply text. Its

Table 5.3: Result of the fixed validation panel of the cascade

Indicator	Value
Rows of the validation manifest	131
Split unit	source session identifier
Split leakage	0
Predefined evaluated rows	9
Text responder of the panel	Qwen2.5-0.5B-Instruct
Rows passing every condition	9/9
Rows with a rule-based fallback reply	0/9
Rows with a language retry	0/9
Recognition service errors	0
Language model load or generation failures	0/9
Minimum Persian-character share in the transcript	1.000
Minimum Persian-character share in the reply	1.000
Non-silent audio replies	9/9
Minimum RMS of the reply audio	0.133529
Final-test rows used	0

findings are: no mechanical failure occurred; the median, 95th percentile and maximum transcript lengths were 445, 3977.8 and 5031 characters; the median, 95th percentile and maximum full generation times were 5584.736, 13045.919 and 13270.249 ms; three rows had transcripts of more than 1000 characters and two rows generation times of more than 10 seconds.

The most important engineering risk observed is the growth of full generation time with input length. The descriptive Pearson correlation between transcript length and full generation time is 0.885, and the longest transcript and the slowest turn were both row 32. Figure 5.1(a) shows this relationship.

This is a positive mechanical, out-of-sample result on a group-disjoint panel with the Qwen2.5-0.5B responder. A nine-row panel is not enough to prove population generalization or semantic quality and is not generalized to the Qwen3-4B responder of the delivered system. The reported correlation is descriptive, not inferential. Full generation time on the available hardware is not the T_{first_audio} measure and cannot meet the 500 ms condition.

Table 5.4: Measurements of each row of the cascade validation panel

Manifest row	Transcript characters	Reply audio duration (s)	RMS	Full generation time (ms)	Pass
0	35	4.644	0.184925	4895.3	yes
16	541	5.352	0.162841	5473.7	yes
32	5031	7.535	0.210650	13270.2	yes
48	1237	9.857	0.191641	7414.3	yes
65	445	9.288	0.162643	6664.3	yes
81	2398	7.465	0.133529	12709.4	yes
97	77	3.448	0.193290	2072.4	yes
113	405	5.259	0.191777	3674.2	yes
130	142	8.824	0.154569	5584.7	yes

5.4 Automatic Approximation of the Intelligibility of Generated Speech

To measure whether the generated speech preserves the content of the reply text, a re-recognition measurement was run on the same nine outputs of the chain with the Qwen2.5-0.5B responder: the generated reply text is the reference and the recognition service's transcript of the output waveform the hypothesis. The protocol of this measurement, including normalization, token definitions and validity conditions, was registered before running, and all input hashes were reproduced. The result is reported in Table 5.5.

Figure 5.1(b) shows the distribution of these two error rates row by row. The striking gap between the character error rate and the word error rate is interpreted in Section 6.2.

This measure is an automatic approximation of "content preservation" with a single voice, on the output of the Qwen2.5-0.5B responder. Because the recognition service takes part both in the system and in the measurement, and because nine outputs are not enough to estimate a confidence interval, this result does not prove human intelligibility, correct pronunciation, naturalness or semantic quality and is not generalized to the Qwen3-4B responder. No pass threshold was set after seeing the result.

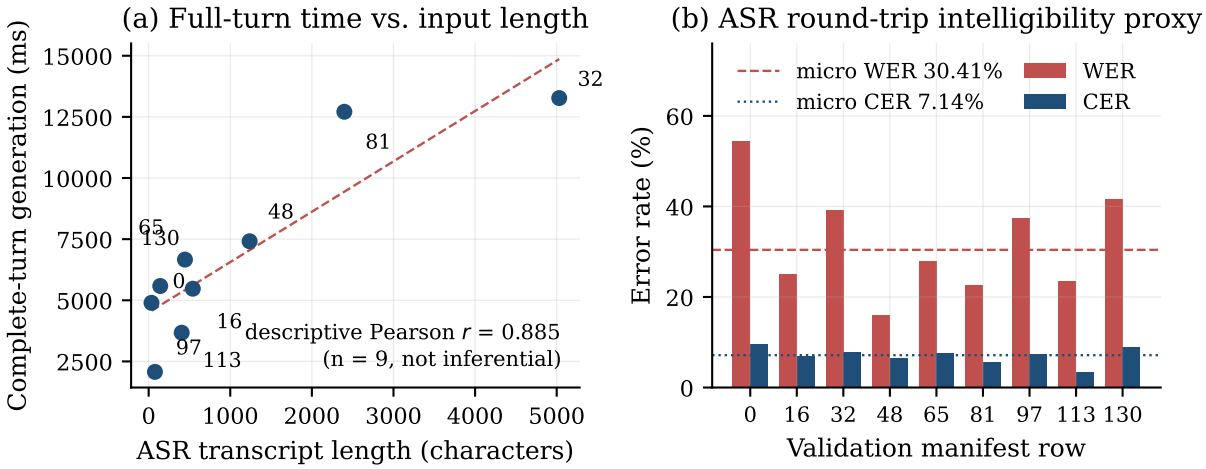


Figure 5.1: (a) Full reply generation time against input transcript length for the nine rows of the validation panel, with the fitted line and the descriptive Pearson correlation. (b) Word and character error rates of re-recognition for the same nine outputs, with the micro-aggregated values.

5.5 Automatic Semantic Test

5.5.1 Qualification Stage

The design of this evaluation has two stages. The first stage ran on seven previously unused validation rows and passed every condition: all 42 judge calls were valid, the candidate's mean relevance was 3.857 against 1.000 for the baseline, a relevance gain of 2.857, a mean coherence of 3.571 with a gain of +2.286, a paired win rate of 85.7% (6/7) and a rate of relevance of at least two of 100%. Only after this complete pass was the final test opened, *once*.

5.5.2 Final Test

The final test ran on 40 rows of the 204-row test manifest, from five held-out source sessions. Three arms were given to the judge double-blind and in randomized order: the locked Qwen2.5-0.5B baseline, the Qwen3-4B candidate with prompt v2, and the automatically aligned reference turn of the next speaker. The judge was Qwen3.8-27B-FP8 at revision 017b9c7af6b5689d5dd426a76e0bc077eb5ca20a, at temperature zero with two repetitions per row. Each arm and each row received two judge calls for relevance and coherence. Table 5.6 shows the result of this test.

Table 5.7 shows that all seven predefined automatic conditions are met. No threshold changed after seeing the result.

Table 5.5: Result of the automatic re-recognition approximation of the generated speech

Indicator	Value
Valid rows / recognition service errors	9/0
Reference words / word edits	171/52
Micro-aggregated word error rate	30.41%
Macro-averaged word error rate	32.01%
Word error rate, median / 95th percentile / maximum	28.00%/49.40%/54.55%
Reference characters / character edits	658/47
Micro-aggregated character error rate	7.14%
Macro-averaged character error rate	7.13%
Character error rate, median / 95th percentile / maximum	7.50%/9.40%/9.62%
Rows with an exact word match	0/9
Rows with an exact character match	0/9

Passing seven automatic conditions does not mean perceptual quality or an independent benchmark. The judge and the candidate are of the same family, and this result stays in the class of automatic approximation.

Figure 5.2 shows the mean score of each arm and the full distribution of scores. A notable point in part (b) is the bimodal distribution of the baseline: the 0.5-billion-parameter model received the full score on some rows and zero on a larger share. The automatically aligned reference arm also has the lowest score; this observation is interpreted in Section 6.3.

This is a positive automatic *result judged by a language model of the candidate's family. It does not prove human evaluation, an independent dialogue benchmark, factuality or safety, naturalness, population generalization, target-hardware fit or browser latency.*

5.5.3 Responder Adaptation with LoRA

A side experiment examined whether low-rank adaptation of the text responder on the response turns of this dataset improves semantic quality. Training ran successfully and the development loss decreased: the response-token loss of the base model was 3.723, after the first epoch 2.643 and after the second 2.562, that is, a 31.18% reduction relative to the base model. The second-epoch checkpoint was selected on the lowest development loss.

Table 5.6: Result of the final automatic semantic test on 40 group-disjoint rows

Indicator	Value
Test rows	40
Source sessions	5
Valid / expected judge calls	240/240
Failed judge calls	0
Judge repetition agreement	1.00
Candidate mean relevance	3.150
Baseline mean relevance	1.425
Aligned reference mean relevance	0.950
Candidate mean coherence	3.325
Baseline mean coherence	1.525
Relevance gain over the baseline	+1.725
Coherence gain over the baseline	+1.800
Paired relevance wins	27/40 (67.5%)
Relevance of at least two	35/40 (87.5%)
Candidate generation retries	0
Invalid candidate audio	0

But semantic evaluation on the forty test rows showed that this adapter does not beat the baseline. The result was recorded as a "valid negative result" and the adapter was not used in the final system. The interpretation of this apparent contradiction is in Section 6.3.

5.6 Re-Selection of Components on Fresh Panels

The final audit showed that the earlier speech recognition panels were no longer independent (newer versions of the recognition models had been trained on the same YouTube channels), the responder judge was of the candidates' family, and the voice had never been measured with more than one automatic listener. All three components were compared again under new, pre-registered protocols.

Table 5.7: Predefined automatic conditions of the final test and the observed values

Condition	Threshold	Observed	Status
Candidate mean relevance	≥ 2.00	3.150	met
Candidate mean coherence	≥ 2.50	3.325	met
Relevance gain	≥ 0.50	1.725	met
Coherence gain	≥ 0	1.800	met
Paired relevance win rate	≥ 0.60	0.675	met
Rate of relevance at least two	≥ 0.70	0.875	met
Full mechanical validity	240/240	240/240	met

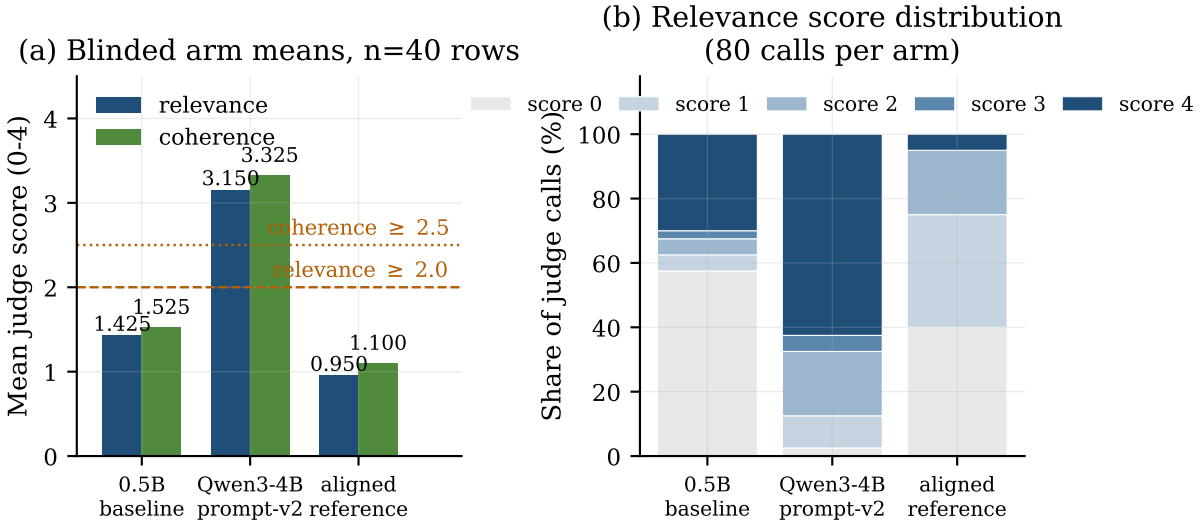


Figure 5.2: (a) Mean relevance and coherence scores of the three blind arms, with the predefined thresholds. (b) Full distribution of the relevance score over the 80 judge calls of each arm.

5.6.1 Speech Recognition on Human Transcripts

The reference of this comparison is the public PSRB benchmark: 344 clips with human transcripts from various sources (47 clips of older speakers and 183 noisy, telephone or reverberant clips) on which none of the candidates was trained. Scoring used a normalizer without punctuation. Latency is the time for a single clip on an H100, and the latency budget of the streaming system was set to 150 ms (Table 5.8).

Seamless is the most accurate candidate but takes more than four times the latency budget; among the candidates within the budget, NeMo version 4 with a CTC decoder has the lowest error and stayed in the system. Seamless was used as the "second independent listener" in measuring speech intelligibility and in selecting backchannels. The error of every candidate is higher on older speakers (15.6% against 8.6% for the deployed candidate).

Table 5.8: Comparison of speech recognition candidates on 344 PSRB clips with human transcripts

Candidate	CER	WER	CER (senior)	Median latency
NeMo v4 CTC (deployed)	9.44%	23.95%	15.63%	58 ms
NeMo v6 CTC	10.15%	24.34%	16.36%	60 ms
NeMo v4 RNNT	13.40%	26.81%	22.10%	61 ms
omniASR CTC 1B	14.04%	36.38%	14.87%	56 ms
Seamless v5 fine-tune (accuracy reference)	**6.92%**	**21.03%**	**10.38%**	671 ms
Seamless v6 (step 55000)	8.99%	23.28%	10.44%	855 ms
Qwen3-ASR 1.7B	20.47%	49.46%	24.97%	1466 ms
Gemma 4 E4B (audio input)	21.04%	42.67%	26.16%	2632 ms
Whisper large-v3	23.02%	47.31%	25.46%	1101 ms
Second round (v3): Persian checkpoints not compared before				
Shenava Koochik (NeMo CTC)	8.95%	35.34%	14.50%	65 ms
BuzzASR (Whisper fine-tune)	15.76%	36.92%	19.99%	579 ms
Rade-ASR-CTC-3B	16.11%	37.33%	30.04%	122 ms
NVIDIA Persian FastConformer	19.22%	45.54%	26.43%	69 ms
Whisper large-v3-turbo, Persian	20.22%	39.21%	22.25%	484 ms
Whisper large-v3, Persian v4	21.54%	41.86%	24.51%	1427 ms
Whisper base, Persian	23.30%	49.14%	28.21%	1027 ms
NeuraSpeech 900h	25.42%	48.73%	37.42%	79 ms
Third round (v4): models released or trained later				
Nemotron Persian, 1.12 s	9.01%	24.69%	15.32%	268 ms
Nemotron Persian, 0.32 s	9.80%	25.98%	16.03%	218 ms
NeMo v8 (epoch 0)	10.18%	24.48%	16.18%	62 ms

A second, pre-registered round added the eight Persian checkpoints that had not been compared on this panel, among them Rade-ASR-CTC-3B, a Persian fine-tune of the three-billion-parameter omniASR model whose download had earlier been left incomplete for lack of disk space. The rule was unchanged, with one addition: a replacement must be significantly better than NeMo version 4. Shenava Koochik, a NeMo model of similar size, is the most accurate candidate within the latency budget, but its gain of 0.50 points has a 95% interval of -1.07 to $+0.04$ and therefore does not replace the deployed model; its word error is also much higher, mostly because of word spacing. Rade, the largest Persian-specialized model, has a character error rate of 16.1%; most of its errors are speech it leaves out (14.0% of the characters, against

5.2% for NeMo), especially at the start of clips and for older speakers (30.0%). The results on its model card, on read speech, did not carry over to this mostly spontaneous panel.

A third pre-registered round tested three models that the earlier rounds lacked: a Persian fine-tune of NVIDIA's Nemotron streaming recognizer (released before the second round but missed by its search), the best checkpoint of a later NeMo training run (v8), and, to check whether 344 clips are enough, a second panel with human transcripts: the 871 read sentences of the FLEURS Persian test set. The rule gained one condition: a replacement must also not be significantly worse on FLEURS. Nemotron with a 1.12-second look-ahead has a slightly lower error than NeMo version 4 (9.01% against 9.44%), but the gain is not significant (-1.00 to $+0.15$), although it is on FLEURS (5.05% against 5.28%), and in the library it was run with it needs 215 ms per short clip, above the budget. NeMo version 8 is significantly worse on both panels. NeMo version 6 was significantly worse than version 4 on PSRB ($+0.71$ points, $+0.40$ to $+1.02$; version 4 makes fewer errors on 111 clips and version 6 on 71) and the two are tied on FLEURS, so the second panel confirms that version 6 is never the better choice. NeMo version 4 therefore stays. NVIDIA's large Nemotron-3.5-Transcribe model, the leading open model on English leaderboards, does not support Persian, and public ASR leaderboards rank English (and a few European languages), so they cannot settle the choice for Persian; the claim is "best among the 24 arms tested on these two panels", not best possible.

5.6.2 Responder with a Panel of Elderly Requests

The second version of the responder comparison has two panels: 60 conversational requests in the voice of an older user in eight categories (health and medication, family, daily life, information, emotional support, home safety, culture and religion, technology help), written with the AI assistant Claude before the replies were generated (not collected from older speakers), and 60 real user turns from sessions that were in no panel or training. The pre-registered replacement rule was a gain of at least 0.25 in mean relevance, no loss in coherence or elderly appropriateness, and a win on at least 55% of rows under **both** judges: the local Qwen3.8-27B and Gemini 3.8 Flash from another family (Table 5.9).

The two judges disagree about the deployed system itself: Gemini rated the relevance of its replies 2.38 and the Qwen judge 3.29, and the exact agreement of the two judges was only 44.6%. Under Gemini, Qwen3.5-4B and Gemma 4 with the elderly prompt are better by a wide margin (gain intervals 0.71–1.23 and 0.93–1.39), but not under the Qwen judge; so, by the rule, no replacement was made. Family bias alone does not explain this disagreement, because Qwen3.5-4B is of the same family as the deployed system and the Qwen judge. This result shows

Table 5.9: Candidate responders under two judges (scale 0–4; gain relative to the deployed system)

	Qwen3.8 judge		Gemini judge	
Arm	Relevance	Appr.	Relevance (gain, win)	Appr.
Qwen3-4B, prompt-v2 (deployed)	3.29	2.83	2.38	2.02
Qwen3-4B, elderly prompt	2.91	2.98	2.83 (+0.44, 47%)	2.57
Qwen3.5-4B, elderly prompt	**3.31**	**3.58**	3.36 (+0.98, 58%)	3.55
Gemma 4 E4B, elderly prompt	3.00	3.49	**3.54** (+1.16, 60%)	**3.77**

that the absolute level of the earlier semantic test, which was measured with the Qwen judge only, depends on the judge; only human scores can resolve the disagreement. Qwen3.5-4B with the elderly prompt is the strongest replacement candidate.

Replacement rule and deployment qualification. Before any participant could receive the listening study, a rule was registered: the deployed responder changes to Qwen3.5-4B with the elderly prompt only if participants prefer its replies (Section 5.13) and this same configuration passes a deployment qualification. The qualification was run with the same panels and the same browser loopback test and met all four of its conditions: a whole-stack peak memory of 11,326 MB (limit 12,288); the main panel with no unanswered turn, a mean of 345, a 90th percentile of 398 and a maximum of 487 ms (deployed system: 372, 431 and 491), and a 90th percentile of 393 ms for elderly speech; coherence after interruption under both judges (relevance 2.73 against 2.70 under Gemini and 3.27 against 3.07 under Qwen, with all 30 interruptions stopping the reply); and no Latin letter in 428 replies. Despite its later first token, the new responder sounds sooner, because two thirds of its replies open with the short address «عزیزم،» and the first clause is complete sooner. The replacement decision therefore depended only on the preference section of the listening study, which was not conducted; by the rule, the deployed responder is unchanged.

Larger responders. All the responders above have at most four billion parameters, because the stack was kept inside a 12 GiB card. The proposal's hardware range goes up to 24 GB, and at four bits much larger models fit it. A pre-registered comparison therefore asked whether a larger responder gives better replies than the best 4B (Qwen3.5-4B with the elderly prompt). Five larger models of 9 to 36 billion parameters, four of them at four bits, were served on the same path as the 4B arms (vLLM, greedy decoding, the Persian-only output constraint) under a 22 GiB memory limit. The 4B arms were generated again on that path. All replies were scored on the same 120 rows by the same two judges, under the replacement rule above with the best 4B

as the reference (Table 5.10). Gemma 4 31B at four bits does not fit: its weights take 18.7 GiB and leave 0.05 GiB for the key-value cache, while one 2048-token request needs 1.73 GiB.

Table 5.10: Larger responders under a 22 GiB memory limit (120 written requests and turns; scale 0–4; gain relative to the best 4B, Qwen3.5-4B)

			Relevance (gain)		Median time	
Arm	Params	Weights	Qwen3.8	Gemini	First token	Reply
Qwen3-4B (deployed)	4.02B	bf16	3.25	2.46	38 ms	170 ms
Qwen3.5-4B (best 4B)	4.66B	bf16	3.23	3.35	68 ms	267 ms
Gemma 4 E4B	8.61B[a]	4-bit	2.88	3.50	54 ms	240 ms
Qwen3.5-9B	9.65B	bf16	3.24 (+0.01)	3.48 (+0.13)	66 ms	609 ms
Gemma 4 12B	12.0B	4-bit	3.04 (−0.19)	3.61 (+0.26)	56 ms	449 ms
Gemma 4 26B-A4B[b]	25.8B	4-bit	2.77 (−0.46)	3.55 (+0.20)	53 ms	201 ms
Gemma 4 31B	31.3B	4-bit	does not fit			
Qwen3.5-35B-A3B[b]	36.0B	4-bit	3.13 (−0.10)	3.49 (+0.14)	121 ms	292 ms

[a] Stored parameter count; the name refers to about 4 billion "effective" parameters. [b] Mixture of experts, with about 4 (Gemma) and 3 (Qwen) billion parameters active per token.

No larger model beats the best 4B. Under the Qwen judge none gains more than 0.01 in relevance, and Gemma 4 26B is worse, with an interval of -0.66 to -0.26. Under Gemini every larger model gains, and Gemma 4 12B reaches the threshold ($+0.26$, interval 0.08 to 0.44). Even so, it wins only 26% of the rows, because most rows score the same. The judges split along the model families again: Gemini prefers the Gemma models and the Qwen judge the Qwen models, and their exact agreement on relevance is 52%. The larger Gemma models do score higher on coherence and elderly appropriateness under both judges, but not on relevance, the primary measure. On this panel, then, size did not buy reply quality, and no 24 GB profile was qualified. Latency does not rule the larger models out: every first token arrives within 53–121 ms. The third version of the comparison, below, revisits this question with judges from other families and finds the opposite.

5.6.3 Responder Comparison Version Three: Quantized Builds and Two Memory Profiles

The comparisons above left four gaps: both judges came from candidate families and agreed on fewer than half of the scores, the panel had 120 rows, few quantized builds were tried, and the serving settings were fixed. A third pre-registered comparison closed them (protocol frozen before any reply was generated, with five dated amendments). It covered 15 models of the

Qwen3.5, Qwen3.6, Qwen3.8 and Gemma 4 families, from 0.8 to 35 billion parameters, with every quantized build that vLLM and llama.cpp can run, official and community. Two judges from families with no candidate scored the replies listwise from 0 to 10: Claude Opus 5.5 (J1) and GPT-6 Astra (J2). Every call contained one planted reply written for another row, and a judge counted as unreliable if that reply scored 6 or more in more than 10% of calls; the highest rate in any stage was 8.6%. The panels were new: 100 development and 252 test rows, half of them requests of an older user written by the two judge families and half real user turns with their history. Two profiles were defined: 12 GiB, with the responder within 10,682 MiB so that the whole stack fits a 12 GiB card, and 24 GB, with the responder within 22,528 MiB. Both require a 2,048-token context and two concurrent requests, and a latency gate of a median first token of at most 150 ms and at least 30 tokens per second.

Selection was adaptive. A model screen ranked the best build of each model. A quantization screen on Qwen3.5-4B and Gemma 4 E4B found that the GGUF builds from Q4_K_M up, FP8, INT8 and MXFP4 lose nothing measurable, that most 4-bit vLLM formats (AWQ, GPTQ, AutoRound and NVFP4) lose 0.5 to 0.9 points, and that three bits or fewer lose more; the official 4-bit Gemma build trained with quantization (QAT) is the exception. The models within half a point of the best fitting one went on, their builds and key-value cache precisions were compared, one finalist was chosen per model and profile, and the finalists were measured and scored on the test panel (Table 5.11).

Table 5.11: Finalists of responder comparison version three on the test panel (252 rows; scale 0–10; latency on the shared H100 at the deployment configuration)

Model	Weights, engine	Peak memory	First token	J1	J2
Gemma 4 12B (24 GB winner)	4-bit QAT, vLLM	10,914 MiB	39 ms	6.47	7.09
Gemma 4 12B (12 GiB finalist)	Q5_K_M, llama.cpp	9,772 MiB	179 ms[a]	6.70	7.18
Gemma 4 31B	Q5_K_M, llama.cpp	22,194 MiB	152 ms[a]	6.86	6.89
Gemma 4 26B-A4B[b]	NVFP4, vLLM	17,282 MiB	71 ms	6.24	6.54
Qwen3.8-27B	AWQ 4-bit, vLLM	21,150 MiB	113 ms	6.16	6.41
Qwen3.5-27B	NVFP4, vLLM	20,656 MiB	112 ms	6.15	6.16
Qwen3.5-4B (best 4B)	bf16, vLLM	9,634 MiB	56 ms	5.27	5.34
Qwen3-4B (deployed)	bf16, vLLM	9,392 MiB	35 ms	2.84	3.58

[a] Fails the latency gate. [b] Mixture of experts with about 4 billion active parameters.

In the 24 GB profile, Gemma 4 12B with Google's official 4-bit weights, served by vLLM, has the best score among the configurations that fit and meet the gate, and it beats both references under both judges by the registered rule: its overall score is 1.20 (J1) and 1.75 (J2) points above Qwen3.5-4B and 3.63 and 3.52 points above the deployed responder, every 95% interval is above

zero, and it loses nothing in relevance, coherence or appropriateness. In the 12 GiB profile no configuration both fits and meets the gate. The vLLM builds of Gemma 4 12B need 10.9 GB, about 200 MiB too much, and an exploratory test of serving settings did not bring them under the limit without dropping below 30 tokens per second; the llama.cpp build that fits takes 169 to 214 ms to its first token. Gemma 4 31B has the best mean score of the 24 GB finalists (6.87), but it missed the gate in six of its eight latency measurements (139 to 336 ms).

Two corrections were made before the result was final, and both versions are reported. The first decision named the FP8 build of Gemma 4 12B. A later check found that two builds the rules required had not been run, because the cache precision that the original selection assumed for them cannot start in vLLM 0.30. Neither fits its profile, but scoring all builds of Gemma 4 12B together again moved its 24 GB finalist from FP8 to QAT. On the test panel the two builds score the same (differences of -0.03 and $+0.12$, both intervals across zero), and QAT is smaller (10.9 against 15.4 GB) and as fast. Two sensitivity analyses were registered after the user's question about load on the shared GPU and about reasoning settings. Five more spaced latency measurements of every finalist, with the other processes' GPU utilization recorded once per second, changed no gate result: load from other users was present in only 3 of 40 measurements, where it could halve the decode rate, and the first-token time correlated only weakly with it. Scoring the first 60 test rows again with more judge reasoning (extended thinking for J1, high effort for J2) kept the rankings (rank correlations 0.93 and 0.98). The two judges also agree with each other: their configuration means have a rank correlation of 0.88, and 77% of their reply scores are within one point.

This result reverses the size comparison above. With judges from other families, a larger panel and a finer scale, every finalist of 9 to 31 billion parameters scores at least 0.8 points above Qwen3.5-4B under both judges, and Gemma 4 is the strongest family at every size. Natural user turns separate the configurations most: the two Qwen 27B models lose about 1.5 points on them relative to written requests, Gemma 4 12B the least.

Deployment qualification of the 24 GB winner. As the protocol specifies, the winner was then run as the responder of the live system and measured with the four checks of the replacement rule above, with a whole-stack memory limit of 22,528 MiB. Three checks passed. The whole stack peaked at 12,466 MiB (recognizer 1,610 and vLLM 10,856); on the plain panel no reply was missing, and the mean and 90th percentile from end of speech to first reply audio were 372 and 468 ms (elderly panel: 90th percentile 373 ms); none of 419 replies contained a Latin letter. Coherence after interruption failed: all 30 interrupted replies stopped, but the replies after the interruption were 0.37 points less relevant than the controls under both judges (Gemini 2.37 versus 2.73, Qwen3.8 2.50 versus 2.87), more than the 0.25 the rule allows. Both 95% intervals

include zero ($[-0.90, +0.13]$ and $[-1.10, +0.37]$), but the frozen rule compares the means, so the configuration is not qualified; it remains an opt-in preset of the live server, and the default is unchanged. The deficit lies in the late interruptions (relevance 2.0 versus 2.9 under Gemini, 1.9 versus 3.2 under Qwen3.8). An exploratory count made after this outcome shows why: when the recognizer garbles the follow-up, Gemma 4 12B asks the user to repeat, while Qwen3.5-4B guesses. Gemma asked in 14 of its 60 follow-up replies, Qwen3.5-4B in 3 of 60 on the same scenarios, and the follow-ups Gemma asked about had a median character error rate of 0.345 against 0.161 for the others. The written-request comparison had clean text input and could not show this. The judges, who see the intended follow-up rather than the recognizer's transcript, score a request to repeat as irrelevant; whether older users prefer it is a question for human evaluation.

Post-hoc checks in 12 GiB. Two addenda written after the decision, and before their measurements, checked whether Gemma 4 12B could still serve the 12 GiB stack, with the same four checks and a 12,288 MiB limit. With one sequence instead of two, the vLLM build fits (11,768 MiB), but one of 201 utterances received no reply (the end-of-speech detector never closed that turn) and coherence failed under the Qwen judge (-0.27). An exploratory diagnosis then traced the slow first token of the llama.cpp builds: Gemma 4 uses sliding-window attention, and with its default settings llama.cpp saves and restores checkpoints of that cache around every reused prompt. With a full-size window cache the first token took 24.8 ms instead of about 180 ms, at 10,088 MiB. In the live stack this build fit (11,700 MiB) and wrote no Latin letter, but failed latency (90th percentile 824 ms; the first 125 turns had a median of about 365 ms, then every stage slowed at once, the recognizer too, which points to other users' load on the shared host) and coherence under both judges (-0.33). In all three Gemma runs the model asked the user to repeat in 11 to 14 of 60 follow-ups, against 3 for Qwen3.5-4B. No Gemma 4 12B configuration was therefore qualified, and the 12 GiB stack keeps the deployed responder, with Qwen3.5-4B as its qualified alternative.

The next models in 12 GiB (exploratory). The stage 2 rule had kept only Gemma 4 12B as a 12 GiB contender, so a further exploratory addendum, written before any of their replies existed, ran the next two models that fit on the 252 test rows: Gemma 4 E4B (GGUF Q8_0) and Qwen3.5-9B (GGUF Q4_K_M), both on llama.cpp at about 5.9 GB. Judged in one group with the two references and Gemma 4 12B as an anchor (which reproduced its scores, 6.58 and 7.22), both beat Qwen3.5-4B by the decision rule under both judges: E4B by $+1.02$ and $+1.16$ (6.44 and 6.67), Qwen3.5-9B by $+0.68$ and $+0.55$ (6.10 and 6.06), all intervals above zero. Both met the latency gate in their development runs (first token 107 ms) but not in their test runs (158 to 167 ms), which fell in the same afternoon as the slowed qualification above. They are therefore

candidates for a later 12 GiB check, not results: they need a quiet latency measurement and their own deployment qualification, including coherence after interruption.

5.6.4 Voice with Two Automatic Listeners

All six Persian Piper voices were generated with the same 60 replies, and their intelligibility was measured by transcribing them back with two independent recognition models (NeMo and Seamless); DNSMOS is also reported as an automatic indicator of signal quality (not of Persian naturalness) (Table 5.12).

Table 5.12: Automatic intelligibility of the Persian Piper voices (mean character error rate of two listeners)

Voice	Mean CER	Δ vs Mana (95% CI)	DNSMOS OVRL	First 4 words
mana (deployed)	**5.82%**	–	3.13	84 ms
ganji_adabi	6.93%	$[-1.1, +4.9]$ pp	**3.40**	92 ms
gyro	7.30%	$[+0.1, +4.0]$ pp	3.30	61 ms
reza_ibrahim	8.39%	$[+0.6, +5.2]$ pp	3.26	102 ms
ganji	9.02%	$[-1.3, +11.2]$ pp	3.29	61 ms
amir	11.73%	$[+4.5, +7.6]$ pp	3.32	63 ms

The Mana voice is the most intelligible, and three voices are significantly less intelligible than it. ganji_adabi is on par in intelligibility and has the best DNSMOS score; these two voices were therefore chosen for the human listening study to measure naturalness.

In the third version, ParsVoice-XTTS (XTTS-v2 fine-tuned on 2,200 hours of Persian speech, cloning the ManaTTS speaker's voice from two CC0-licensed recordings) was also measured. On a typical clip it is as intelligible as Mana (median clip character error rate 5.6–6.2%), but about one clip in ten goes off track and produces meaningless speech; the mean error of the two listeners was 13.55%, significantly worse than Mana (interval +0.9 to +19.3 percentage points). Its first streaming audio chunk also takes 307 ms on an H100. By the pre-registered rule, it therefore did not enter the listening study.

The lab VITS model (star.text_to_speech) was measured with the same panel and rules once the checkpoint of its ezafe detection was available. As currently served, the mean error of the two listeners was 9.89%, significantly worse than Mana (interval +1.4 to +8.1 percentage points), although its DNSMOS score (3.39) was higher. The model was then optimized without retraining under a pre-registered protocol: correct reading of numbers (a thousands separator that dropped part of a number, times, dates, blood pressure, fractions and units), a fix of a word-

segmentation bug that dropped the final verb «می‌خواهید», in-process espeak-ng with identical phonemization, identical but ten times faster resampling, running the model on the GPU, and tuning the sampling noise and speech rate on development texts. On the same 60 replies the optimized version reached an error of 9.19%, the highest DNSMOS of all voices (3.43) and a first-four-words time of 116 ms on an H100 (416 ms on the CPU, against 538 ms for the current service), but it is still significantly less intelligible than Mana (interval +0.8 to +8.1 percentage points) and, by the same rule, did not enter the listening study. The remaining gap comes from the acoustic model and the text front end it was trained with, not from inference settings.

A fourth, pre-registered round measured three newer Persian voices with the same texts, listeners and Mana reference voice, each fed clause by clause as in the live system: Chatterbox Persian (500 million parameters, non-commercial licence), Gooya v1 and MOSS-TTS-Nano-Persian. Chatterbox is as intelligible as Mana (6.47% against 5.82%, interval -2.2 to $+3.3$ points) with a DNSMOS just below it (3.12 against 3.13), so by the rule it would not have entered the study, and it needs about 2 seconds on an H100 to synthesize four words, against 84 ms for Mana on a CPU. Gooya v1 (18.2%) and MOSS (21.4%) are significantly less intelligible and as slow. Because the listening study was not conducted, this round could not have changed the deployed voice; it shows that, among the twelve voices and configurations tested, none is both as intelligible and as fast as Mana.

5.7 Interruption Detection

5.7.1 Synthetic Test

On synthetic data, the gradient-boosting classifier reached on 32 held-out samples (with 128 training samples) an accuracy of 1.000, an interruption-class F1 of 1.000, a false accept rate of 0.000 and a false reject rate of 0.000, while the energy and zero-crossing baseline had an accuracy of 0.375, an F1 of 0.545 and a false accept rate of 1.000. The 95% Wilson interval for the synthetic accuracy is 89.28–100%. The inference time per step on the CPU was 7.931 ms, which fits within the 20 ms budget of each frame.

Perfect accuracy on synthetic data is not a meaningful result about real performance; the synthetic data are built from the same generative model that makes the features separable. This number is of value only as a software regression test and is not presented in this report as evidence of the detector's effectiveness.

5.7.2 Approximation on Recorded Data

The meaningful evaluation was on events drawn from the real audio of the dataset. Of 1,730 discovered events, 1,048 were selected and split into three parts by session. The decision threshold was chosen on 24 validation sessions only (value 0.70) and then applied once to 22 test sessions with 132 events. The result is in Table 5.13.

Table 5.13: Result of the interruption detector approximation on recorded audio events with held-out sessions

Measure	Proposed detector	Energy baseline	95% interval of the detector (session-level block)
Accuracy	81.06%	78.03%	74.44% – 87.18%
Interruption F1	78.99%	71.84%	72.34% – 85.71%
False accept rate	9.09%	0.00%	2.63% – 16.92%
False reject rate	28.79%	43.94%	19.30% – 37.23%

The confusion matrix of the proposed detector on the test set is: 60 true negatives, 6 false positives, 19 false negatives and 47 true positives. The event-level 95% Wilson intervals are 73.53%–86.83% for accuracy, 4.23%–18.45% for the false accept rate and 19.27%–40.64% for the false reject rate; the F1 interval at the same level is an event-level bootstrap (69.47%–86.33%). Figure 5.3 shows the threshold search on the validation sessions and the final comparison with the baseline.

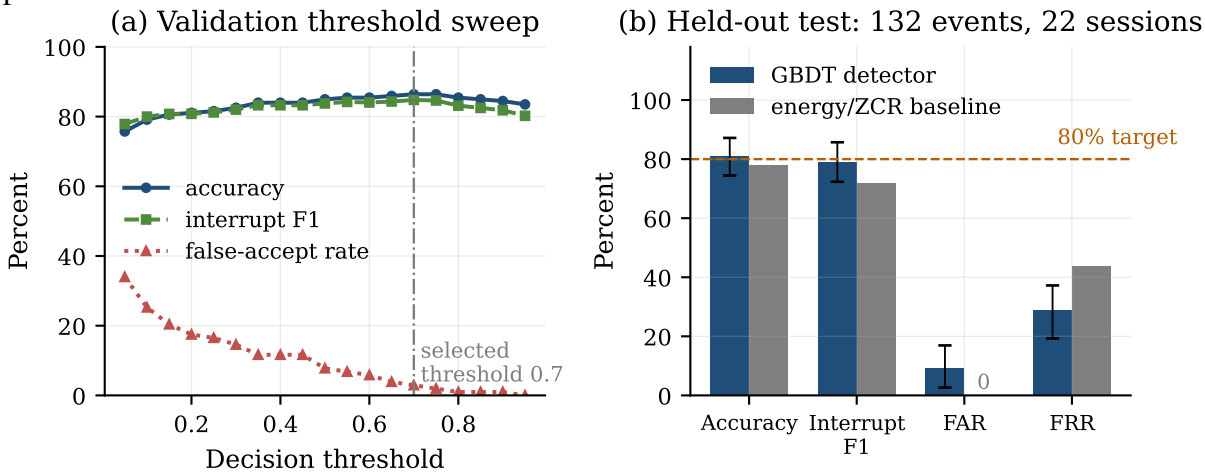


Figure 5.3: (a) Search for the decision threshold on the validation sessions; the vertical line is the selected threshold of 0.70. (b) Comparison of the proposed detector with the energy baseline on the test sessions; the error bars are session-level block confidence intervals.

Against the baseline, the proposed detector shows a clear improvement in the F1 of the interruption class and a marked reduction of the false reject rate. The energy baseline has a

false accept rate of zero, but at the cost of rejecting 43.94% of real interruptions, which in an interactive system means repeatedly ignoring the user.

The point accuracy of 81.06% is above 80%, but the lower bound of the session-level block confidence interval is 74.44% and falls below 80%. More importantly, the reference labels of this evaluation come from automatic speaker diarization and have no independent human review. Requirement R5 is therefore "not proven" by this evidence.

5.7.3 Deployment Simulation

The deployed controller (Section 4.4) was measured in a simulation that reconstructs the microphone during playback of the assistant's reply: residual echo of the reply (18–32 dB below the user and 20–60 ms later), background noise, a room response and one real event at the start, middle or end of the reply: an interruption (real Persian speech), a real backchannel, a real noise burst, or nothing. Six development runs used only the audio of the training split (Table 5.14), and the chosen variant was run once on 600 scenarios of the test split (Table 5.15).

Table 5.14: Development runs of the simulation (training split only)

Run	Change	Dev accuracy / F1
dev3	duration/MLP confirm + double-talk detector	0.687 / 0.624
dev4	lexical ASR confirm with echo rejection	0.840 / 0.765
dev5	+ coupling prior, wider echo matching, function words	0.920 / 0.862
dev6	+ one-frame duck (latency tie-break)	0.913 / 0.856
dev7	+ 200 ms confirm pre-roll (not kept)	0.900 / 0.840

Table 5.15: One-time result of the simulation on 600 test scenarios (95% Wilson intervals)

Measure	Value
Accuracy (interrupt = positive)	**0.933** (0.910–0.951)
F1	**0.880**
Interruptions stopped (recall)	0.980 (0.943–0.993)
Stop rate: backchannel / noise / no event	0.087 / 0.093 / 0.067
Onset $\rightarrow$ duck, median / p90	**120** / 247 ms (66% $\leq$ 150 ms)
Onset $\rightarrow$ stop, median / p90	786 / 1344 ms

Accuracy and F1 are both above 0.80, and the median time to duck is 120 ms. A complete stop takes longer (median 786 ms), because the controller waits until a word is heard; during this interval the assistant's volume is ducked. The largest jump in the development runs came from two places: the lexical confirmation (which separates backchannels and echo from interruptions) and fixing the initialization of the double-talk detector's coupling factor. The seventh

run, after the evaluation, tested adding 200 ms of audio before the onset to the input of the lexical confirmation: more interruptions were stopped (0.98 versus 0.95) but more backchannels and noise too, so by the same rule it was set aside.

This result is on simulated mixtures of real recordings, not live users; the echo is modelled as attenuated and delayed assistant speech, and the labels come from extraction rules and a recognizer with a list of listener tokens, not from people.

5.8 Continuity of the Full-Duplex Transport

The automatic regression test of the WebSocket layer passed with four tests. These tests confirm that the 3,200-sample pre-roll before the interruption is combined with the 1,600 samples received after it to build a 4,800-sample input for the next turn. They also cover receiving the stop acknowledgement from the client, runtime identity telemetry, turn cancellation, concurrent connections, reconnection, isolation of session state, truncated or silent input, recovery from generation and out-of-memory errors, and a metrics-only retention mode that writes no audio file.

The final repository audit also recorded 206 passing tests with 65% branch coverage (against a required 60%) and type checking without errors on 49 files.

This evidence proves continuity at the level of the server transport. A physical browser run and a target-hardware run were not performed; requirement R2 therefore remains met in software and officially unproven.

5.9 Direct Track: Seven Experimental Generations

5.9.1 Overall Result

Table 5.16 summarizes the seven experimental generations. The common pattern of all generations is the same: the loss decreases and the autoregressive runtime condition is not met.

The minimum loss values are "not directly comparable" across generations, because the text window, the validation set and the number of evaluation samples differ between generations. For versions v6.2 and v7a the loss is computed on the training data itself and is an in-sample measure. This column is shown only to illustrate the downward trend of the loss within each generation.

Figure 5.4 sets these two axes against each other: offline loss decreased in every generation, but no generation reached the condition of 9 of 9.

Table 5.16: Summary of the seven experimental generations of the direct track. The "best run" column is the largest number of the nine rows of the runtime panel passed among all checkpoints of that generation.

Version	Window (s)	Candidates	Minimum loss	Best run	Eligible
v1	20	16	1.402	0/9	no
v2	100	20	1.735	2/9	no
v3	100	5	1.879	1/9	no
v4	100	5	1.879	1/9	no
v5	100	5	1.868	1/9	no
v6.2	12	4	2.060	0/9	no
v7a	12	4	1.264	6/9	no

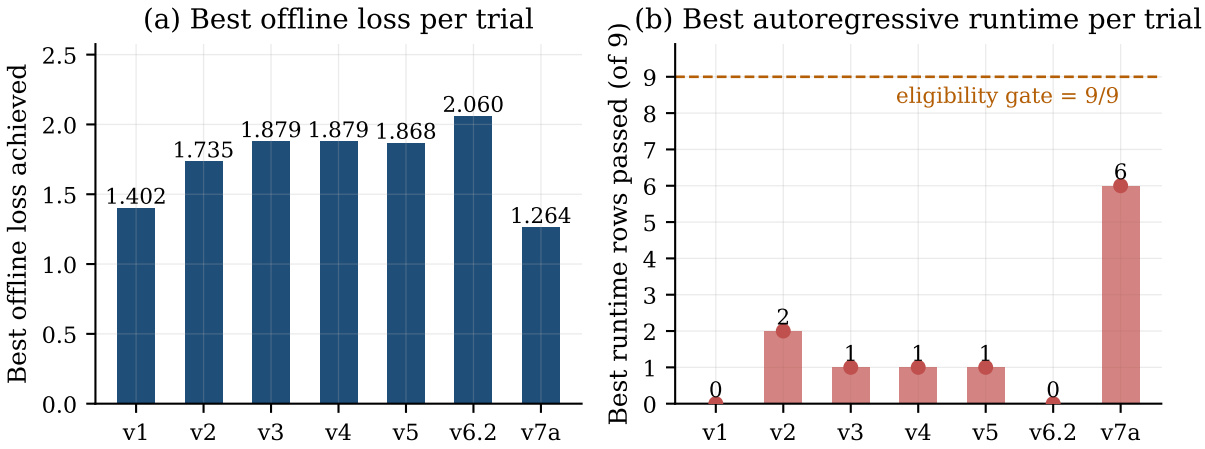


Figure 5.4: (a) Best offline loss of each generation. (b) Largest number of rows passing the nine-row runtime panel in each generation, against the acceptance condition of 9 of 9.

5.9.2 Version One: Positive Held-Out Loss, Failed Runtime

Version one was trained for up to 8000 steps with a 20-second window. Figure 5.5(a) shows the training loss and part (b) the re-evaluated validation loss of 16 checkpoints. The validation loss decreased monotonically, and the checkpoint of step 8000 with a loss of 1.402444 was selected.

On the held-out test set with 204 rows and 331 segments, this adapter gave a total loss of 1.7272 with a 95% confidence interval of 1.5866–1.8678, against 3.5670 for the unadapted base model and 9.9871 for a control group in which the sign of all B matrices of the adapter was flipped. That is, in terms of the training objective, the adapter improved significantly over both the base model and the control. The total loss is the sum of the mean cross-entropies of the

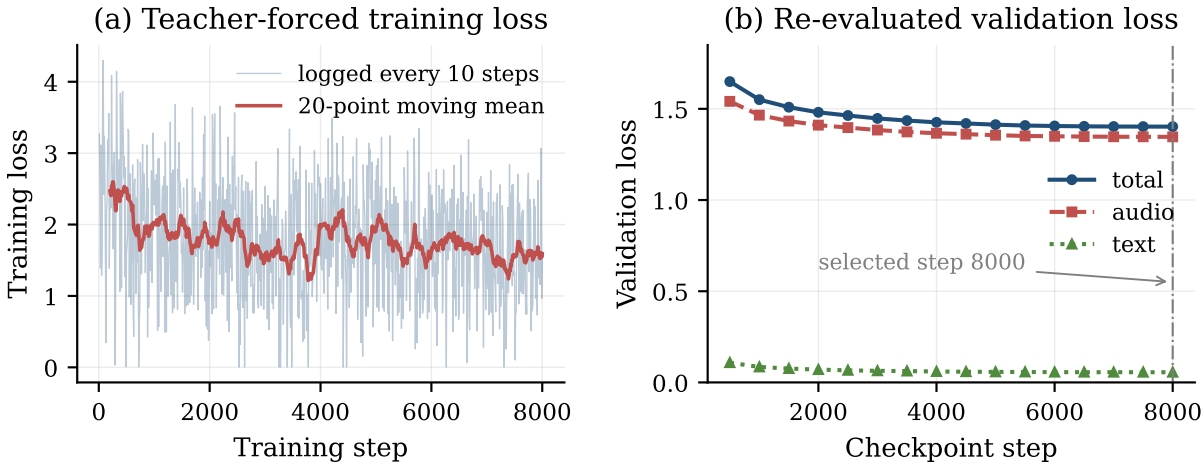


Figure 5.5: (a) Teacher-forced training loss of version one, with a rolling mean. (b) Re-evaluated validation loss by text and audio component; the vertical line is the selected checkpoint.

text and audio tokens; in terms of perplexity (the exponential of the mean loss), the perplexity of the text tokens fell from 2.53 to 1.19 and that of the audio tokens from 13.98 to 4.71. This is the "reduction of perplexity on a Persian test set" that the second objective of the proposal asks for; the second part of that objective, "intelligible output", was not met.

Nevertheless, the same selected adapter produced almost silent audio and no text in the official streaming server. This contradiction is the observation that made the runtime acceptance condition necessary for the following generations.

5.9.3 Version Two: The Clearest View of the Gap

Version two, with a 100-second text window and 20 candidate checkpoints, is the clearest illustration of the central pattern of this project. Table 5.17 shows several points of this trend and Figure 5.6 gives the full picture.

Table 5.17: A sample of version-two checkpoints: the validation loss decreases monotonically while the number of rows passing the runtime panel does not increase

Step	Total loss	Text loss	Audio loss	Panel passes
100	2.312407	0.402789	1.909618	0/9
400	1.894230	0.201763	1.692467	2/9
800	1.792216	0.164635	1.627582	1/9
1000	1.768085	0.155691	1.612394	1/9
1500	1.739114	0.144936	1.594178	0/9
2000	1.734574	0.143414	1.591161	0/9

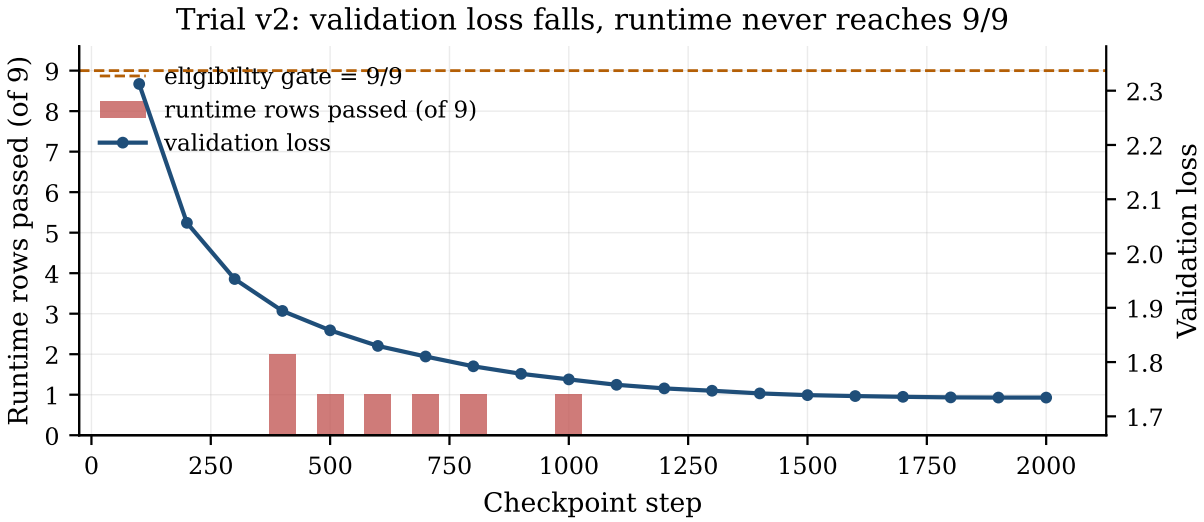


Figure 5.6: Version two: the validation loss on 202 segments decreases monotonically over the checkpoints (line), while the number of rows passing the nine-row runtime panel (bars) never reaches the acceptance condition.

The decisive point is that the best runtime behaviour occurs at step 400, where the loss is *higher*, and as the loss keeps falling until step 2000, the runtime behaviour gets *worse* and reaches zero. That is, in this setting the validation loss is not even positively correlated with generation quality.

5.9.4 Versions Three to Five: Testing Targeted Hypotheses

The next three versions tested specific hypotheses (Table 4.1). Version three froze all embeddings and raised the rank to 128. Version four trained only the two text embeddings, and version five lowered the first-codebook loss weight from 100 to 10. Table 5.18 shows the row-by-row result of the last two versions.

Table 5.18: Result of each checkpoint of versions four and five on the nine-row runtime panel

	Version four			Version five		
Step	**Loss**	**Speech**	**Pass**	**Loss**	**Speech**	**Pass**
100	2.082125	9/9	1/9	2.082676	9/9	0/9
200	1.948315	7/9	0/9	1.943459	7/9	0/9
300	1.899640	4/9	0/9	1.889862	6/9	0/9
400	1.881632	5/9	1/9	1.870719	5/9	1/9
500	1.879061	5/9	0/9	1.867859	4/9	1/9

The common pattern of these two tables is notable: as the loss decreases, the number of rows that produce *any speech at all* falls from 9 to 4 or 5. That is, while the model fits the training objective better, it leans more towards silence in autoregressive generation. The peak allocated memory of version four was 28.640 GB.

All three versions closed "fail-closed" with zero eligible checkpoints, no adapter was selected, and the firewall of the final test was never opened.

5.9.5 Version Six: Separating Capacity from Generation

Version v6.2 is an in-sample falsification test: training and evaluation on the same 32 samples. The result, as shown in Table 5.19 and Figure 5.7, is two-sided and clear.

Table 5.19: Version v6.2: a positive in-sample learning signal against a negative autoregressive runtime result

Step	Total loss	Text loss	Audio loss	Panel passes
50	2.636405	0.812474	1.823931	0/9
100	2.245344	0.638987	1.606357	0/9
150	2.071542	0.550852	1.520690	0/9
200	2.060297	0.551183	1.509114	0/9

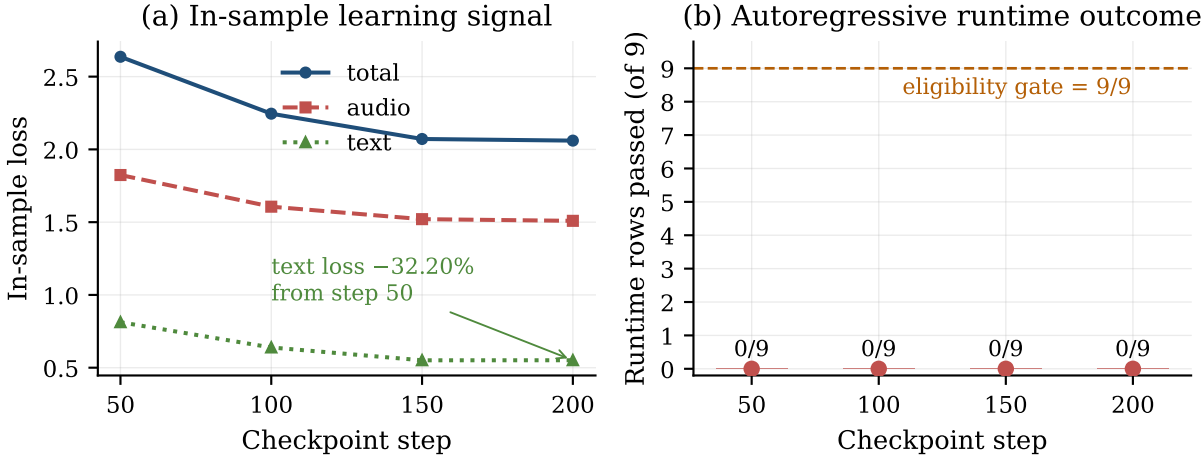


Figure 5.7: Version v6.2: (a) the in-sample loss decreases steadily, and the text loss falls by 32.20% from step 50 to the best later step. (b) Nevertheless all four checkpoints pass 0 of the 9 rows of the runtime panel.

The best text loss after step 50 is 0.550852, which is 32.20% lower than the value at step 50 (0.812474) and above the predefined criterion of 30%. That is, this adapter has the *capacity* to fit the training objective.

At the same time, all four checkpoints pass 0 of the 9 rows of the runtime panel, and in each checkpoint only one of the nine rows produces text and speech together. The precise conclusion is:

> Version v6.2 learned the limited in-sample training objective, but the learned behaviour did not transfer to reliable autoregressive generation of Persian speech and text.

No checkpoint of any of the first six generations is eligible for deployment, and no final test was opened for the direct track. The statement "Moshika was successfully fine-tuned for Persian" is false *on this evidence. The correct conclusion is "a negative experiment with a positive learning signal".*

5.9.6 Version Seven: Removing the Tokenizer Bottleneck

Version v7a, on the same 32 samples and the same nine-row panel, showed the largest improvement in autoregressive generation of all the Moshi experiments (Table 5.20). The in-sample teacher-forced text loss went from 1.124 at step 50 to 0.0149 at step 150, and from that step on the official server produced audible speech (RMS about 0.07–0.19) with fluent pseudo-Persian text on 8 of the 9 rows.

Table 5.20: Number of accepted rows of the nine-row panel in the in-sample capacity test

Experiment (same 32 rows)	Step 50	Step 100	Step 150	Step 200
v6 (Persian script, turn-level span, sampled text)	0/9	1/9	1/9	1/9
v6.1 (v6 weights, greedy text; controlled baseline)	0/9	0/9	1/9	3/9
v6.2 (Persian script + text-input dropout)	0/9	0/9	0/9	0/9
v7a (romanized phonemes + word timing)	0/9	0/9	**6/9**	4/9

Still, the "all nine rows" condition was not met: three rows of step 150 were rejected with prefix errors of 0.76 and 0.91 (slightly above the 0.75 limit) and one row that was always silent. The pre-registered secondary measure is negative too: the speech of step 200 has a mean prefix error of 0.88 with the deployed recognizer, and no row reaches the 0.75 limit. By the protocol, full training of v7b was therefore not started and the sealed test was not opened. The result is two-sided: the tokenizer bottleneck is real, and removing it turned almost silent output into audible speech with text, but it is not the only obstacle; despite a near-zero teacher-forced loss, the generated text drifts from the memorized target, which is the sign of the same gap between teacher forcing and free generation (Section 2.4.2).

5.10 The Speech-Input Track

Table 5.21 compares the three speech-input arms with the deployed system on 60 natural turns and 60 elderly requests (with a simulated elderly voice: two male Piper voices, speed 0.85 and room noise).

Table 5.21: The speech-input track against the cascade (Qwen3.8-27B judge)

Arm	Rel. nat.	Rel. eld.	Rel. all	Coh.	First token
ASR → Qwen3-4B (deployed)	**2.85**	2.75	**2.80**	3.36	86 ms
ASR → Gemma 4 text	2.03	**2.95**	2.49	3.69	98 ms
Gemma 4 audio, zero-shot	1.65	2.32	1.98	3.60	**67 ms**
Gemma 4 audio + LoRA	1.97	2.13	2.05	**3.81**	200 ms*

* Serving overhead of an unmerged adapter in vLLM.

The adapter lowered the held-out loss from 1.296 to 0.712, but the relevance of the best speech arm is 0.75 below the deployed system (difference interval -1.05 to -0.46); by the pre-registered rule the result is therefore negative, and the cascade stayed. Three points follow from this comparison. First, removing recognition lowers quality even with the same language model (2.49 versus 1.98); the bottleneck is Gemma's understanding of Persian speech, not Persian text generation. Second, self-distillation on in-domain speech helps but hurts outside the domain: on real turns, the adapter closed most of the gap to its text teacher (1.65 to 1.97, teacher 2.03), but on the requests with a synthetic voice it got worse (2.32 to 2.13). Third, the time saving is negligible, because CTC recognition adds only about 20 ms to the time to first token.

With the second judge (Gemini), the picture is reversed: the deployed system gets the lowest relevance (1.91), text Gemma with the elderly prompt 3.00, and the speech path 2.18 zero-shot and 2.52 with the adapter. The decision rule was registered with the first judge and the official result stays negative, but this disagreement is the same disagreement of the two judges about the deployed responder. Under both judges, removing recognition lowers quality. The larger Gemma 4 12B does not understand Persian speech better either: its speech path is weaker than E4B under both judges (1.48 and 1.63), while its text mode is better.

5.11 Latency, Memory and Browser Loopback

The final system was measured with the automated loopback test of Section 6.4 of the proposal: a headless Chromium browser opens the real client page with a fake microphone that plays recorded Persian utterances of 1–5 seconds, and the audio travels the same path as a real user's.

Times were measured with the client's clock: from the frame holding the acoustic end of speech to the scheduled start of the first clause of the reply. The protocol was registered before running, and every planned turn (even an unanswered one) is in the denominator.

Memory. Requirement R6 is a 12–24 GB GPU, and such hardware was not available; this deviation is declared. To measure the "fit", the language model server was limited to a 0.105 share of the H100's memory, so that the GPU side of the whole system fits in a 12 GiB card (12,288 MiB). The peak measured in the final run was **11,336 MiB** (11.1 GiB): 9,730 MiB reserved by the language model and 1,606 MiB for recognition (10,994 MiB in the first run); the synthesizer and the voice activity detector run on the CPU. The compute speed, however, is the H100's.

Two runs. The first run (v1) used a speculative start after 160 ms of silence. Because the first clause was ready about 390 ms after the end of speech, that is, much later than the confirmation of the end of the turn (256 ms), starting the speculative work earlier lowers latency without changing when a turn is confirmed or cut. Under a pre-registered protocol, values of 64, 96, 128 and 160 ms were measured on 60 development utterances that were in no test panel, and the rule chose 96 ms (audible 90th-percentile latency 488 versus 541 ms for the previous value). The second run (v2) measured the same test panels again with this value; this is a second measurement of the same panels and is reported as such (Table 5.22).

Table 5.22: Latency from end of speech to the assistant's first audio in the browser loopback test (ms). Main panel: 61 PSRB utterances and 140 corpus utterances; elderly: the same method with speed 0.85 and 3 dB quieter; barge-in: the base turns of the two interruption panels. The last column adds the browser's reported output latency.

Panel	Run	Turns	Mean	Median	p90	Max	≤500 ms	Audible p90
Plain	v1	201	418	415	487	537	93.5%	529
	v2	201	**372**	375	**431**	**491**	**100%**	**473**
Elderly	v1	80	435	433	508	575	86.3%	550
	v2	80	**382**	371	**446**	546	98.8%	488
Barge-in	v1	120	399	388	464	509	99.2%	506
	v2	120	**366**	363	**415**	497	100%	457

In the final run no turn went unanswered, and the mean (372) and 90th percentile (431) of the main panel are below 500 ms; the strict reading "every turn below 500 ms" is also met for the main panel (maximum 491). Adding the browser's reported output latency (about 42 ms), the 90th percentile is 473 and 98% of turns are below 500 ms. The 90th percentile for simulated elderly speech is 446 ms, and only one turn of 80 exceeded 500 ms. The stage shares (medians,

from the speculative start): recognition 65, first token 42 ms after that, completion of the first clause 85 and its audio 56 ms, plus the 96 ms of silence before the speculative start.

Interruptions in a real browser. In 60 turns with a real interruption and 60 turns with a real backchannel (selected with an independent recognizer), 47 interruptions were stopped and 8 backchannels were stopped by mistake: accuracy **0.825** and F1 **0.817** (first run 50 interruptions and the same 8 wrong stops: 0.850 and 0.847; a difference of three events in 60 is within sampling variation). The median time to duck from the detected onset frame was 44 ms (90th percentile 60) and the median time to stop 507 ms; the time from the real acoustic onset in the simulation was 120 ms (Section 5.7.3).

The reply after an interruption. The proposal asks for interruptions to be simulated at the start, middle and end of the reply and for the coherence of the new reply to be measured. In 30 scripted dialogues, an elderly request and then a dependent follow-up question (such as «صبر کن، من زانوم درد می‌کنه، باز هم پیاده‌روی کنم؟», meaning "wait, my knee hurts, should I still go walking?") were played in a synthesized user voice: once over the assistant's reply (0.8, 1.8 or 3.0 seconds after the end of the request) and once as a normal turn after the reply had ended. Of the 27 interruptions that arrived during playback, 24 stopped the reply (start and middle 10 of 10, end 4 of 7). The two automatic judges rated the reply to the follow-up question equally in both conditions: relevance 1.67 against 1.67 under Gemini and 2.90 against 2.80 under Qwen, with the same use of the context of the first request; the pre-registered criterion (a drop of at most 0.25) was therefore met. The absolute level of the scores is the level of the responder itself. In the first run of this test, the synthesizer's pauses inside the user's sentences made end-of-speech detection close the requests prematurely; that run was not scored, and the sentences were synthesized again without internal pauses (this phenomenon also occurs in real speech; Section 5.12).

Semantic end-of-turn detection (tested twice and rejected). To reduce premature closing of a turn at mid-sentence pauses, a pre-registered "gate" was tested: at the speculative start, a model estimates the probability that the utterance is complete, and if it is low, confirmation of the end of the turn is postponed until 640 or 1024 ms of silence. Three zero-shot candidates (Smart Turn v3.2 [65], which does not support Persian, a completeness judgement of the transcript by the same Qwen3-4B, and a rule on words such as «که» and «را» with which a Persian utterance cannot end) and a version of Smart Turn fine-tuned on Persian speech were measured on 850 development trials (425 podcast utterances from sessions outside the test panels, plain and simulated elderly) with an exact replay of the end-of-speech detector. Without a gate, 19.8% of trials are closed prematurely. All candidates reduced this (the best zero-shot candidate to 13.3%), but none met the condition "delaying at most 5% of genuine ends" (lowest

13.1%, the lexical rule). Some of these delays are correct, because many of the development utterances are cut mid-sentence; but zero-shot Smart Turn is close to chance on Persian, and Persian data with natural turn endings were scarce (738 training examples). The first fine-tuned model was declared invalid because of a data artifact (the audio was cut at the moment of the speaker change), and the protocol was amended before the corrected run. No gate qualified in this first round.

A second pre-registered round fixed both data problems. Each development clip's ending was labelled by Gemini in two independent passes (158 complete, 224 unfinished and 43 uncertain endings), and false holds were counted only on complete endings. Smart Turn was also fine-tuned again on 5,498 examples mined from 233 hours of restored diarized windows (the label filter was cut short when the shared Gemini budget ran out). With correct holds no longer counted, the Qwen3-4B completeness judgement ($\theta = 0.3$, hold 640 ms) met the development rule: premature closings fell from 19.8% to 13.3% while 4.4% of complete endings were held. The retrained Smart Turn still held 29% or more. The gate then went to the loopback test, whose panels were opened for the first time. It split fewer utterances than before (17 against 22 over both panels, not significant), but it also held 44 finished turns (14.8%), and each hold delayed the reply by up to 384 ms. The 90th percentile of the latency rose to 656 ms on both panels, above the 500 ms limit, so the pre-registered rule removed the gate and the deployed detector stayed unchanged.

This automated loopback test is on the shared H100 host: the memory of a 12 GB card was respected but not its speed, and there was no human user and no physical microphone and loudspeaker path (and therefore no acoustic echo). This is not the target-hardware condition of the proposal.

5.12 Error Analysis

The proposal asks for the failure modes to be classified. Table 5.23 puts the failures of each stage side by side, from the receipts of this chapter.

5.13 Human Study

The instrument of the small listening study is a self-contained Persian page that runs without recording audio and without asking for a name: consent and age group, naturalness scores (1 to 5) for 8 sentences in the Mana voice, the same 8 sentences in the ganji_adabi voice, 6 samples of natural human speech as anchors and an attention check, and then three recorded dialogues with the live system (a question and answer, a backchannel in the middle of a reply, and an interruption

with a new question), with scores for response speed, reaction to the interruption, satisfaction and willingness to use. To arbitrate between the two automatic judges of the responder, eight elderly requests are shown with two replies (the deployed system and Qwen3.5-4B with the elderly prompt) in random order, and the participant picks the better reply. By a rule registered before any participant received the study, the deployed responder changes to Qwen3.5-4B only if participants prefer its replies (the confidence interval of the preference difference above zero, and no participant over 60 picking the deployed reply more often) and this same configuration passes the deployment qualification (memory, latency, coherence after interruption). The protocol and the method of analysis were registered in advance.

The study was not conducted: no participants were available before submission, so the page was never sent. The author's own ratings were not used in its place, because a single rater, who is also the author, cannot meet the registered sample (at least five participants, two of them over 60). The page and its analysis script are released and can be run as registered.

Author pilot. To check the page, its result code and the analysis script from start to finish, the author completed the page once as a participant, without knowing which system produced each clip or reply (age group under 40, on loudspeakers), and passed the attention check. Naturalness was 4.13 for both Mana and ganji_adabi and 5.0 for the human anchor. In the three dialogues response speed was rated 5, the reaction to the backchannel and to the interruption 3 and 4, and the answer to "would use" was yes each time. In the eight reply pairs the author picked Qwen3.5-4B six times, the deployed reply once and "no difference" once. These are one listener's ratings, from the author and not from the target age group; they are reported only as a pilot of the instrument and are not used for any requirement or decision.

Apart from the author's single-listener pilot, there is no human result: no claim is made about the naturalness of the speech, the experience of older adults or a human preference between the responders, and the criterion MOS $\geq$ 3.5 was not evaluated.

5.14 The Latency, Size and Quality Trade-off

Table 5.24 puts all measured candidates of each component side by side with their size, quality and latency (Question 3 of the proposal). The parameter counts were taken from the model files themselves.

Three patterns appear. First, for recognition and synthesis size does not buy quality. The two smallest recognizers (115 and 117 million parameters, both NeMo models fine-tuned on Persian) are more accurate than every larger candidate except Seamless. That includes Rade, a three-billion-parameter model fine-tuned on Persian. Seamless, the only model more accurate

than them, is itself fine-tuned on Persian and more than ten times slower. In synthesis too, the smallest model (Piper, 15.7 million parameters) is both the most intelligible and the fastest. Second, for the responder size does buy quality once the judges come from families with no candidate. The comparison judged by a Qwen model and Gemini found no larger responder better under both judges (Table 5.10), but the third comparison, judged by Claude and GPT-6 Astra, found every 9 to 31 billion parameter finalist at least 0.8 points better than the best 4B, and its 24 GB winner, Gemma 4 12B at four bits, about 1.2 to 1.75 points better while answering in 39 ms (Table 5.11). That gain was measured on clean text: in the live stack the same model asks the user to repeat when the recognizer errs and failed the coherence check after interruption, so it was not qualified. Beyond 12 billion parameters the gains are small and cost latency: Gemma 4 31B scores only 0.1 higher and misses the latency gate. Within a 12 GiB card, no responder better than the 4B ones meets both the memory limit and the latency gate. The speech-input model has only a latency advantage (67 versus 86 ms), which separate recognition in about 60 ms can make up for. Third, the largest model, the 7.7-billion-parameter end-to-end model, did not reach intelligible Persian speech with low-cost training. For real-time Persian with these resources, the best trade-off point is small recognizers and synthesizers specialized for Persian, with the largest responder that the memory profile allows at four bits.

5.15 Final Status of the Requirements

Table 5.25 sets the status of each of the seven requirements of Table 1.1 against the evidence obtained.

The evidence for R2, R3 and R5 is automated (browser loopback and simulation from real recordings) and on an H100, not a human test or target hardware. R4 has no human labels. R5 was not met by the acoustic features specified in the project definition alone; lexical confirmation with the speech recognizer replaced them.

Of the four expected outputs of the project definition, three have been produced and documented: a working prototype for the cascade track, documented source code with an open licence and an automated test suite, and the Persian conversational dataset. For the fourth item, the listening-study instrument and its method of analysis are registered and released, but the study was not conducted, so there are no empirical findings on older adults.

Table 5.23: Failure modes of the system and the measured cause of each

Failure mode	Frequency	Measured cause
Latency above 500 ms	0 of 201 main turns and 1 of 80 elderly turns (first run: 13 and 11)	in the slow turns, the stage furthest above its median was “completion of the first clause”, that is, token generation stalls on the shared GPU; neither recognition nor synthesis
Missed interruption	13 of 60 in the browser; 2% in simulation	4 times there was no duck (no frame louder than the expected echo); 9 times it ducked but the confirmation transcript had no content word and the reply continued
Premature turn end mid-sentence	15 of 201 main and 7 of 80 elderly utterances	a mid-sentence pause longer than the 256 ms end-of-turn confirmation; when the user goes on, the system stops the reply and keeps both fragments in the history, but the first, incomplete reply starts; a semantic end-of-turn gate was tested and rejected
Wrong stop on a backchannel	8 of 60 in the browser; 8.7% in simulation	the recognizer heard a listener token as a content word; an observed example: «آره آره» as «کار» (first phoneme clipped)
Recognition error	character error 9.44%; older speakers 15.6%	the speech of older adults (consistent with the literature, Section 3.5); a word error rate much higher than the character error rate because of spelling variation and joined writing (Section 6.2)
Irrelevant or weak reply	Gemini judge: 2.38 of 4	the two automatic judges disagree about the responder’s quality; only human judging is conclusive
Unintelligible synthesized speech	character error 5.82% (Mana)	residual pronunciation errors of the synthesizer; in the lab VITS model, the ezafe and suffixes after a ZWNJ
Unintelligible direct-track speech	prefix error 0.88 (v7a)	the tokenizer bottleneck and the gap between teacher forcing and free generation (Section 5.9)

Table 5.24: The size, quality and latency trade-off of the candidates (quality and latency from the tests of the previous sections)

Stage	Candidate	Params	Quality	Latency
ASR	NeMo v4 CTC (deployed)	117M	CER 9.44%	58 ms
	Shenava Koochik	115M	CER 8.95%	65 ms
	omniASR CTC 1B	976M	CER 14.04%	56 ms
	Rade-ASR-CTC-3B (Persian)	3.08B	CER 16.11%	122 ms
	Seamless v5 fine-tune	1.50B	**CER 6.92%**	671 ms
	Whisper large-v3	1.54B	CER 23.02%	1101 ms
	Qwen3-ASR 1.7B	2.35B	CER 20.47%	1466 ms
	Gemma 4 E4B (audio input)	8.61B[a]	CER 21.04%	2632 ms
Responder[b]	Qwen3-4B, prompt-v2 (deployed)	4.02B	2.80 / 1.91	86 ms
	Gemma 4 E4B, elderly prompt	8.61B[a]	2.49 / 3.00	98 ms
Speech input[b]	Gemma 4 E4B, audio in	8.61B[a]	1.98 / 2.18	67 ms
	+ speech LoRA	8.61B[a]	2.05 / 2.52	200 ms
Responder[d]	Qwen3.5-4B, elderly prompt	4.66B	3.23 / 3.35	68 ms
	four larger models	9.65–36.0B	2.77–3.24 / 3.48–3.61	53–121 ms
Responder[e]	Qwen3.5-4B, bf16	4.66B	5.27 / 5.34	56 ms
	Gemma 4 12B, 4-bit (24 GB)	12.0B	**6.47 / 7.09**	39 ms
	Gemma 4 31B, Q5_K_M	31.3B	6.86 / 6.89	152 ms
TTS	Piper Mana (deployed, CPU)	15.7M	**CER 5.82%**	84 ms
	Lab VITS, optimized (H100)	36.3M	CER 9.19%	116 ms
	ParsVoice-XTTS (H100)	–	CER 13.55%	1751 ms
	Chatterbox Persian (H100)	0.5B	CER 6.47%	1981 ms
Direct S2S	Moshika LoRA (v7a)	7.69B	unintelligible[c]	–

[a] Stored parameter count; the model name refers to about 4 billion "effective" parameters. [b] Relevance of the reply to the same 120 spoken turns; the first number is the Qwen judge and the second the Gemini judge; latency is the first token after the end of the audio. [c] Prefix error 0.88 with the deployed recognizer. [d] Relevance on the 120 written requests and turns of Table 5.10, Qwen judge / Gemini judge, under a 22 GiB limit; latency is the first token of the reply text. Gemma 4 31B did not fit. [e] Overall score (0–10) on the 252 test rows of Table 5.11, J1 / J2 (judges from families with no candidate); latency is the first token of the reply text.

Table 5.25: Final status of the seven requirements of the project definition and the class of the supporting evidence

ID	Status	Supporting evidence
R1	met (streaming cascade)	40-row final semantic test; all 401 loopback turns were answered
R2	met in loopback and simulation	interruption accuracy/F1 0.83/0.82 in the browser and 0.93/0.88 in simulation
R3	met by the proposal's criterion and, on the main panel, by the every-turn reading	mean 372 and 90th percentile 431 ms; all turns below 500; with the browser's output latency a 90th percentile of 473
R4	met	108.584 hours of audited export (automatic labels)
R5	accuracy goal met by another method	two-stage controller with lexical confirmation; energy/pitch/MFCC classifier alone 0.574
R6	declared deviation	whole-system memory measured at 11.1 GiB; speed on an H100
R7	not met (study not conducted)	registered instrument and protocol; no participants were available

Chapter 6

Discussion and Limitations

This chapter interprets the results of Chapter 5. Four key observations are examined in detail, and then the limitations and the ethical and legal considerations are listed.

6.1 The Gap Between Reducing Loss and Valid Generation

The most important finding of this project is a pattern that repeated in all seven experimental generations of the direct track: the teacher-forced loss decreased monotonically, while the autoregressive runtime condition was never met. Version two is the clearest illustration of this pattern, because its best runtime behaviour occurs at step 400, where the validation loss is *higher*, and as the loss falls further until step 2000, the runtime behaviour reaches zero (Figure 5.6).

There are three possible explanations for this observation, and the project's evidence helps to separate them.

Explanation one: a lack of learning capacity. The low-rank adapter might lack the capacity to represent Persian. Version v6.2 was designed specifically to falsify this hypothesis: training and evaluation on the same 32 samples. The 32.20% reduction of the in-sample text loss shows that the adapter has the *capacity* to fit the objective. This explanation is therefore rejected.

Explanation two: exposure bias. The explanation more consistent with the evidence is the concept introduced in Section 2.4.2. In training, the model always receives the "correct" previous text and audio tokens as its condition, and predicting one step ahead is an easy problem, especially in the "inner monologue" architecture, where the time-aligned text is present in the input. In autoregressive generation, however, the model must build both the text and the audio

from scratch. If the first tokens drift slightly from the reference distribution, the model moves into a region it has not seen in training, and errors grow cumulatively.

An observation that strengthens this explanation is visible in Table 5.18: as the loss decreases, the number of rows that produce *any* speech falls from 9 to 4 or 5. That is, as the model fits the objective better, it leans towards silence in independent generation. This behaviour is not consistent with a model that has merely "not learned enough"; it is consistent with a model that has learned to predict correctly in the presence of the reference input text and collapses in its absence.

Version v6.2 tried to compensate for exactly this: scheduled dropout of the text input raised the share of hidden reference tokens from 0.25 to 0.75, so that the model would learn to rely on its own predictions. This intervention kept the learning signal but did not transfer to valid generation. A likely explanation is that with 32 training samples and 200 steps, the budget available to relearn a stable generative behaviour is far below what is needed.

Explanation three: the English-centred base model. The base model is strongly English-centred. In the first generations it was observed that the output of intermediate checkpoints is *English* speech, not Persian. That is, the adapter could not shift the distribution of the generated language and affected only acoustic details. Version four's attempt to train the text embeddings selectively was a direct answer to this hypothesis, and it was not enough either.

Methodological conclusion. Regardless of which explanation contributes most, one practical conclusion is certain, and in our view it is the most important scientific contribution of this report:

> In low-rank adaptation of a speech-to-speech model to a new language, the teacher-forced validation loss is not a sufficient, or even a reliable, criterion for checkpoint selection. A runtime test of autoregressive generation must be part of the selection criterion, not a check after selection.

Had the project based its selection criterion on loss alone, the step-8000 checkpoint of version one, with the lowest validation loss and a significant improvement over the base model (1.7272 versus 3.5670 on held-out data), would have been reported as "a Persian speech-to-speech model", while that same checkpoint produced almost silent audio and no text in the official server.

6.2 Why the Word Error Rate Is Much Higher Than the Character Error Rate

In the re-recognition measure on the same 9 outputs of the mechanical panel with the Qwen2.5-0.5B responder, the character error rate is 7.14% and the word error rate 30.41%; that is, the word error rate is more than four times higher. At first sight this ratio looks worrying, but it is to a large extent a *definitional* effect in Persian.

Three mechanisms contribute to this gap. First, the **word boundary in Persian**. In the definition used, the ZWNJ counts as a word boundary. If the recognition service writes a compound word with a full space while the reference writes it with a ZWNJ, one word becomes two, and this single difference produces two word edits, while at the character level there is only one character of difference. Second, **inconsistent written forms**. Many Persian words have several accepted written forms. Third, the **length effect**. The mean word length in the data of this panel is about four characters; under these conditions, every one-character error automatically becomes a complete word error.

There is supporting evidence for this interpretation in the data: in all 9 rows, the number of characters in the hypothesis and the reference is almost the same (for example 52 and 52 in the first row), while the number of words differs (11 and 12). That is, the character content is almost preserved and the difference is mostly in segmentation.

Nevertheless, two limitations of this interpretation must be stated explicitly. First, part of the error can come from wrong pronunciation by the speech engine rather than from segmentation; separating these two shares is not possible without human listening. Second, the recognition service takes part both in the system and in the measurement; their shared error can shift the result in either direction. A character error rate of 7.14% can therefore not be translated into "the generated speech is fully understandable for humans".

6.3 Why Adapting the Responder Lowered the Loss but Did Not Improve Quality

The experiment of Section 5.5.3 has an apparent contradiction: low-rank adaptation of the responder reduced the development loss by 31.18% but did not beat the baseline in the semantic evaluation. The answer to this contradiction lies in the nature of the training *target*.

The training target of this adapter was to reproduce the "next speaker's turn" in the conversational data. But the next speaker's turn in a natural conversation programme is not an assistant's answer to a question. That turn can be a short affirmation, a change of topic, a joke, a counter-question or the continuation of the speaker's own story. Trained on this target, the model learns the *style* of natural turn-taking better and the loss decreases, but it does not learn instruction following and relevant answering, and may even damage them.

The direct and decisive evidence for this interpretation is in the final semantic test: the "automatically aligned next-speaker reference" arm received the lowest relevance score, 0.950, lower even than the 0.5-billion-parameter baseline (1.425) (Figure 5.2). That is, the automatic judge rated as irrelevant the very target the adapter had tried to imitate.

The practical consequence of this observation for later research is clear: automatically aligned turns from natural conversation are *valuable* material for learning speech timing, turn-taking and interaction phenomena, but without a stage of selection and rewriting they are a *weak* target for training instruction-following answering. This point probably also explains part of the failure of the direct track, because the direct model was trained on the same pairs.

6.4 A Structural Conflict Between the Requirements

The analysis of Table 3.1 reveals a structural conflict in the project definition that is worth recording. Requirement R2 asks for full-duplex capability and requirement R3 for a latency of at most 500 ms, while requirement R6 requires running on a GPU with 12–24 GB of memory.

The models with *native* full-duplex capability, which model the user's stream and their own at the same time, did not fit within the target memory range at the time of this project. The base model of this project's direct track used up to 28.640 GB of memory in training. On the other hand, the first cascade architecture fitted within the target memory range, but because the whole reply was built before playback, its latency (a median of 5584.7 ms for full reply generation, Table 5.4) was far from the target.

The project resolved this conflict by making the cascade streaming: speculative end-of-speech detection, streaming token generation, clause chunking and synthesizing each clause as soon as it is complete. The result was a mean of 372 and a 90th percentile of 431 ms in the browser loopback test, with the GPU side at 11.1 GiB; full duplex was achieved with an external two-stage controller instead of a native model. The cost of this solution is the loss of the *native* capabilities of full-duplex models, such as continuous turn prediction and the assistant's own backchannels.

6.5 The Value and Boundary of the Automatic Semantic Result

The result of the final semantic test is positive and passed all seven predefined conditions. The design of this test has several methodological strengths: a qualification stage separate from the final test, row indices and thresholds locked before running, blinding and randomization of the arm order, deterministic decoding of the judge, and full validity of all 240 calls with a repetition agreement rate of 1.00.

Nevertheless, three limitations separate it from a human result. First, the **same family of judge and candidate**. The judge and the candidate are of the same model family; the literature on automatic evaluation has shown that judges favour the output of models of their own family [37]. Second, an **uncalibrated rubric**. The scoring rubric is not calibrated against the scores of Persian speakers; we therefore do not know what level of real user satisfaction a score of 3.150 out of 4 corresponds to. Third, the **semantic scope**. This test measures only relevance and coherence and says nothing about the factual correctness of the content, the safety of the reply or the naturalness of the speech.

Within these boundaries, the correct interpretation is: the cascade produces replies that an automatic judge systematically, and by a wide margin (+1.725 in relevance), rates as more relevant than the baseline. This is valid engineering evidence that the system works, not a claim of perceptual quality.

6.6 Limitations

The limitations of this project, stated explicitly and listed, are:

1. **Target hardware.** The evaluation was not run on a physical 12–24 GB GPU. The memory of the whole system was measured by limiting the language model server (11.1 GiB), but the compute speed is the H100's; latency on a consumer card will probably be higher.

2. **Automated loopback testing.** Latency and interruptions were measured with a headless browser and a fake microphone, not with a human user, a physical microphone and loudspeaker and real echo; in the simulation the echo was modelled as attenuated and delayed speech. In the final run all turns of the main panel are below 500 ms, but the maximum for simulated elderly speech (546) is above it, and with the browser's output latency 2% of the turns of the main panel exceed 500.

3. **Human study.** Only a small listening study (5–10 participants, remote, based on recordings) was designed, and it was not conducted because no participants were available. Live interaction with older adults was not measured.

4. **Automatic labels and judging.** The interruption labels come from extraction rules, Gemini and a list of listener tokens, reply quality was measured with an LLM judge and speech with an automatic recognizer; none of these is a human label. The first two responder judges agree exactly only 44.6% of the time; the two judges of the third comparison, from families with no candidate, rank the configurations alike (rank correlation 0.88), but they are still models.

5. **Interruption detection method.** The project definition asks for interruption detection with energy, pitch and MFCC features; a classifier based on these features alone reached only 0.574 accuracy on real events, and the deployed controller relies on the speech recognizer, which brings the time to a complete stop to about half a second.

6. **Listening review of the data.** No row of the dataset is "human-verified".

7. **Direct track.** No experimental generation of Moshika and no Gemma speech-input adapter became eligible for deployment.

8. **Generalization.** The dataset is derived from four conversational channels, and the elderly user's voice in the tests is simulated (slowed machine speech), not recordings of older adults.

9. **Full reproduction.** The adapter tensors of versions two to five have been deleted, and only their hashes and results remain.

6.7 Ethical, Privacy and Legal Considerations

6.7.1 Provenance and Rights of the Data

The dataset is derived from an internal archive of Persian spoken content. Internal research use of these resources was recorded with the supervisor's approval. This approval is not a "licence to redistribute": the raw audio, the raw captions and their derivatives were not placed in the code repository and are not published in the project's delivery package either. The statement "publicly available data has a free licence for training or redistribution" is false and is not used in this report.

6.7.2 Privacy in the Human Study

The system has three data retention modes: keeping the audio file, keeping only a reduced feature vector, and keeping only metrics. The suggested default for the human study is the features-only mode, so that no reconstructable speech of a participant is stored. An automatic test also checks that no audio file is written in the metrics-only mode.

The principle of data minimization was also kept in the automatic evaluations: the result artifacts do not store transcript text, reply text or judge explanations in plain text, and keep only hashes and aggregate values. The retrospective descriptive analysis of Section 5.3 was also explicitly run without reading plain text.

6.7.3 Licence and Its Boundaries

The source code and documentation belonging to this project are published under the Apache-2.0 licence. This licence does *not* cover the following, and the separate terms of each still apply: third-party software components, the weights of language and speech models, the source dataset and its derivatives, trained adapters, restricted media and captions, and the materials of human-study participants. In particular, the speech engine used has a different licence, and its obligations remain separate.

6.7.4 Use and Misuse

A system that generates natural Persian speech can be misused. In its current form, two factors limit this risk: the output voice is a single, known synthesized voice and no voice cloning of individuals is implemented; and the text responder works with a locked prompt. Nevertheless, the project performed no content safety measurement, and the responder model can produce incorrect content; using this prototype in any setting where an incorrect answer has serious consequences is not permitted without a safety measurement layer.

6.8 Statements This Report Avoids

For transparency, the following eight statements **cannot be claimed** with the evidence of this project. Each statement is quoted to make clear that these are not claims of this report but an explicit list of things that **have not been said** and should not be concluded from this project:

- “Moshika was successfully fine-tuned for Persian.”
- “The system’s latency is below 500 ms.”
- “The official interruption detection accuracy is above 80%.”
- “The speech output is natural and fully understandable for humans.”
- “The system is suitable for use by older adults.”
- “The nine-row panel proves that the system generalizes to Persian speech.”
- “The source data have a free licence for redistribution or training.”
- “The dataset was labelled and verified by humans.”

Chapter 7

Conclusions and Future Work

7.1 Summary

The goal of this project was to design, implement and evaluate a full-duplex speech-to-speech prototype for Persian. Two tracks were pursued in parallel: adapting an end-to-end speech-to-speech model, and a modular cascade with full-duplex control. The outcome was asymmetric.

The **streaming cascade** reached a working prototype that operates with an open microphone: it detects the end of speech automatically and speculatively, generates and plays the reply as a stream, clause by clause, and reacts to the user's interruption during playback. In loopback testing with a real browser, the mean latency from end of speech to the assistant's first audio was 372 ms and its 90th percentile 431 ms, and none of the 201 turns went unanswered or exceeded 500 ms; the GPU side of the whole system used 11.1 GiB of memory. The 40-row final semantic test of the Qwen3-4B responder passed all seven automatic conditions (relevance 3.150 against 1.425 for the baseline).

Interruption detection reached the accuracy goal with a two-stage controller: one frame of speech louder than the expected echo ducks the assistant's voice, and then the speech recognizer, after setting aside listener tokens, function words and the assistant's own echo, decides to stop or continue. In a deployment simulation built from real recordings, an accuracy of 0.933 and an F1 of 0.880 with a median of 120 ms to the duck were obtained, and in a real browser an accuracy of 0.825 and an F1 of 0.817. The energy/pitch/MFCC acoustic classifier alone was limited to 0.574 on real podcast events; this negative result is the reason for the lexical design.

The **components** were re-selected on fresh panels under pre-registered protocols: the NeMo recognizer was the most accurate candidate within the latency budget on human transcripts, the Mana voice the most intelligible voice to two automatic listeners (against the other Piper

voices, ParsVoice-XTTS and the lab VITS model, even after optimizing its inference), and no replacement responder reached the replacement threshold.

The **direct track** did not reach a deployable model, but its main cause was found: Moshi's English tokenizer needs 6.6 tokens per Persian word, and 98.5% of turns exceed the one-token-per-frame budget. Removing this bottleneck with phoneme romanization doubled the best generation result (6/9 against 3/9) and turned silent output into audible speech, but the speech was still not intelligible. The experience of J-Moshi [42] confirms this diagnosis: bringing Moshi to Japanese also began with replacing the tokenizer, but then needed about 60,000 hours of Japanese dialogue pre-training, more than five hundred times the 108.6 hours of this project's data. The speech-input Gemma 4 model, even with a self-distillation adapter, was weaker than the cascade.

The **dataset** is 6,754 "user speech and response" pairs with 108.584 hours of audited audio and zero group leakage. Its labels are automatic and have no human labels.

7.2 Answer to the Research Question

The practical question of this project had two parts. Part one: with the budget of a bachelor's project, how far can one go in building a full-duplex Persian speech-to-speech system? The answer is that a streaming cascade with interruption control meets the proposal's numeric criteria for latency (mean and 90th percentile below 500 ms) and interruption detection (above 80%) in automated tests and fits in the memory of a 12 GB card; but an end-to-end Persian model was not achievable with the existing architectures and tokenizers within this budget.

Part two: what can be proven with auditable evidence? The answer is in Table 5.25. R1 and R4 are met; R2, R3 and R5 are met with automatic evidence (browser loopback and simulation), R3 by the proposal's criterion and, for the main panel, even by the every-turn reading, and R5 by a method other than the specified acoustic features; R6 is a declared deviation (memory was measured, speed on target hardware was not); and R7 is not met because the listening study could not be conducted. The methodological contribution of this report lies in this separation: for each measure the permitted claim boundary is stated, and negative results are reported as fully as positive ones.

7.3 Answers to the Research Questions of the Proposal

Question 1: low-cost adaptation of an existing speech-to-speech model. Low-rank adaptation of Moshika with a LoRA adapter on an H100 improved the training objective: on the held-out test, the perplexity of the text tokens went from 2.53 to 1.19 and that of the audio tokens from 13.98 to 4.71. But in none of the seven generations was intelligible Persian speech produced. The main cause is the tokenizer: the English tokenizer needs 6.6 tokens per Persian word, while the model produces at most one text token per 12.5 Hz frame. Phoneme romanization brought this load to 0.67 tokens per frame and doubled the best run (6/9), but it was not enough. Answer: not with "the least retraining cost"; adapting this architecture to Persian needs at least a text representation compatible with Persian and training beyond a small adapter.

Question 2: architectural changes needed for full duplex below 500 ms. In the cascade, these changes were necessary and sufficient: (1) speculative end-of-speech detection that starts the work before the end is confirmed and holds the audio until confirmation; (2) streaming token generation with a "Persian script only" constraint at decoding time; (3) clause chunking that builds the first fragment from two to four words and synthesizes each clause immediately; (4) a playback queue in the client and an audio-holding gate in the server, which together play the role of a "dual buffer"; and (5) two-stage interruption control: ducking within one frame and lexical confirmation that sets aside listener tokens and echo, together with keeping only the heard part of an interrupted reply in the history. With this design, the mean latency was 372 and the 90th percentile 431 ms, and the reply after an interruption stayed as relevant as the reply to a normal turn (Section 5.11). In the direct track, the change needed before any architectural change is the tokenizer.

Question 3: the trade-off between latency, size and quality. For recognition and synthesis size did not buy quality (Table 5.24). The two smallest recognizers fine-tuned on Persian (115 and 117 million parameters) were more accurate than every larger candidate but one, including a three-billion-parameter Persian fine-tune; the only more accurate model, itself fine-tuned on Persian, was more than ten times slower. The smallest synthesizer (15.7 million parameters) was both the most intelligible and the fastest, and the largest model, the 7.7-billion-parameter end-to-end model, did not reach intelligible speech. For the responder the answer depends on who judges. Under a Qwen judge and Gemini, responders of 9 to 36 billion parameters gave no gain over the best 4B (Table 5.10); under two judges from families with no candidate, a third comparison of 15 models and their quantized builds found Gemma 4 12B at four bits 1.2 to 1.75 points (of 10) better than the best 4B and 3.5 points better than the deployed responder, with a 39 ms first token and 10.9 GB of memory (Table 5.11). Larger models gained little more and

cost latency, and with the compared serving settings no better responder fit a 12 GiB card within the latency gate (a llama.cpp setting found afterwards brought Gemma 4 12B within both). In the live stack, however, this model asked the user to repeat when the recognizer garbled a follow-up, and its replies after late interruptions fell short of the pre-registered coherence rule in all three configurations tested, so it was not qualified for deployment. The best trade-off point for real-time Persian is small recognizers and synthesizers specialized for Persian with the largest responder the memory profile allows at four bits, provided that it is checked in the live stack, where recognition errors change what a good reply is.

Question 4: the perception of older adults. The instrument and protocol of the listening study are ready and pre-registered, but no participants were available, so the study was not conducted and this report makes no claim about the perception of older adults.

7.4 Future Work

7.4.1 Completing the Evidence

1. **Target hardware and a physical path.** Running the same loopback protocol on a 12–24 GB card, and a test with a physical microphone and loudspeaker that includes real echo and the browser's echo cancellation.

2. **Human study.** Running the registered listening study with 5–10 participants (at least two aged over 60); and as a next step, a short live interaction session with older adults.

3. **Human labels.** A listening review of a sample of interruption events and dataset pairs by two independent reviewers, to validate the automatic Gemini labels and the extraction rules.

4. **Human judging of the responder.** The first two automatic judges disagreed considerably about the responders (Gemini rated the responders with the elderly prompt about one point better and the Qwen judge did not); two later judges from other families agree with each other and prefer the larger Gemma models. The replacement rule for the 12 GiB stack was registered before any participant could receive the listening study, and the configuration of Qwen3.5-4B with the elderly prompt has passed its deployment qualification; only the participants' preference, which could not be collected, is missing for it to replace the deployed responder. Whether people share the automatic judges' preference for the 24 GB

responder, and whether older users accept its requests to repeat after recognition errors, has not been measured.

7.4.2 Improving the System

1. **A responder that knows the recognizer can err.** The 24 GB responder asked the user to repeat after most garbled follow-ups, and the 4B responders guessed. Giving the responder the recognizer's confidence, or the instruction to answer the most likely request and confirm briefly, could keep the larger model's quality in the live stack; it needs its own pre-registered test.

2. **More accurate lexical confirmation.** The first phoneme of short backchannels is sometimes clipped («آره» as «کار»). Adding 200 ms of audio before the onset to the recognizer's input was tested and rejected: more interruptions were stopped (0.98 versus 0.95) but more backchannels and noise too, and accuracy fell (Section 5.7.3). Separating listener words from content words with a clipped onset needs labelled data and a learned model.

3. **End-of-speech detection for older adults.** The long pauses of slow speech sometimes close the turn before the end of the sentence (87 server turns for 80 planned turns). The system compensates when the user continues by stopping the reply. A semantic end-of-turn gate was tested in two rounds (Section 5.11): the best one, an LLM completeness judgement, split fewer utterances in the loopback test but held 15% of finished turns and pushed the 90th percentile above 500 ms. A gate that holds for less time, or a Persian end-of-turn model trained on turn ends labelled by people rather than on podcast speaker boundaries, is the next step.

4. **A more accurate recognizer within the same budget.** Seamless had a lower character error rate but more than four times the latency budget; distilling it into a CTC model or running it in streaming mode could raise accuracy without a latency cost.

7.4.3 The Direct Track

The tokenizer finding points to a clear path: a direct Persian model needs a tokenization that fits within the one-token-per-frame budget (phoneme-based or with a Persian vocabulary), pre-training on Persian dialogue at a scale close to J-Moshi's (tens of thousands of hours, for which no public Persian corpus exists), and a remedy for the gap between teacher forcing and free generation, such as scheduled input dropout at a larger scale or training on the model's own output.

The frozen-model approach of Freeze-Omni [2] and stabilizing Persian speech generation with plentiful single-speaker data such as ManaTTS [3] before dialogue training are also promising paths. For the speech-input model, training on more varied speech (not only podcasts) is needed, because self-distillation on one domain hurt outside that domain.

7.5 Closing Remarks

This project showed that a full-duplex Persian voice assistant with an open microphone, latency below half a second and reliable interruption detection can be built within the budget of a bachelor's project with open components; and it showed that an end-to-end Persian speech-to-speech model needs more than low-rank adaptation, including a tokenization suited to the language. The negative result of the direct track, together with its measured cause, is worth no less than the positive result of the cascade: offline loss reduction can be misleading, and a runtime generation test must be part of the selection criterion.

Bibliography

[1] A. Défossez, L. Mazaré, M. Orsini, A. Royer, P. Pérez, H. Jégou, E. Grave, and N. Zeghidour. Moshi: A speech-text foundation model for real-time dialogue. Technical report, Kyutai, 2024. arXiv:2410.00037.

[2] X. Wang, Y. Li, C. Fu, Y. Zhang, Y. Shen, L. Xie, K. Li, X. Sun, and L. Ma. Freeze-Omni: A smart and low latency speech-to-speech dialogue model with frozen LLM. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 63345–63354. PMLR, 2025.

[3] M. F. Qharabagh, Z. Dehghanian, and H. R. Rabiee. ManaTTS Persian: A recipe for creating TTS datasets for lower-resource languages. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 9177–9206, Albuquerque, New Mexico, 2025. Association for Computational Linguistics.

[4] J. Xu, Z. Guo, J. He, H. Hu, T. He, S. Bai, K. Chen, J. Wang, Y. Fan, K. Dang, B. Zhang, X. Wang, Y. Chu, and J. Lin. Qwen2.5-Omni technical report. *arXiv preprint arXiv:2503.20215*, 2025.

[5] Z. Xie and C. Wu. Mini-Omni2: Towards open-source GPT-4o with vision, speech and duplex capabilities. *arXiv preprint arXiv:2410.11190*, 2024.

[6] Q. Fang, Y. Zhou, S. Guo, S. Zhang, and Y. Feng. LLaMA-Omni 2: LLM-based real-time spoken chatbot with autoregressive streaming speech synthesis. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 18617–18629, Vienna, Austria, 2025. Association for Computational Linguistics.

[7] Kyutai Labs. Moshi-finetune: LoRA fine-tuning for Moshi. `https://github.com/kyutai-labs/moshi-finetune`, 2024.

[8] J. Sohn, N. S. Kim, and W. Sung. A statistical model-based voice activity detection. *IEEE Signal Processing Letters*, 6(1):1–3, 1999.

[9] A. de Cheveigné and H. Kawahara. YIN, a fundamental frequency estimator for speech and music. *The Journal of the Acoustical Society of America*, 111(4):1917–1930, 2002.

[10] S. B. Davis and P. Mermelstein. Comparison of parametric representations for monosyllabic word recognition in continuously spoken sentences. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, 28(4):357–366, 1980.

[11] A. Graves, S. Fernández, F. Gomez, and J. Schmidhuber. Connectionist temporal classification: Labelling unsegmented sequence data with recurrent neural networks. In *Proceedings of the 23rd International Conference on Machine Learning*, pages 369–376, 2006.

[12] A. Graves. Sequence transduction with recurrent neural networks. In *ICML Workshop on Representation Learning*, 2012. arXiv:1211.3711.

[13] A. Gulati, J. Qin, C.-C. Chiu, N. Parmar, Y. Zhang, J. Yu, W. Han, S. Wang, Z. Zhang, Y. Wu, and R. Pang. Conformer: Convolution-augmented transformer for speech recognition. In *Proceedings of Interspeech*, pages 5036–5040, 2020.

[14] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, pages 5998–6008, 2017.

[15] O. Kuchaiev, J. Li, H. Nguyen, O. Hrinchuk, R. Leary, B. Ginsburg, S. Kriman, S. Beliaev, V. Lavrukhin, J. Cook, and Others. NeMo: A toolkit for building AI applications using neural modules, 2019. arXiv:1909.09577.

[16] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever. Robust speech recognition via large-scale weak supervision. In *Proceedings of the 40th International Conference on Machine Learning*, pages 28492–28518, 2023.

[17] J. Kim, J. Kong, and J. Son. Conditional variational autoencoder with adversarial learning for end-to-end text-to-speech. In *Proceedings of the 38th International Conference on Machine Learning*, pages 5530–5540, 2021.

[18] Rhasspy Project. Piper: A fast, local neural text-to-speech system. https://github.com/rhasspy/piper, 2023.

[19] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, and Others. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*, volume 35, pages 27730–27744, 2022.

[20] M. Ranzato, S. Chopra, M. Auli, and W. Zaremba. Sequence level training with recurrent neural networks. In *Proceedings of the 4th International Conference on Learning Representations*, 2016.

[21] S. Bengio, O. Vinyals, N. Jaitly, and N. Shazeer. Scheduled sampling for sequence prediction with recurrent neural networks. In *Advances in Neural Information Processing Systems*, volume 28, pages 1171–1179, 2015.

[22] N. Houlsby, A. Giurgiu, S. Jastrzebski, B. Morrone, Q. De Laroussilhe, A. Gesmundo, M. Attariyan, and S. Gelly. Parameter-efficient transfer learning for NLP. In *Proceedings of the 36th International Conference on Machine Learning*, pages 2790–2799, 2019.

[23] X. L. Li and P. Liang. Prefix-tuning: Optimizing continuous prompts for generation. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics*, pages 4582–4597, 2021.

[24] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen. LoRA: Low-rank adaptation of large language models. In *Proceedings of the 10th International Conference on Learning Representations*, 2022.

[25] N. Zeghidour, A. Luebs, A. Omran, J. Skoglund, and M. Tagliasacchi. SoundStream: An end-to-end neural audio codec. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 30:495–507, 2021.

[26] A. Défossez, J. Copet, G. Synnaeve, and Y. Adi. High fidelity neural audio compression. *Transactions on Machine Learning Research*, 2023.

[27] Z. Borsos, R. Marinier, D. Vincent, E. Kharitonov, O. Pietquin, M. Sharifi, D. Roblek, O. Teboul, D. Grangier, M. Tagliasacchi, and N. Zeghidour. AudioLM: A language modeling approach to audio generation. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 31:2523–2533, 2023.

[28] C. Wang, S. Chen, Y. Wu, Z. Zhang, L. Zhou, S. Liu, Z. Chen, Y. Liu, H. Wang, J. Li, L. He, S. Zhao, and F. Wei. Neural codec language models are zero-shot text to speech synthesizers. *arXiv preprint arXiv:2301.02111*, 2023.

[29] D. Lee, C. Kim, S. Kim, M. Cho, and W.-S. Han. Autoregressive image generation using residual quantization. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 11523–11532, 2022.

[30] H. Sacks, E. A. Schegloff, and G. Jefferson. A simplest systematics for the organization of turn-taking for conversation. *Language*, 50(4):696–735, 1974.

[31] G. Skantze. Turn-taking in conversational systems and human-robot interaction: A review. *Computer Speech & Language*, 67:101178, 2021.

[32] J. H. Friedman. Greedy function approximation: A gradient boosting machine. *Annals of Statistics*, 29(5):1189–1232, 2001.

[33] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and É. Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[34] V. I. Levenshtein. Binary codes capable of correcting deletions, insertions, and reversals. *Soviet Physics Doklady*, 10(8):707–710, 1966.

[35] E. B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927.

[36] B. Efron and R. J. Tibshirani. *An Introduction to the Bootstrap*. Chapman & Hall/CRC, 1994.

[37] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica. Judging LLM-as-a-judge with MT-Bench and chatbot arena. In *Advances in Neural Information Processing Systems*, volume 36, pages 46595–46623, 2023.

[38] D. Zhang, S. Li, X. Zhang, J. Zhan, P. Wang, Y. Zhou, and X. Qiu. SpeechGPT: Empowering large language models with intrinsic cross-modal conversational abilities. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 15757–15773, 2023.

[39] Q. Fang, S. Guo, Y. Zhou, Z. Ma, S. Zhang, and Y. Feng. LLaMA-Omni: Seamless speech interaction with large language models. In *Proceedings of the 13th International Conference on Learning Representations*, 2025.

[40] Z. Du, Y. Wang, Q. Chen, X. Shi, X. Lv, T. Zhao, Z. Gao, Y. Yang, C. Gao, H. Wang, F. Yu, H. Liu, Z. Sheng, Y. Gu, C. Deng, W. Wang, S. Zhang, Z. Yan, and J. Zhou. CosyVoice 2: Scalable streaming speech synthesis with large language models. *arXiv preprint arXiv:2412.10117*, 2024.

[41] A. Zeng, Z. Du, M. Liu, K. Wang, S. Jiang, L. Zhao, Y. Dong, and J. Tang. GLM-4-Voice: Towards intelligent and human-like end-to-end spoken chatbot. *arXiv preprint arXiv:2412.02612*, 2024.

[42] A. Ohashi, S. Iizuka, J. Jiang, and R. Higashinaka. Towards a Japanese full-duplex spoken dialogue system. In *Proceedings of Interspeech 2025*, 2025. arXiv:2506.02979.

[43] T. A. Nguyen, E. Kharitonov, J. Copet, Y. Adi, W.-N. Hsu, A. Elkahky, P. Tomasello, R. Algayres, B. Sagot, A. Mohamed, and E. Dupoux. Generative spoken dialogue language modeling. *Transactions of the Association for Computational Linguistics*, 11:250–266, 2023.

[44] R. Roy, J. Raiman, S.-g. Lee, T.-D. Ene, R. Kirby, S. Kim, J. Kim, and B. Catanzaro. PersonaPlex: Voice and role control for full duplex conversational speech models, 2026. arXiv:2602.06053.

[45] R. Ardila, M. Branson, K. Davis, M. Kohler, J. Meyer, M. Henretty, R. Morais, L. Saunders, F. Tyers, and G. Weber. Common voice: A massively-multilingual speech corpus. In *Proceedings of the 12th Language Resources and Evaluation Conference*, pages 4218–4222, 2020.

[46] M. J. Ranjbar Kalahroodi, H. Faili, and A. Shakery. ParsVoice: A large-scale multi-speaker Persian speech corpus for text-to-speech synthesis, 2025. arXiv:2510.10774.

[47] A. N. Mohammadi. Corpus of conversational Persian transcripts. Linguistic Data Consortium, LDC2019T11, 2019.

[48] X. Chen, Y. Wu, Z. Wang, S. Liu, and J. Li. Developing real-time streaming transformer transducer for speech recognition on large-scale dataset. In *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2021. arXiv:2010.11395.

[49] V. Noroozi, S. Majumdar, A. Kumar, J. Balam, and B. Ginsburg. Stateful conformer with cache-based inference for streaming automatic speech recognition, 2023. arXiv:2312.17279.

[50] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis. Efficient streaming language models with attention sinks. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2309.17453.

[51] R. Vipperla, S. Renals, and J. Frankel. Ageing voices: The effect of changes in voice parameters on ASR performance. *EURASIP Journal on Audio, Speech, and Music Processing*, 2010:525783, 2010.

[52] A. Pradhan, A. Lazar, and L. Findlater. Use of intelligent voice assistants by older adults with low technology use. *ACM Transactions on Computer-Human Interaction*, 27(4):31:1–31:27, 2020.

[53] J. Kowalski, A. Jaskulska, K. Skorupska, K. Abramczuk, C. Biele, W. Kopeć, and K. Marasek. Older adults and voice interaction: A pilot study with Google Home. In *Extended Abstracts of the 2019 CHI Conference on Human Factors in Computing Systems*, pages LBW0187:1–LBW0187:6, 2019.

[54] H. Bredin, R. Yin, J. M. Coria, G. Gelly, P. Korshunov, M. Lavechin, D. Fustes, H. Titeux, W. Bouaziz, and M.-P. Gill. Pyannote.audio: Neural building blocks for speaker diarization. In *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing*, pages 7124–7128, 2020.

[55] T. J. Park, N. R. Koluguri, J. Balam, and B. Ginsburg. Multi-scale speaker diarization with dynamic scale weighting. In *Proceedings of Interspeech*, pages 5080–5084, 2022.

[56] D. Snyder, G. Chen, and D. Povey. MUSAN: A music, speech, and noise corpus, 2015. arXiv:1510.08484.

[57] J. Thiemann, N. Ito, and E. Vincent. The diverse environments multi-channel acoustic noise database (DEMAND): A database of multichannel environmental noise recordings. *Proceedings of Meetings on Acoustics*, 19:035081, 2013.

[58] T. Ko, V. Peddinti, D. Povey, M. L. Seltzer, and S. Khudanpur. A study on data augmentation of reverberant speech for robust speech recognition. In *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 5220–5224, 2017.

[59] International Telecommunication Union. Methods for subjective determination of transmission quality. *ITU-T Recommendation P.800*, 1996.

[60] A. Yang, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Li, D. Liu, F. Huang, H. Wei, H. Lin, J. Yang, J. Tu, J. Zhang, J. Yang, J. Zhou, J. Lin, K. Dang, and Others. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2025.

[61] M. McAuliffe, M. Socolof, S. Mihuc, M. Wagner, and M. Sonderegger. Montreal forced aligner: Trainable text-speech alignment using Kaldi. In *Proceedings of Interspeech 2017*, pages 498–502, 2017.

[62] A. Yang, A. Li, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Gao, C. Huang, C. Lv, and Others. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

[63] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, and Others. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32, pages 8024–8035, 2019.

[64] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz, and Others. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45, 2020.

[65] Daily. Smart Turn v3: Open-source semantic turn detection, 2025. Model `pipecat-ai/smart-turn-v3`, BSD-2-Clause.

Appendix A

Implementation and Reproduction Details

This appendix collects details that are needed to reproduce the results but did not fit in the main text: the repository layout, the exact hyperparameters of the experiments, the reproduction commands and the map of evidence artifacts.

A.1 Repository Layout

Table A.1 shows the top-level layout of the code repository.

A.2 Hyperparameters of the Direct-Track Experiments

Table A.2 shows the exact hyperparameters of the seven experimental generations. All runs used the AdamW optimizer and gradient checkpointing.

Versions v4 and v5 differ only in the first-codebook loss weight: 100 against 10. In addition to the hyperparameters above, version v6.2 had scheduled text-input dropout from 0.25 to 0.75 and an audio loss weight of 0.1, and was trained and evaluated on the same 32 samples. Version v7a was trained on the same 32 samples, with the inner-monologue text as a phoneme romanization and word-level timing.

A.3 Hyperparameters of the Detector and the Responder

Table A.3 collects the configuration of the interruption detection module and the text responder adapter.

Table A.1: Top-level layout of the project's code repository

Path	Contents
`src/thesis_s2s/runtime/`	cascade, full-duplex server, text-to-speech, session recording
`src/thesis_s2s/bargein/`	feature extraction, detector, playback controller, evaluation on recorded data
`src/thesis_s2s/data/`	dataset pipeline, speaker diarization, pair extraction, export and audit
`src/thesis_s2s/eval/`	measures, comparison tests and the fail-closed evidence aggregator
`src/thesis_s2s/model/`	the early experimental artifact, for historical reference only
`configs/`	configuration of the data, detector, measures and the seven direct-track generations
`scripts/`	preflight, training, re-evaluation, checkpoint selection and locked evaluations
`docs/`	predefined protocols, the review waiver document and recorded decisions
`results/`	write-once result artifacts
`tests/`	automated test suite
`third_party/`	lock file of upstream repositories and models

A.4 Reproduction Commands

The following commands reproduce the main stages. The `PYTHONPATH` variable must point to the `src` directory; the subcommand of each stage is listed in Table A.4.

The locked evaluations run as separate scripts in the `scripts` directory; Table A.5 lists them.

These scripts operate on write-once artifacts and must not be used to reopen a final test or revise a finalized selection. The figures of this report were generated from these artifacts with `scripts/make_thesis_figures.py`.

A.5 Map of Evidence Artifacts

Table A.6 names, for each claim of this report, its reference artifact.

Table A.2: Hyperparameters of the seven generations of low-rank adaptation in the direct track

Version	Max steps	Window (s)	Rank	Seed	Peak memory (GB)	Trained embeddings
v1	8000	20	64	20260823	22.920	upstream key
v2	2000	100	64	20260827	–	upstream key
v3	500	100	128	20260827	26.504	none
v4	500	100	128	20260827	28.640	two text embeddings
v5	500	100	128	20260827	28.640	two text embeddings
v6.2	200	12	64	20260901	16.577	three text tensors
v7a	200	12	64	20260901	22.660	three text tensors

A.6 Runtime Environment

Table A.7 shows the main components of the runtime environment and the identifiers of the models used.

The exact revision and weight hash of each model are recorded in the lock file of upstream repositories and repeated in each result artifact, so that the claim “this is exactly the model that was used” is auditable.

A.7 Building This Report

The Persian original of this report is typeset with XeLaTeX and the XePersian package; this English edition uses XeLaTeX with polyglossia, so that the Persian examples in the text are typeset in the font of the Persian edition. To reproduce both, running `./build.sh everything` in the report directory is enough (`./build.sh en-all` for the English editions only); the script runs the typesetting and bibliography chain in an isolated container, so no local TeX installation is needed.

Two technical points corrected while preparing the report may be useful to later users of the same template: first, the name of the localization option of XePersian has changed in recent versions and the previous option is no longer accepted; second, the right-to-left table patch of the bidi package is not compatible with the current release of the array package, and its dated release must be requested. Both corrections are recorded with comments in `styles/common.tex` (and the second also in `styles/english.tex`, because polyglossia loads bidi for Persian).

A third point concerns the typesetting of numbers in the Persian edition. In right-to-left text, the bidirectional algorithm splits a decimal number at the separator into two parts and reverses their order; as a result the value 3.150 would be displayed as “150.3”. For this reason, in the Persian edition every measured quantity is placed in an explicit left-to-right run, while the structural numbering (the numbers of chapters, sections, pages, tables and figures) keeps the automatic Persian digits.

Table A.3: Configuration of the interruption detection module and the text responder adapter

Parameter	Value
Interruption detection module	
Sampling rate	16000 Hz
Frame length	20 ms
Context window length	400 ms
Number of MFCCs	13 with deltas
Fundamental frequency search range	60–400 Hz
Number of trees	200
Maximum tree depth	4
Learning rate	0.08
Minimum samples per leaf	8
Decision horizon after the acoustic onset	120 ms
Selected decision threshold	0.70
Inference time per step on the CPU	7.931 ms
Text responder adapter	
Rank and scale factor	16 and 32
Dropout	0.05
Training epochs	2
Random seed	20260906
Eligible training rows	2576 of 6419
Eligible development rows	12 of 131
Response-token loss of the base model	3.723
Loss after epochs one and two	2.643 and 2.562
Reduction relative to the base model	31.18%

Table A.4: Reproduction commands of the project's main stages

Stage	Subcommand
Corpus selection plan	`plan-conversation-corpus`
Window reconstruction	`prepare-conversation-episodes`
Audit of prepared windows	`audit-prepared-episodes`
Speaker diarization	`diarize-conversation-episodes`
Audit of diarization	`audit-diarized-episodes`
Noise-condition labels	`annotate-conversation-noise`
Corpus yield estimate	`estimate-conversation-yield`
Building response pairs	`build-conversations`
Audit of pairs	`audit-conversations`
Stereo training export	`export-moshi-data`
Audit of the export	`audit-moshi-data`
Running the system	`serve`
Aggregate evidence status	`evidence-status`
Hardware preflight	`gpu-preflight`

Table A.5: Locked evaluation scripts

Evaluation	Script
Mechanical panel of the cascade	`evaluate_cascade_validation.py`
Re-recognition measure	`evaluate_cascade_intelligibility.py`
Two-stage semantic test	`evaluate_qwen4b_responder_v2.py`
Responder comparison version three	`evaluate_responder_v3.py`
Requests to repeat after interruption	`count_clarification_requests.py`
llama.cpp first-token diagnosis	`diagnose_llamacpp_first_token.py`
Interruption detector approximation	`evaluate_recorded_bargein_proxy.py`
Generating the figures of this report	`make_thesis_figures.py`

Table A.6: Map of claims and reference artifacts. All paths are relative to the `results` directory.

Claim	Reference artifact
Aggregate status and acceptance conditions	`eval/EVIDENCE_STATUS.json`
Readable summary	`eval/SUMMARY.md`
Mechanical panel of the cascade	`eval/cascade_real_service_validation_panel.json`
Retrospective descriptive analysis	`eval/cascade_validation_descriptive_analysis.json`
Intelligibility approximation	`eval/cascade_intelligibility_proxy.json`
Semantic qualification stage	`eval/qwen4b_responder_v2_development_proxy.json`
Final semantic test	`eval/qwen4b_responder_v2_final_test_proxy.json`
Responder adaptation	`training/responder_lora_v1.json`
Responder comparison version three	`eval/responder_v3_result.json`
Qualification of the 24 GB responder	`eval/responder_v3_24gb_qualification.json`
Requests to repeat (exploratory)	`eval/responder_v3_repeat_requests.json`
Post-hoc 12 GiB checks	`eval/responder_v3_12gib_*_qualification.json`
Next models in 12 GiB (exploratory)	`eval/responder_v3_12gib_exploratory.json`
llama.cpp diagnosis (exploratory)	`eval/responder_v3_llamacpp_first_token_diagnosis.json`
Interruption detector approximation	`eval/interrupt_recorded_proxy.json`
Synthetic detector test	`eval/interrupt_bench.json`
Dataset audit	`conversation_audit.json`
Training export report	`moshi_export_report.json`
Checkpoint selection of version one	`moshi_checkpoint_selection.json`
Held-out loss of version one	`moshi_heldout_model_eval.json`
Selection and runtime of version two	`moshi_v2_checkpoint_selection.json`
Result of version v6.2	`moshi_v6_text_dropout_reevaluation.json`
Runtime of version v6.2	`moshi_v6_text_dropout_runtime.json`
Server-side latency	`eval/latency_bench.json`
Human study	`eval/human_study.json`
Final repository audit	`release/final_audit.json`

Table A.7: Main components of the runtime environment and the models used

Component	Identifier
Text responder of the final system	Qwen3-4B-Instruct-2507 cdbee75f17c01a7cc42f958dc650907174af0554
Responder of the 9/9 mechanical panel and re-recognition	Qwen2.5-0.5B-Instruct 7ae557604adf67be50417f59c2c2f167def9a775
Responder baseline in the semantic test	Qwen2.5-0.5B-Instruct
Responder of the 24 GB preset	Gemma-4-12B-it-QAT-W4A16 1d2c2d7f2466070e69d6fb3fd5ce9a7d75f2f6ee
Base model of the direct track	Moshika, seven billion parameters
Audio codec of the direct track	Mimi
Speech recognition	hybrid RNNT-CTC model of the Conformer family
Text-to-speech	Piper with a Persian voice
Judge of the semantic test	Qwen3.8-27B-FP8 017b9c7af6b5689d5dd426a76e0bc077eb5ca20a
Training and evaluation hardware	NVIDIA H100 NVL, 93.09 GB

چکیده

هدف این پروژه، طراحی، پیاده‌سازی و ارزیابی یک نمونه اولیه سامانه گفتاربه‌گفتار فارسی تمام‌دوطرفه است؛ یعنی میکروفون کاربر در هنگام پخش پاسخ فعال می‌ماند و سامانه به وقفه کاربر واکنش نشان می‌دهد. دو مسیر بررسی شد: تطبیق مدل سرتاسری Moshika با داده فارسی، و یک سامانه آبشاری جریانی شامل بازشناسی گفتار NeMo، پاسخ‌گوی Qwen3-4B-Instruct-2507 و گفتارساز Piper. از آرشیو گفتاری فارسی یک مجموعه داده مکالمه با 6,754 جفت و 108.584 ساعت صوت با تفکیک گروهی بر پایه نشست ساخته شد.

سامانه آبشاری نهایی پایان گفتار را خودکار و پیش‌دستانه تشخیص می‌دهد، پاسخ را جریانی و بندبه‌بند می‌سازد، و وقفه را در دو مرحله مدیریت می‌کند: کم‌کردن صدا پس از یک قاب گفتار بلندتر از پژواک مورد انتظار دستیار، و تأیید واژگانی با بازشناس پس از کنارگذاشتن واژه‌های شنونده و پژواک خود دستیار. در آزمون حلقه‌ای با مرورگر واقعی، میانگین تأخیر پایان گفتار تا نخستین صدای پاسخ 372 و صدک نود آن 431 میلی‌ثانیه بود، همه نوبت‌ها زیر 500 میلی‌ثانیه، و بخش گرافیکی سامانه 11.1 GiB حافظه گرفت. تشخیص وقفه در شبیه‌سازی استقرار از ضبط‌های واقعی به دقت 0.933 و 0.880 F1 با میانه 120 میلی‌ثانیه تا کم‌کردن صدا، و در مرورگر به دقت 0.825 و 0.817 F1 رسید؛ طبقه‌بند صوتی انرژی/زیروبمی/MFCC به‌تنهایی روی رخدادهای واقعی 0.574 گرفت. پاسخ‌گو در آزمون معنایی نهایی 40 ردیفی از هر هفت شرط خودکار گذشت (مرتبط‌بودن 3.150 در برابر 1.425). اجزا روی پنل‌های تازه دوباره برگزیده شدند، از جمله بازشناس روی رونوشت انسانی و گفتارساز با دو شنونده خودکار. یک مقایسه پیش‌ثبت‌شده از 15 مدل پاسخ‌گو و ساخت‌های کوانتیده آن‌ها، با داوری دو مدل از خانواده‌هایی بدون نامزد، Gemma 4 12B چهاربیتی را برای کارت 24 گیگابایتی برگزید (1.2 تا 1.75 امتیاز از 10 بالاتر از بهترین مدل چهارمیلیاردی، با 39 میلی‌ثانیه تا نخستین توکن)؛ در سامانه زنده از مرزهای حافظه و تأخیر گذشت اما پس از وقفه‌های دیررس به قاعده انسجام نرسید، پس پیش‌تنظیمی اختیاری می‌ماند. برای کارت 12 گیگابایتی، بررسی‌های پسین همین مدل (از جمله تنظیمی در llama.cpp که نخستین توکن را از حدود 180 به 25 میلی‌ثانیه رساند) نیز به همان قاعده انسجام نرسیدند.

مسیر مستقیم به مدل قابل استقرار نرسید؛ نشان داده شد که توکن‌ساز انگلیسی Moshi برای هر واژه فارسی 6.6 نشانه می‌خواهد و از بودجه یک نشانه در قاب فراتر می‌رود. لاتین‌نویسی واجی بهترین نتیجه را دو برابر کرد اما گفتار هنوز فهم‌پذیر نبود، و مدل ورودی گفتاری Gemma 4 نیز از سامانه آبشاری ضعیف‌تر بود. شواهد تأخیر و وقفه خودکار و روی H100 است (حافظه یک کارت 12 گیگابایتی رعایت شد، سرعت آن نه) و برچسب‌ها خودکارند؛ مطالعه شنیداری انسانی به دلیل نبود شرکت‌کننده انجام نشد.

کلیدواژه‌ها: گفتاربه‌گفتار، تمام‌دوطرفه، تشخیص وقفه، زبان فارسی، مدل زبانی بزرگ، سامانه آبشاری جریانی، توکن‌ساز، تطبیق کم‌پارامتر